

Resumen

Comprender cómo funcionan los grandes modelos de lenguaje (LLM) como GPT, LLAMA y otros similares es clave para aprovechar al máximo la inteligencia artificial moderna. Estos modelos usan procesos matemáticos avanzados y redes neuronales complejas que hacen posible analizar y generar lenguaje con alta precisión.

¿Qué significa tokenizar el lenguaje?

Tokenizar significa dividir el lenguaje humano en pequeñas unidades llamadas tokens, ya sean palabras, sílabas o letras. Aunque parezca infinito, en realidad, el lenguaje posee un número limitado de estas pequeñas unidades:

- El inglés, por ejemplo, utiliza aproximadamente 50,000 tokens.
- Los grandes modelos (como GPT-4) pueden abarcar hasta 256,000 tokens diferentes.

Estos tokens se organizan en espacios multidimensionales según su relación con otras palabras. Palabras con conceptos similares permanecen cercanas unas a otras en estos espacios.

¿Cómo funciona el sistema de vectores en LLM?

Cada palabra se representa mediante un vector numérico, que indica su relación con otras palabras. Por ejemplo:

- Vector mujer está muy cerca al vector hombre en términos similares al vector reina y rey por la dimensión de género.
- Palabras como caminé y caminar se sitúan próximas por compartir el vector de tiempo verbal.

Este método permite operaciones matemáticas entre palabras:

- Mamá menos género equivale a pariente.
- Regente más mujer equivale a reina.

¿Qué papel juegan las redes neuronales?

Las redes neuronales detectan patrones en la manera en la que las palabras (tokens) aparecen relacionadas en textos reales (llamados corpus de datos).

- Un 70% del corpus es para entrenar la red y un 30%, para probar su efectividad.
- Las redes neuronales poseen capas ocultas que ajustan constantemente valores para reconocer patrones en textos.

Gracias a estos patrones, aprenden cómo las palabras suelen combinarse naturalmente (por ejemplo: después de "yo" generalmente viene "amo") y permiten generar textos coherentes de forma autónoma.

¿Qué es la atención y cómo mejora la predicción del lenguaje?

El *modelo de atención* ayuda a decidir qué palabras anteriores son relevantes para predecir una palabra posterior. Se basa en:

- **Consulta (query)**: el último token escrito (ejemplo: "perro").
- **Llave (key)**: palabras relacionadas cerca de este último token ("gato", "maulla").
- **Valor (value)**: una ecuación matemática que usa esta cercanía para calcular probabilidades y decidir la próxima palabra posible (en este caso, "ladra").

Así, no se analizan todas las palabras, solo aquellas más relevantes, optimizando la eficiencia del modelo.

¿Cómo entrenan y ajustan los modelos para comportarse como Chats?

Inicialmente, estos modelos solo autocompletaban textos, no sostenían conversaciones. El proceso *RLHF (Reinforcement Learning with Human Feedback)* permitió transformar estos modelos en verdaderos chats al recompensarlos para que interactuaran y respondieran adecuadamente.

- OpenAI, por ejemplo, empleó a miles de personas para enseñar al modelo cómo realizar conversaciones naturales y eficaces.
- Este entrenamiento les permite adaptarse para distintas respuestas, no siempre eligiendo la opción más probable, impulsando la creatividad.

Te invitamos a explorar más sobre estos fascinantes temas y a comentar cualquier duda que te surja sobre los modelos de lenguaje o sobre inteligencia artificial.

Intro to Large Language Models by Andrej Karpathy (founding member OpenAI 2015 and ex Director of AI Tesla 2017) https://www.youtube.com/watch?v=zjkBMFhNj_g

Fuente: https://www.youtube.com/watch?v=zjkBMFhNj_g, <https://leerob.com/ai>

El query en el modelo de atención no necesariamente es solo la última palabra del prompt, sino que es un vector que representa la consulta que se hace sobre el contexto completo. Sin embargo, en el caso específico del proceso de generación de texto, el último token o palabra del prompt se utiliza como referencia para determinar qué palabras cercanas en el contexto son relevantes para predecir la siguiente palabra. Así, aunque el último token es fundamental, el query puede involucrar otros elementos del contexto para hacer una elección más informada.

La clase se centra en el funcionamiento de los grandes modelos de lenguaje (LLMs). Se abordan conceptos como tokenización, embeddings, y la arquitectura de Transformers. Se explica cómo se entrenan estos modelos, dividiendo el lenguaje en tokens y evaluando la correlación entre ellos en un espacio multidimensional. Se presenta el modelo de atención, que permite identificar palabras significativas en el contexto. Finalmente, se destaca la importancia del aprendizaje reforzado con retroalimentación humana (RLHF) para mejorar la interacción de los modelos como chatbots.

1. Cómo funciona nuestra atención

Cuando escuchamos la frase “el gato maúlla y el perro...”, la mayoría completa con “ladra”. ¿Por qué? Porque nuestro cerebro presta atención a ciertas palabras clave y deduce lo más probable. Esto es lo que se llama **atención**.

2. ¿Qué hacen los modelos de lenguaje?

Los modelos de lenguaje como GPT-4 funcionan prestando atención a ciertas palabras para predecir otras. Para lograrlo:

- Se toma todo el lenguaje humano (libros, redes, correos, Wikipedia, etc.).
- Se **rompe en pedacitos** llamados **tokens** (pueden ser palabras, sílabas o letras).
- Por ejemplo, la palabra "satisfacción" puede dividirse en "sat", "is", "f", "acción".

3. ¿Qué son los tokens y cómo se usan?

- Hay una cantidad limitada de tokens. Por ejemplo, en inglés suelen usarse unos 50,000.
- Los modelos más avanzados pueden usar hasta 256,000 tokens.

Cada token se representa como un vector en un espacio con muchas dimensiones, lo que permite:

- Medir **similitud** entre palabras. Ej.: "gato", "perro" y "lobo" están cerca porque son animales.
- Detectar **relaciones** como:
 - "rey" → "reina" es como "hombre" → "mujer"
 - "Italia" → "Roma" es como "Colombia" → "Bogotá"

4. ¿Qué hacen las redes neuronales?

Las redes neuronales:

- Se entrenan con un **70 %** de datos, y se validan con el **30 %** restante.
- Detectan patrones en los tokens (palabras, sílabas, letras).
- Están formadas por:
 - Una **capa de entrada** (los tokens).
 - Varias **capas ocultas** (donde se encuentran los patrones).
 - Una **capa de salida** (el token predicho).

5. ¿Cómo predicen la siguiente palabra?

- La red aprende que después de “yo” puede venir “amo”, o que “mi mamá” puede seguirse con “me”.

- Esta predicción depende de miles de millones de **parámetros** ajustados matemáticamente.
- Esta tecnología existe desde los años 50, pero solo recientemente hay suficiente capacidad computacional para entrenarla.

6. El modelo de atención

El modelo de atención se basa en tres componentes:

- **Query (consulta):** palabra actual.
- **Key (llave):** palabras anteriores relevantes.
- **Value (valor):** información matemática de esas palabras.

Ejemplo:

- En “el gato maúlla y el perro...”, el modelo detecta que “maúlla” y “gato” son claves para predecir qué sigue después de “perro”.

7. ¿Cómo se elige la palabra final?

- Se calcula una **probabilidad** para cada posible palabra siguiente.
- Normalmente se elige la de mayor probabilidad.
- Pero para generar respuestas creativas, los modelos usan una función llamada **temperatura**, que permite elegir a veces la segunda o tercera opción.

8. ¿Cómo aprendieron los modelos a conversar?

- Al principio, los modelos solo completaban texto.
- OpenAI entrenó a sus modelos con ayuda de 6000 personas que:
 - Premiaran respuestas de estilo conversacional.
 - Corrigieran cuando el modelo no se comportaba como un chat.

Este proceso se llama **RLHF (Aprendizaje Reforzado con Retroalimentación Humana)**.

Aquí tienes una **introducción integral y bien estructurada** sobre los temas que mencionaste, explicados con el rigor de un profesor de ingeniería de software y lógica de programación. Voy a organizar los conceptos en bloques, incidiendo en fundamentos esenciales y apoyándome en fuentes actualizadas:

1. Tokenización y Embeddings

- **Tokenización:** convierte texto en unidades discretas llamadas *tokens* (pueden ser palabras, subpalabras o caracteres). Por ejemplo, los modelos GPT usan tokenización por pares de bytes, donde un token representa entre ~0.75 palabras ([Wikipedia](#)).
 - **Embeddings:** cada token se transforma en un vector numérico que codifica propiedades semánticas y sintácticas. Estos vectores permiten que palabras relacionadas estén cerca en el espacio vectorial ([Wikipedia](#), [Cohere](#)).
-

2. Estructura de las Redes Neuronales (Transformers)

- Los modelos lingüísticos modernos usan la arquitectura **Transformer**, basada en atención y capas feed-forward. No usan RNNs ni convoluciones ([dlsi.ua.es](#)).
 - Su diseño puede dividirse en:
 - **Codificador (encoder):** procesa la secuencia de entrada.
 - **Decodificador (decoder):** genera la salida, usando información del encoder ([dlsi.ua.es](#), [Wikipedia](#)).
-

3. Atención: Mecanismo Central

- El **mecanismo de atención** calcula cuán relevante es cada token del input respecto a otro. Asigna pesos "blandos" (soft weights) a cada componente, mejorando la eficiencia y precisión ([Wikipedia](#)).
- En Transformers, cada token genera tres proyecciones lineales:
 - **Query (Q)** – ¿qué busca este token?
 - **Key (K)** – ¿cómo se define cada token?
 - **Value (V)** – ¿qué aporta cada token?
Luego, se calcula $\text{softmax}(QK^T / \sqrt{d}) \cdot V$, donde la división evita valores demasiado grandes en los productos escalares ([dlsi.ua.es](#)).

- También se usa **multi-head attention**, es decir, múltiples cabezas atendiendo a diferentes aspectos contextuales (epichka.com).
-

4. Positional Encoding

Los Transformers no captan orden natural, así que se añade una codificación posicional sinusoidal al embedding de cada token, lo que les permite interpretar la posición de cada palabra ([Wikipedia](https://es.wikipedia.org/wiki/Positional_encoding)).

5. Predicción de Tokens: Logits, Softmax y Temperatura

- El modelo genera un vector de **logits** (valores sin normalizar), que luego se convierten a probabilidades usando softmax (programmer.ie).
 - **Temperatura (T)** controla la aleatoriedad:
 - **T baja** → distribución más concentrada (más determinista).
 - **T alta** → distribución más plana (más creativa, variada) (medium.com).
-

6. Tamaño del Modelo, Parámetros, Datos

- Los **LLMs** son redes con miles de millones de parámetros entrenadas en enormes volúmenes de texto (billones de tokens) ([Wikipedia](https://es.wikipedia.org/wiki/Modelo_de_lenguaje_genuino)).
 - Su rendimiento crece con más parámetros, datos y poder computacional, según las **leyes de escala** ([Wikipedia](https://es.wikipedia.org/wiki/Leyes_de_escala)).
-

7. Pipeline de Entrenamiento y Evaluación

- Proceso típico:
 1. **Pre-entrenamiento** autoregresivo: predecir el siguiente token ([Wikipedia](#)).
 2. **Ajuste fino** (fine-tuning) con datos específicos o instrucciones.
 3. **RLHF** (Reinforcement Learning with Human Feedback): se entrena un modelo de recompensa basado en preferencias humanas, utilizado para ajustar el LLM mediante optimización de política (como PPO) ([Wikipedia](#)).
- **RLHF** mejora la alineación con valores humanos, aunque puede disminuir la diversidad de salidas ([arxiv.org](#)).

8. Resumen Tabulado

Concepto	Descripción clave
Tokens & Embeddings	Texto → unidades discretas → vectores numéricos
Transformers	Redes basadas en atención, sin RNNs
Atención (QKV, softmax)	Cálculo de relevancia y combinación contextual
Positional Encoding	Añade orden al texto
Logits, Softmax, Temperatura	Control de determinismo vs creatividad
Parámetros & Datos	Escala con tamaño y cómputo
Pipeline RLHF	Más alineación humana, menos diversidad

Si deseas, ahora puedo:

- **-xcl**: realizar un checklist con los puntos más importantes.
- **-xi**: interpretar resultados, como cómo cambia la probabilidad al modificar la temperatura.
- **-xj**: ofrecer ejemplos o analogías claras (como cómo QKV imita un diccionario difuso).

- o avanzar con lo que necesites para profundizar.

¿Te gustaría que desplegara alguno de estos enfoques?

Resumen

La inteligencia artificial (IA) y específicamente los modelos GPT han revolucionado la forma en que interactuamos con la tecnología. Desde las bases sentadas por Alan Turing en los años cuarenta, hasta la aparición de ChatGPT, es esencial entender cómo ha evolucionado este campo y cómo influye en nuestro día a día.

¿Cómo inició la Inteligencia Artificial?

El origen de la inteligencia artificial se remonta a los años cuarenta con la teoría computacional desarrollada por Alan Turing. Este científico propuso la capacidad de que máquinas comprendieran el conocimiento humano mediante comandos binarios, y posteriormente, con su *prueba de Turing*, estableció las primeras bases para evaluar si una máquina podía emular la conversación humana.

Poco tiempo después, los investigadores McCulloch y Pitts introdujeron la primera neurona artificial, una innovación crucial para el posterior desarrollo del campo.

¿Qué fue el AI Winter y cómo afectó al desarrollo tecnológico?

Entre 1970 y principios del año 2000 ocurrió el denominado AI Winter, un periodo caracterizado por significativa reducción de recursos e inversiones en IA, principalmente debido al fracaso de varios proyectos gubernamentales que no cumplían con las expectativas. Durante estos años, agencias destacadas como DARPA vieron necesario cortar significativamente sus financiaciones debido a críticas y escepticismos generalizados.

Sin embargo, pese a estos desafíos, el trabajo continuó, destacándose aportes clave como el desarrollo de las *Convolutional Neural Networks* (CNN) por parte del joven Yann LeCun en 1989. Este avance marcó un punto importante en la mejora de procesos de reconocimiento de imágenes y datos visuales.

¿Cómo han evolucionado los modelos GPT?

NVIDIA, con su lanzamiento del software CUDA en 2007, capacitó a los desarrolladores para realizar cálculos complejos mediante tarjetas gráficas. Esta innovación propició en 2011 el entrenamiento del modelo AlexNet, utilizando hardware que inicialmente estaba destinado al gaming. Avances posteriores liderados por científicos como Ilya Sutskever y Geoffrey Hinton contribuyeron al lanzamiento de transformers y sentaron las bases para la aparición del primer GPT en 2018.

Desde entonces, OpenAI ha protagonizado la evolución acelerada de estos modelos:

- **GPT-1 (2018)**: introdujo los transformers con apenas 117 millones de parámetros.
- **GPT-2 (2019)**: incrementó la complejidad hasta los 1.5 billones de parámetros.
- **GPT-3 (2020)**: dio un gran salto, alcanzando 175 billones de parámetros.
- **ChatGPT y GPT-3.5**: facilitaron a nivel masivo el acceso a estos modelos, mostrando un impresionante desempeño en interacciones conversacionales.
- **GPT-4**: agregó el factor humano en su entrenamiento mediante *Reinforcement Learning with Human Feedback*, llegando a impresionantes 1.7 trillones de parámetros.
- **DeepSeek y LLAMA**: modelos recientes que han optimizado sus capacidades con técnicas avanzadas frente a las limitaciones tecnológicas actuales.

¿Cuál es el presente y futuro cercano de la IA?

Hoy en día, el impacto de la inteligencia artificial se evidencia diariamente, modificando la interacción entre humanos y tecnologías digitales. Empresas líderes como Meta y Google están integrando estos avances en diversos productos, indicando la relevancia creciente de los grandes modelos de lenguaje.

La rápida escalabilidad de estos modelos plantea inquietudes en torno al acercamiento hacia la Inteligencia Artificial general (*Artificial General Intelligence, AGI*), y cómo estos progresos continuarán transformando múltiples aspectos del conocimiento y las interacciones humanas.

¿Cómo crees que impactarán estos desarrollos tecnológicos en nuestro futuro próximo? Deja tus reflexiones y preguntas en los comentarios.

1. La historia de la inteligencia artificial comenzó con Alan Turing en los años 40, planteando que las máquinas pueden procesar información similar a los humanos.
2. La primera neurona artificial fue creada en los años 50 por McCauleigh y Peters, sentando las bases para la IA moderna.
3. Los modelos de redes neuronales, como las CNN, fueron desarrollados en los años 80 y son fundamentales para los Transformers actuales.
4. OpenAI lanzó GPT-1 en 2018, marcando el inicio de los modelos de lenguaje generativos, seguido por mejoras rápidas con GPT-2 y GPT-3.
5. GPT-4 introdujo el aprendizaje por refuerzo con retroalimentación humana, mejorando el desempeño y acercándose a la inteligencia artificial general.

Resumen

Los LLMs como ChatGPT entienden nuestras comunicaciones mediante procesos matemáticos específicos llamados tokenización, vectorización y embeddings. Estos métodos permiten transformar el lenguaje natural, audios e imágenes en información que una máquina puede comprender y procesar eficazmente.

¿Qué es la tokenización y cómo funciona?

La **tokenización** consiste en dividir textos en partes más pequeñas y manejables llamadas tokens. Un token puede ser una palabra, como en el ejemplo: "hola ¿cómo estás?", que se divide en tres tokens individuales: hola, cómo, y estás. Además de por palabras, la tokenización puede ser por subpalabras o fragmentos.

¿Cómo se lleva a cabo la vectorización?

Luego de tokenizar, continúa el proceso con la **vectorización** que consiste en asignar vectores o grupos numéricos a cada token inicial. En nuestro ejemplo:

- "hola" podría ser representado como (1,2)
- "cómo" como (1,3)
- "estás" como (3,4)

Estos vectores se crean inicialmente de forma aleatoria. Después, es posible representarlos en un plano cartesiano para visualizar sus relaciones.

¿Por qué es importante el proceso de embedding?

El **embedding** optimiza y ubica estratégicamente estos vectores, agrupándolos cerca de otros con significados similares. Con ejemplos como rey, hombre y reina, al aplicar álgebra vectorial:

- Rey - hombre = Reina

Obtenemos nuevos significados a partir de cálculos matemáticos. Esto facilita abordar conceptos abstractos en modelos numéricos, mejorando las capacidades del LLM.

¿Cómo utilizar Hugging Face y PyTorch para LLMs con tokenización y embeddings en Python?

Usando las librerías Transformers (Hugging Face) y Torch (PyTorch), los procesos mencionados pueden aplicarse en Python.

Tokenización con Hugging Face

Instalando primero Transformers:

```
!pip install transformers
```

Luego, cargando el tokenizer:

```
from transformers import BertTokenizer
```

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
tokens = tokenizer.tokenize("hola, ¿cómo estás?")
```

Esto segmentará tu texto según el modelo utilizado, ya sea en palabras o subpalabras.

Embeddings con PyTorch

La implementación de embeddings mediante PyTorch:

```
import torch
from transformers import BertModel

model = BertModel.from_pretrained('bert-base-uncased')
king_embedding =
model.embeddings.word_embeddings(torch.tensor(tokenizer.convert_tokens_to_ids(["king"])))
man_embedding =
model.embeddings.word_embeddings(torch.tensor(tokenizer.convert_tokens_to_ids(["man"])))
queen_embedding =
model.embeddings.word_embeddings(torch.tensor(tokenizer.convert_tokens_to_ids(["queen"])))
```

Esto permite transformar tokens en vectores y posteriormente evaluar la similitud entre ellos mediante la función coseno:

```
cosine_similarity = torch.nn.CosineSimilarity(dim=0)
similitud = cosine_similarity(king_embedding, queen_embedding)
```

Los valores cercanos a 1 indican mucha similitud y aquellos cercanos a -1 indican diferencias considerables.

¿Qué puedes practicar con estos conceptos?

Te invitamos a practicar cambiando las palabras en los ejemplos dados. Explora relaciones entre palabras, por ejemplo, animales similares al gato o al perro. Gracias a estos conceptos, puedes profundizar tus conocimientos sobre cómo los grandes modelos del lenguaje comprenden nuestro mundo. Te animamos a dejar tus experiencias y descubrimientos en los comentarios.

Conceptos clave:

Tokenizador: algoritmo que parte las palabras en subpalabras o sílabas y a su vez forma un vector con estas

embedding: el resultado de pasar los tokens por una red neuronal ya pre-entrenada en categorizar tokens por su significado o atributos en un espacio de N dimensiones

si en un caldo los tokens son todas las verduras picadas, los embeddings serian el sabor que identifica ese trozo o cucharada (grupos de conceptos), de esa forma una papa sabe "parecido" a una yuca (tokens semánticamente próximos en el espacio vectorial), pero muy distinto a un trozo de cebolla, esa analogía me sirvió

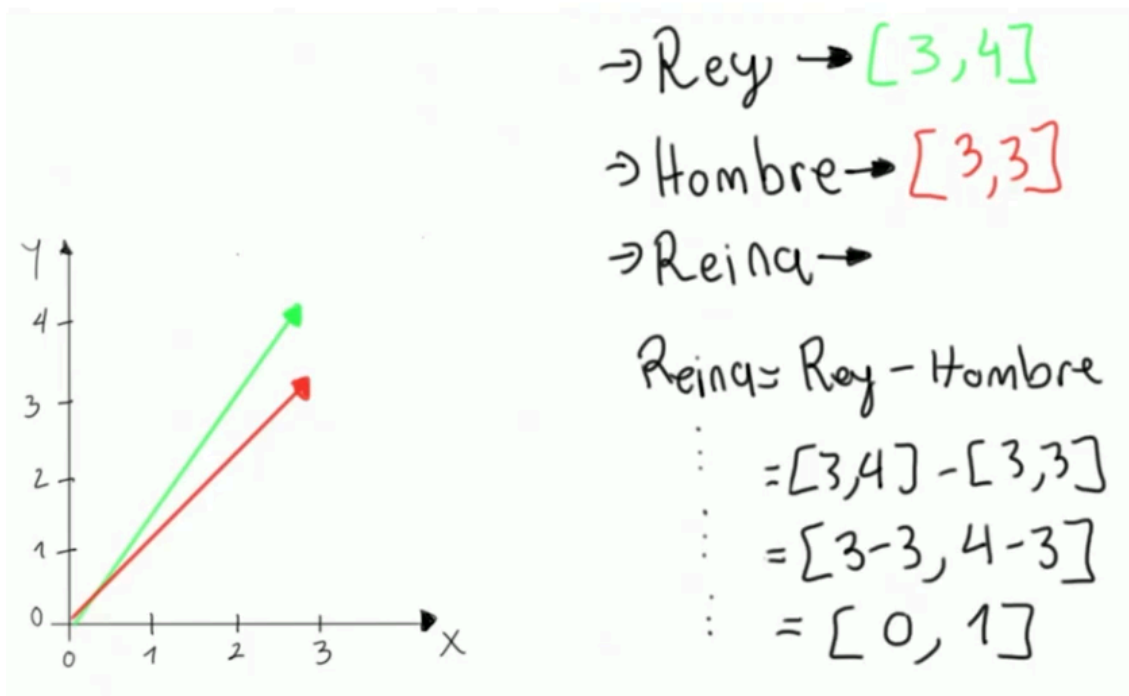
si quieren verlo de forma más visual, word2beck fue uno de los primeros generadores de embeddings, el que se ve en la clase es bert pero es casi la misma idea

<https://projector.tensorflow.org/>

Los conceptos más importantes de la clase incluyen:

1. **Tokenización:** Proceso de dividir texto en partes más pequeñas (tokens), como palabras o subpalabras, para su análisis.
2. **Vectorización:** Asignación de vectores a cada token, permitiendo que las palabras se representen numéricamente.
3. **Embeddings:** Técnica que organiza los vectores en un espacio vectorial, capturando relaciones semánticas. Permite realizar operaciones matemáticas entre palabras, como sumar o restar vectores.
4. **Transformers:** Arquitectura utilizada en los LLMs que facilita el procesamiento de texto mediante atención y paralelismo.

Estos conceptos son fundamentales para comprender cómo funcionan los modelos de lenguaje.



Resumen

Las redes neuronales, específicamente los multilayer perceptron (MLP), son una tecnología esencial en machine learning. Estas redes, conocidas como *feed forward*, poseen estructuras claras con múltiples capas, incluyendo una capa de entrada, varias capas ocultas y una capa de salida.

¿Qué son las redes neuronales multilayer perceptron?

Los multilayer perceptron son redes neuronales que se componen principalmente de tres capas:

- **Capa de entrada:** esta depende directamente del problema que se enfrenta. Por ejemplo, el español con cincuenta mil palabras tendría un número idéntico de neuronas en esta capa.
- **Capas ocultas:** en estas capas se lleva a cabo el aprendizaje del modelo, permitiendo entender las características y propiedades del lenguaje o problema específico.
- **Capa de salida:** también depende del problema planteado y su número suele coincidir con la capa de entrada para tareas como predicción de palabras.

¿Cómo funciona una neurona en una red neuronal?

Cada neurona en las capas ocultas posee tres componentes básicos:

- **Peso (weight):** señala la importancia de una característica específica para el modelo.
- **Sesgo (bias):** ayuda a la red neuronal a no memorizar exacta y rígidamente los datos del entrenamiento inicial.
- **Función de activación:** permite captar patrones en los datos proporcionados, basándose en cómo se activan las neuronas del cerebro humano.

Estas neuronas realizan cálculos matemáticos partiendo de datos de entrada, que multiplican por pesos específicos. Luego se suma el sesgo, resultando en una función lineal básica, a la que posteriormente se añade una función no lineal que permite modelar comportamientos complejos.

¿Qué es la función de pérdida y cómo mide el aprendizaje?

La función de pérdida determina si la red neuronal está realmente aprendiendo. Básicamente, compara los resultados predichos por la red con los resultados reales conocidos, siendo el objetivo reducir esta diferencia al máximo, idealmente hasta el valor cero. Una función común para evaluar la pérdida en problemas lineales es el error cuadrático medio.

¿En qué consiste el método Back Propagation?

El método de *back propagation* (propagación hacia atrás) consiste en tomar el valor obtenido en la función de pérdida para actualizar los pesos y sesgos de la red neuronal. Este método

emplea conceptos matemáticos avanzados, como derivadas y derivadas parciales, buscando mínimos locales en la función de pérdida para mejorar continuamente el aprendizaje del modelo.

¿Qué indica el teorema universal de aproximación?

Este teorema establece que cualquier problema que pueda formularse matemáticamente puede aproximarse mediante una red neuronal, siempre que sean usadas funciones de activación no lineales. Esto permite modelar distintos tipos de comportamientos, incluyendo curvas complejas y espacios multidimensionales.

¿Qué limitaciones tienen los multilayer perceptron?

Los MLP presentan dificultades para captar contextos más amplios, como frases o párrafos, algo esencial para comprender adecuadamente el lenguaje humano. Por esta razón, surgieron estructuras que manejan mejor esta complejidad, tales como:

- **LSTM (Long Short-Term Memory)**: conservan información contextual extendida.
- **CNN (Convolutional Neural Networks)**: capturan información espacial o de contexto visual.

Otra limitación relevante es el problema conocido como *vanishing gradients*, donde el modelo queda atrapado en mínimos locales sin importar la potencia de cálculo empleada. Técnicas como el *dropout* pueden ayudar a mitigar esto.

Si te interesa profundizar más, en los cursos avanzados de álgebra lineal y cálculo diferencial se trabajan esos conceptos matemáticos fundamentales. ¿Qué aspectos crees que puedes profundizar para fortalecer tu aprendizaje sobre redes neuronales?

Si estás leyendo esto para entender conceptos como estos recuerda que necesitas tener conocimientos previos en varias áreas de las matemáticas.

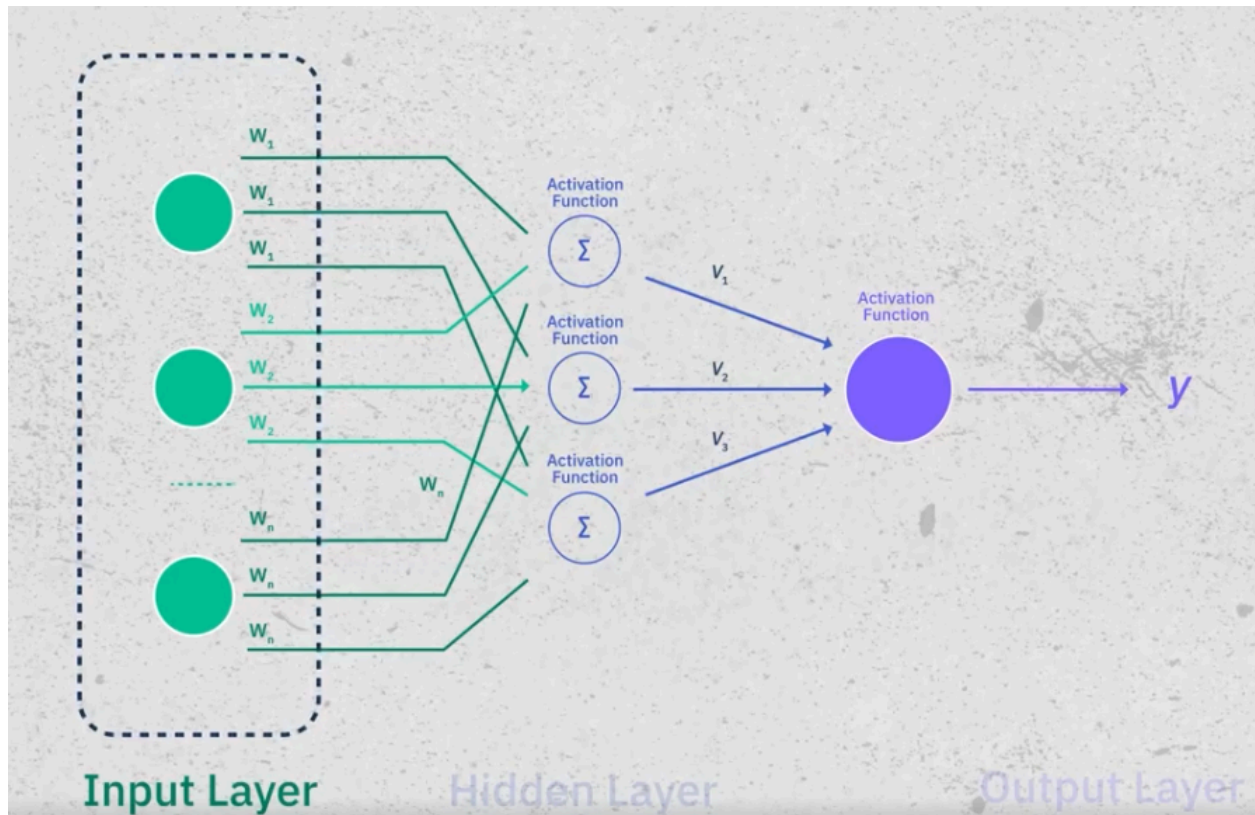
Conceptos de álgebra lineal: Porque las redes neuronales operan con vectores, matrices y operaciones como multiplicaciones matriciales.

Conceptos de cálculo diferencial: Porque el entrenamiento de redes se basa en minimizar una función de pérdida utilizando derivadas (gradientes).

Conceptos de probabilidad y estadística: Porque muchos modelos asumen distribuciones de probabilidad, y el error, la entropía o funciones como *cross-entropy loss* se basan en conceptos estadísticos.

Y conceptos aún mas profundos de redes neuronales, de algoritmos de optimización, de backpropagation ya que el aprendizaje implica modificar los pesos para minimizar errores.

No te rindas tu puedes....



Suena complejo pero en la realidad sería más como: Imagina que tienes un **montón de datos de casas** y quieres que la computadora aprenda a adivinar sus precios.

1. Prepara y Separa tus Datos:

- **Tú:** Tomas tu archivo gigante de casas (con su tamaño, habitaciones y precio).
- **Tú:** Le dices a la computadora que **separe** ese archivo: 80% para que **aprenda** (entrenamiento) y 20% para **probar** si aprendió bien.
- **Tú:** Le dices que "normalice" los números (ponga todo en la misma escala, de 0 a 1) para que la computadora no se confunda con los números grandes y pequeños.

2. Define el "Cerebro" de la Computadora (Código):

- **Tú (en el código):** Le dices a la computadora: "Construye un cerebro artificial. Que tenga una entrada para cada característica de la casa (tamaño, habitaciones, etc.), un par de capas internas para pensar, y una salida para el precio."
- **Tú (en el código):** Le dices: "Quiero que uses el **Error Cuadrático Medio (ECM)** para saber qué tan mal te equivocas." También le dices qué tan "grande" debe ser cada ajuste que haga al equivocarse (la **tasa de aprendizaje**).

3. Pon el "Cerebro" a Aprender (Correr el Código):

- **Tú (en el código):** Le dices: "¡Listo! Empieza a aprender con el 80% de las casas."
- **Computadora (detrás de escenas):** El cerebro empieza a adivinar precios. Por cada error que comete, usa el **ECM** para calcular el tamaño del error y **automáticamente se ajusta** para equivocarse menos la próxima vez. Hace esto miles de veces (las "épocas").

4. Revisa el Aprendizaje con el ECM (¡Tu Medidor!):

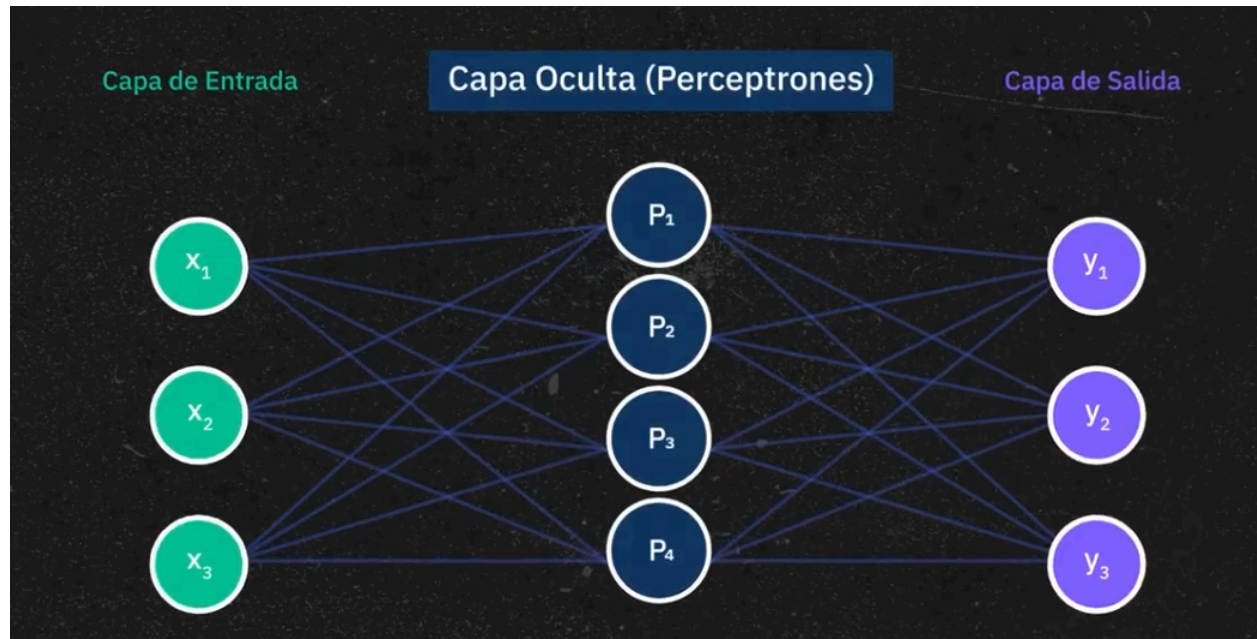
- **Tú (después de que termina de aprender):** Le dices a la computadora: "Ahora, adivina los precios del 20% de las casas que nunca has visto."
- **Computadora:** Adivina esos precios y te muestra el **ECM final** para esas casas nuevas.
- **Si el ECM es un número pequeño** (ej. 0.05, 0.001): ¡**VA BIEN!** Significa que tu cerebro artificial está adivinando los precios de casas nuevas con mucha precisión.
- **Si el ECM es un número grande** (ej. 100, 5000): ¡**NO VA BIEN!** Significa que el cerebro aún se equivoca mucho.

5. Corrige y Repite (Si no Va Bien):

- **Tú:** Si el **ECM es alto**, vuelves al **Paso 2** (diseño del "cerebro") o al **Paso 3** (la configuración del aprendizaje).
- **Tú:** Quizás le das más capas para pensar, o más neuronas.
- **Tú:** O ajustas la tasa de aprendizaje (que se corrija más o menos).
- **Tú:** Luego, repites el **Paso 3** y 4. Sigue haciendo esto hasta que el **ECM** sea lo más pequeño posible en las casas nuevas.

Así de simple: Separas los datos, defines las reglas de aprendizaje (donde el **ECM** es clave), la computadora aprende y se ajusta sola, y tú revisas el **ECM** para saber si necesitas seguir corrigiendo tus configuraciones.

El error cuadrático medio (ECM) es una métrica utilizada para evaluar el desempeño de modelos de regresión. Se calcula como la media de los cuadrados de las diferencias entre los valores predichos por el modelo y los valores reales. Un ECM más bajo indica que el modelo tiene un mejor ajuste a los datos, mientras que un valor más alto sugiere un peor rendimiento. Es fundamental en el entrenamiento de redes neuronales, ya que guía la actualización de pesos durante el proceso de aprendizaje.



Otras funciones de pérdida usadas son:

- Error absoluto medio (MAE)
- $\sum_{i=1}^n |Y_i - \hat{Y}_i|$ (Y_i siendo valor real y $\hat{Y}_i$ el valor predicho) Aquí no se penaliza tanto los errores grandes a diferencia de MSE que al hacerlo cuadrático los penaliza más.
- Entropía cruzada:
- $-\sum_{i=1}^n p_i \log(p_i)$ (p_i es la probabilidad predicha) Se usa más para clasificación.

REDES NEURONALES MULTILAYER PERCEPTRON (MLP) Tecnología esencial en *machine learning*. Son redes *feed forward*: la información fluye en una sola dirección.

ESTRUCTURA BÁSICA DE UNA MLP

Capa de Entrada

- Depende del problema.
- Ejemplo: idioma con 50.000 palabras = 50.000 neuronas.

Capas Ocultas

- Donde ocurre el aprendizaje.
- Captan patrones y características.

Capa de Salida

- Da la respuesta del modelo.
- Puede coincidir en tamaño con la de entrada (según la tarea).

⚙️ ¿CÓMO FUNCIONA UNA NEURONA?

📌 Tres componentes esenciales:

- **Peso (weight)** → mide la importancia de cada dato.
- **Sesgo (bias)** → ayuda a no memorizar datos rígidamente.
- **Función de activación** → introduce no linealidad, permite detectar patrones complejos.

📊 **Proceso** Entrada $\times$ peso $+$ sesgo $\rightarrow$ función de activación $\rightarrow$ salida.

📈 FUNCIÓN DE PÉRDIDA

🎯 Mide si el modelo está aprendiendo correctamente.

✅ Compara: Predicción de la red **vs** Resultado real

📈 Objetivo: Reducir la diferencia (ideal: llegar a cero)

📐 Función común: Error Cuadrático Medio

🔄 BACK PROPAGATION

🔍 ¿Qué hace? Ajusta pesos y sesgos según el error.

🧠 Usa:

- Derivadas
- Derivadas parciales

🎯 Busca: Mínimos locales en la función de pérdida para mejorar el modelo.

🔧 TEOREMA UNIVERSAL DE APROXIMACIÓN

📖 ¿Qué dice? Una red con funciones de activación no lineales puede aproximar cualquier función matemática.

🌐 Implica:

- Capacidad para modelar comportamientos complejos.
- Útil en tareas con datos multidimensionales.

⚠️ LIMITACIONES DE LOS MLP

❶ **Poca comprensión del contexto amplio** No manejan bien frases o párrafos completos.

🧩 Alternativas:

- **LSTM** → mantiene contexto largo.
- **CNN** → detecta patrones espaciales o visuales.

2 **Vanishing gradients** El modelo se “traba” al aprender, aunque se use mucha potencia.

🔧 Solución:

- Técnicas como **dropout** para reducir este efecto.

$z = wTx + b$	
w	Weight (Peso)
T	Transpuesta
b	Bias (sesgo)

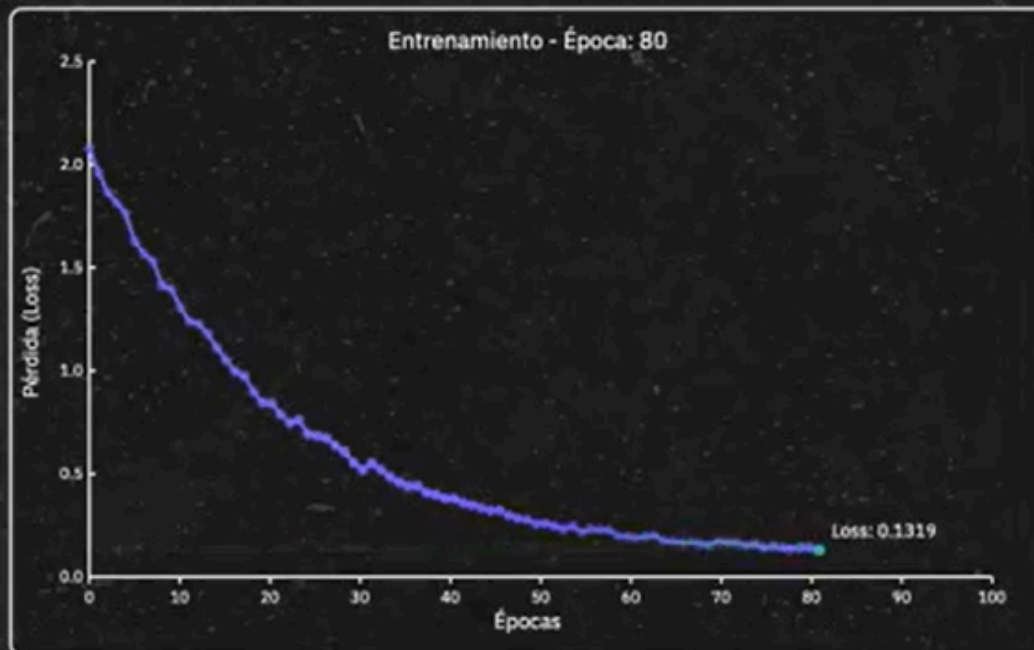
El teorema universal de la aproximación establece que una red neuronal de capa oculta con al menos una neurona puede aproximar cualquier función continua, siempre que se use una función de activación no lineal. Esto significa que, a pesar de su simplicidad, los multilayer perceptrons son capaces de aprender patrones complejos en los datos. Este teorema es fundamental en el desarrollo de modelos de aprendizaje profundo, ya que garantiza que las redes neuronales pueden modelar cualquier problema que pueda ser descrito matemáticamente.

Tablero de la función de pérdida (MSE)

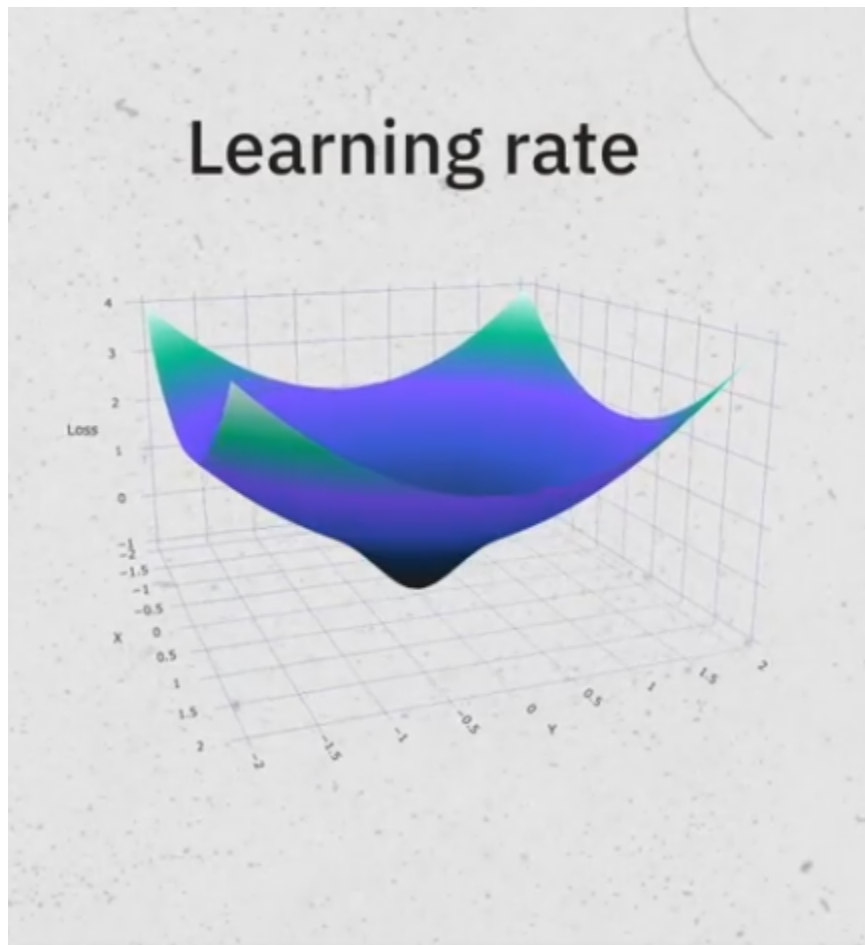
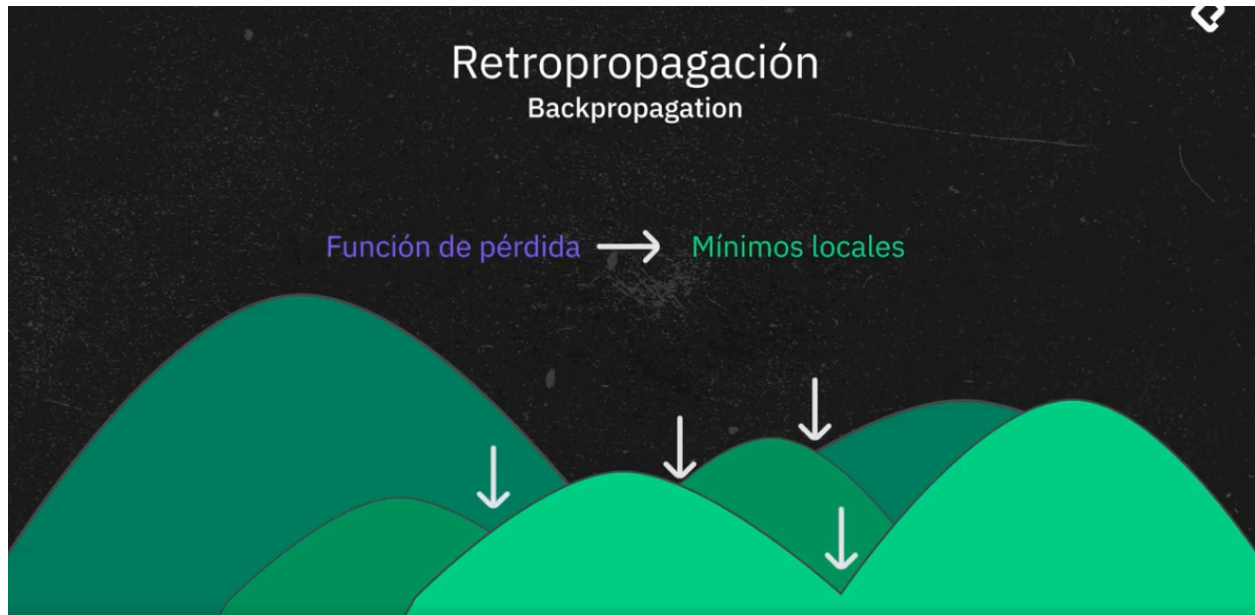
2. Tabla de operaciones para cada punto

Punto	x_i	y_i	$\hat{y}_i = wx_i + b$	Error $e_i = \hat{y}_i - y_i$	e_i^2
● P4	2	2	$2w + b$	$(2w + b) - 2$	$(2w + b - 2)^2$
● P1	3	3	$3w + b$	$(3w + b) - 3$	$(3w + b - 3)^2$
● P5	4	4	$4w + b$	$(4w + b) - 4$	$(4w + b - 4)^2$

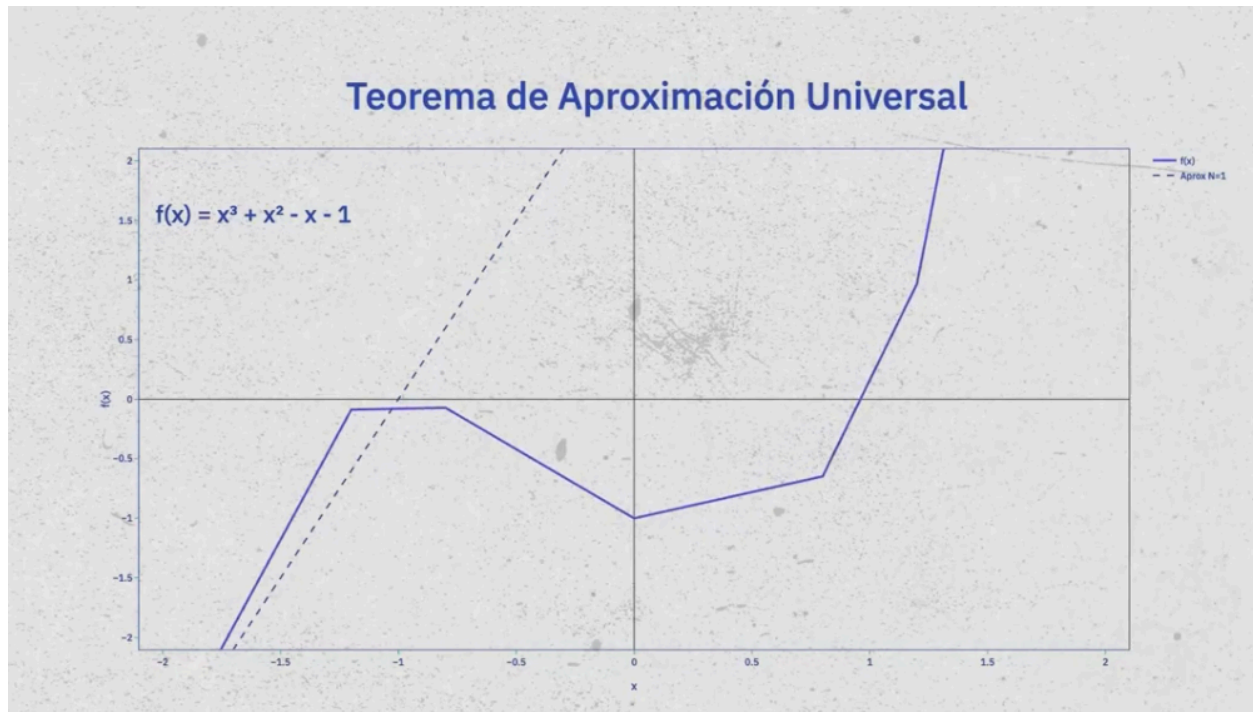
Función de pérdida



$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\varphi_i - y_i)^2$$



Derivadas , derivadas parciales, teorema universal de aproximacion



Resumen

La arquitectura GPT-2 y, en particular, el mecanismo de atención, constituyen elementos fundamentales para comprender cómo funcionan los actuales modelos de lenguaje llamados transformadores. Estos modelos emplean técnicas matemáticas para identificar y mantener contexto entre palabras, potenciando su capacidad predictiva y generativa en tareas del procesamiento de lenguaje natural (NLP).

¿Qué componentes integran la arquitectura de GPT-2?

GPT-2 se construye principalmente sobre 12 estructuras denominadas transformadores. Cada transformador incluye:

- Una *multihead attention*.
- Dos capas llamadas *layer norm*.
- Una capa *feed forward*.

El aspecto crucial dentro de estos transformadores es el mecanismo de atención, fundamental para mantener la coherencia contextual.

¿Cómo opera el mecanismo de atención?

La atención actúa examinando qué palabras previas afectan significativamente a la palabra actual. Matemáticamente se representa así:

$$[\text{Atención} = \text{softmax} \left(\frac{Q K^T}{\sqrt{d_k}} \right) V]$$

Aquí describimos el significado de sus componentes:

¿Qué significa cada símbolo de la atención?

- **Q (Query):** representa preguntas provenientes de cada palabra para identificar qué palabras anteriores influyen en su significado.
- **K (Key o llave):** alude a respuestas aproximadas correspondientes a las preguntas generadas por las consultas (queries).
- **V (Value o valor):** valores necesarios para modificar adecuadamente los *embeddings*, manteniendo el contexto correcto.

Cada componente se obtiene multiplicando los *embeddings* originales por matrices de pesos específicas (W_q , W_k , W_v) que se ajustan durante el entrenamiento del modelo.

¿Cómo interactúan estos elementos en un ejemplo práctico?

Si tomamos la frase "Me encanta el color negro, espero algún día tener un gato así", la palabra "negro" afecta directamente a la palabra "gato".

- **Ejemplo de Query (Q):** la palabra "gato" hace una consulta específica: ¿qué palabras anteriores modifican la palabra actual?
- **Ejemplo de Key (K):** identifica a "negro" como respuesta precisa, previamente codificada en una matriz entrenada.
- **Ejemplo de Value (V):** ajusta el *embedding* original de "gato" hacia uno contextual ("gato negro"), manteniendo la información correcta.

Multiplicando Q por K obtenemos pesos que revelan qué tan relacionadas están las palabras. La operación matemática y la función de *softmax* reducen la escala numérica para optimizar cálculos, manteniendo la precisión y estabilidad computacional.

¿Cuál es el papel de la función Softmax en atención?

Softmax limita la escala numérica de los resultados de la multiplicación QK, transformándolos de un rango inicialmente amplio (desde menos infinito hasta más infinito) a uno manejable y acotado (entre -1 y 1). Esto simplifica y acelera las operaciones informáticas necesarias durante el entrenamiento.

¿Qué función cumple la división por raíz cuadrada de d_k ?

La división por la raíz cuadrada de d_k no afecta directamente al aprendizaje sino que protege la estabilidad numérica del proceso computacional. Previene errores por limitaciones inherentes al hardware y su precisión numérica durante estas operaciones.

¿Qué otros elementos acompañan al mecanismo de atención en GPT-2?

Tras el mecanismo de atención, GPT-2 integra componentes comunes del machine y deep learning como:

- Capas *layer norm*.
- Redes feed-forward.
- Clasificadores tradicionales de tarea (*text classifier o task classifier*).

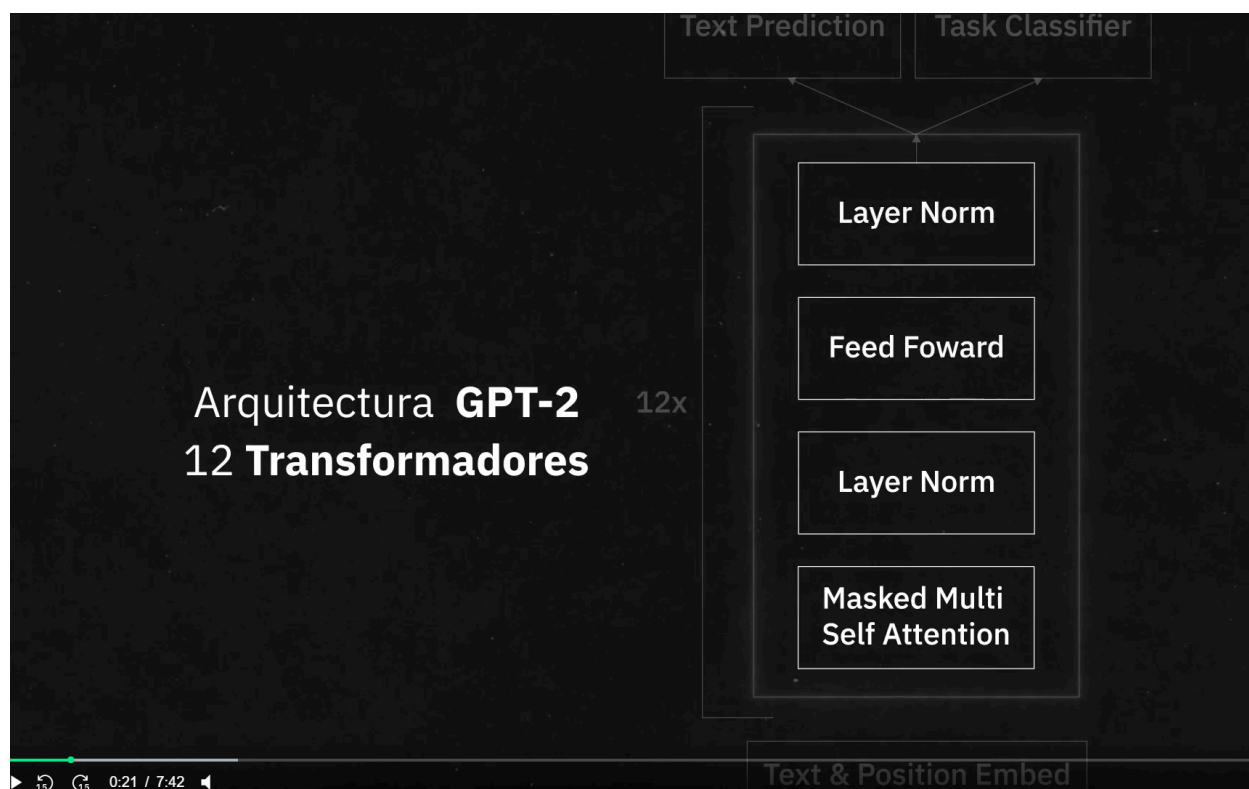
Estos elementos complementarios están diseñados para mejorar la generalización y optimizar el comportamiento del modelo.

¿Cómo continuar profundizando en GPT-2 y sus bases teóricas?

Para quienes desean profundizar, se recomienda explorar el documento original (*paper*) de GPT-2 proporcionado por OpenAI. Adicionalmente, los fundamentos teóricos sobre redes neuronales profundas, normalización y clasificadores se encuentran ampliamente documentados en los recursos adicionales brindados durante el curso.

¡Te animamos a seguir aprendiendo y comentarnos sobre tu experiencia estudiando o construyendo modelos basados en GPT-2!

investigando un poco en internet, algo que me parece fundamental mencionar es que antes del mecanismo de atención, se usaban **redes neuronales recurrentes**, donde la salida de una capa se usaba como entrada de otra y el fallo que tenían estas redes era que después de cierta cantidad de tokens "no recordaban" los anteriores, al solucionarlo con el mecanismo de atención, hacían que el modelo tuviera contexto y eran redes neuronales recurrentes + atención, luego se dieron cuenta que podían eliminar las redes recurrentes y por eso el artículo se titula "**la atención es todo lo que necesitas**", y generaron una nueva arquitectura teniendo como centro el mecanismo de atención



En el contexto del mecanismo de atención, los modelos de lenguaje como GPT-2 no almacenan explícitamente todos los tokens categorizados como adjetivos, sustantivos o verbos. En su lugar, utilizan embeddings para representar palabras en un espacio continuo. Durante el proceso de atención, el modelo aprende a relacionar palabras entre sí y a entender su contexto. Así, aunque no haya un almacenamiento directo de las categorías gramaticales, el modelo puede inferir la función de una palabra en base a su relación con otras palabras en la frase.

$$Attention(Q,K,V) : softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

En la clase se abordaron los siguientes puntos clave sobre la arquitectura de GPT-2 y el mecanismo de atención:

1. **Transformadores en GPT-2:** Se compone de 12 transformadores, cada uno con multi-head attention, feed forward, y normalización.
2. **Mecanismo de Atención:** Utiliza tres conceptos:

- **Query (q):** Preguntas sobre la palabra actual.
- **Key (k):** Respuestas a las preguntas, relacionadas con las palabras anteriores.
- **Value (v):** Valores que ajustan el embedding de la palabra.

3. **Softmax:** Función utilizada para normalizar los valores, facilitando el procesamiento.
4. **Estabilidad Numérica:** Se incluye una división por la raíz cuadrada de la dimensión k para asegurar precisión en cálculos.
5. **Auto-atención:** Permite entender la relación entre palabras en un contexto.

Estos conceptos son fundamentales para comprender cómo funcionan los LLMs modernos.

Me fascinó entender cómo los **12 transformadores** trabajan en conjunto, especialmente el mecanismo de *multihead attention* con sus componentes **Q, K y V**. La analogía de "gato negro" para explicar cómo la *query* busca contexto, la *key* responde con el adjetivo relevante y el *value* ajusta el *embedding* fue brillante.

Lo que más me impactó:

- La **fórmula matemática** detrás de la atención ($\text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$) dejó claro que la "comprensión" del modelo es en realidad un **proceso de ponderación numérica**.
- Los detalles técnicos como la **normalización por $\sqrt{d_k}$** (para estabilidad) y el rol de *softmax* (para acotar valores) mostraron que hasta los elementos "secundarios" son clave en la eficiencia del modelo.

3

Responder

Reportar



Julio Téllez

student • hace 2 meses

Definitivamente sin AI no podría entender este tema, si bien la clase es clara, para mí no lo fue tanto. Me apoye con ChatGPT para entender la formula de atención y me exploto la cabeza. Demasiado interesante esta clase.

Me encanta el color negro, espero algún día tener un gato así.

Q:Query (Pregunta)

				Me	encanta	el	color	negro,	espero	algún	día	tener	un	gato	así.
				E	E	E	E	E	E	E	E	E	E	E	E
				Wq	Wq	Wq	Wq	Wq	Wq	Wq	Wq	Wq	Wq	Wq	Wq
				E*Wq	E*Wq	E*Wq	E*Wq	E*Wq	E*Wq	E*Wq	E*Wq	E*Wq	E*Wq	E*Wq	E*Wq
				Q	Q	Q	Q	Q	Q	Q	Q	Q	Q	Q	Q
Me															
encanta															
el															
color															
negro,															
espero															
algún															
día															
tener															
un															
gato															
así.															

Resumen

PyTorch es un framework robusto creado por Meta, anteriormente Facebook, diseñado específicamente para construir modelos de machine learning y deep learning. Ofrece recursos variados y librerías que facilitan desarrollar arquitecturas neuronales complejas desde cero o con cierta base previa.

¿Qué es un tensor en PyTorch?

El elemento central de PyTorch es el tensor, una estructura similar a una lista, vector o matriz en Python. Básicamente, se trata de una estructura n-dimensional manejada internamente según filas y columnas requeridas para almacenar datos numéricos. En machine learning, estos tensores constituyen la base sobre la que se procesan los modelos.

¿Cuál es el rol de Autograd en PyTorch?

Un aspecto fundamental de PyTorch es Autograd, un sistema computacional diseñado para realizar automáticamente la derivación parcial. Sin Autograd, los cálculos de derivadas parciales para modelos grandes consumirían demasiado tiempo manualmente. Con este mecanismo, PyTorch automatiza estos procedimientos mediante aproximaciones que facilitan notablemente la creación y ajuste de modelos avanzados.

¿Qué es el optimizer y cómo ayuda a entrenar modelos?

El *optimizer* es otro componente clave que maneja el aprendizaje del modelo mediante métodos específicos como el popular algoritmo Adam. Este realiza ajustes internos (por

ejemplo, utilizando *back propagation*) y determina el *learning rate*, ayudando a que el modelo supere mínimos locales y optimizando su aprendizaje.

Tipos de optimizadores disponibles:

- Adam
- Adam X
- Adam W
- Adam Z

¿Cómo implementar un modelo MLP con PyTorch paso a paso?

Utilizando PyTorch, la creación de un modelo *Multi Layer Perceptron* (MLP) sigue estos pasos simples pero indispensables:

1. Importación y configuración: `python import torch import torch.nn as nn import torch.optim as optim import torch.nn.functional as F import time`
2. Definir la arquitectura de red neuronal: `python class MLP(nn.Module): def init(self, input_dim, hidden_dim, output_dim): super(MLP, self).init() self.layer1 = nn.Linear(input_dim, hidden_dim) self.layer2 = nn.Linear(hidden_dim, output_dim) def forward(self, x): x = F.relu(self.layer1(x)) x = self.layer2(x) return x`
3. Entrenamiento y optimización:
4. Instanciar el modelo con dimensiones específicas.
5. Elegir la función de pérdida (error cuadrático medio).
6. Seleccionar un optimizador, frecuentemente Adam.
7. Crear datos ficticios y ejecutar entrenamiento. `python model = MLP(10, 100, 1) criterion = nn.MSELoss() optimizer = optim.Adam(model.parameters(), lr=0.01)`

¿Es posible entrenar modelos con GPU en PyTorch?

Sí, PyTorch permite usar GPU para acelerar los procesos. El paso para verificar disponibilidad es sencillo:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Luego, transfiriendo nuestros tensores al dispositivo elegido, podemos reducir significativamente los tiempos de procesamiento de grandes conjuntos de datos o arquitecturas más complejas.

Ejemplo práctico de uso con GPU:

```
x = torch.randn(2000000, 10).to(device)
y = torch.randn(2000000, 1).to(device)
```

Te invitamos a dejar tu comentario sobre cualquier punto adicional sobre PyTorch o la implementación de modelos avanzados que te interese explorar más a fondo.

me dolió la cabeza, investigue en internet un poco pero creo que le tome la idea, mi aporte.

conceptos clave:

Función de activación: existen muchas y depende del problema que se trata de solucionar, básicamente es un umbral ante el que responde o no una neurona, ¿recuerdan a McFly de volver al futuro?, el no respondía cuando se la montaban, a menos que le dijeran gallina, la función de activación de McFly era esa palabra. (claro no es tan fácil pero se entiende)

Función de coste: como sabes que el modelo esta respondiendo mal? bueno porque sabes la respuesta correcta, la función de coste es la diferencia que hay entre las respuestas azarosas que da el modelo vs las respuestas correctas o esperadas, como es una función se puede graficar, así que empiezas desde el primer valor al azar y vas reajustando hasta que el error sea casi 0, la grafica que se da al hacer esto es **el descenso del gradiente**.

Backpropagation: es un algoritmo que en términos mas mundanos busca la culpa, empezando por la ultima capa (la de salida), es como esa estrategia de los 5 porqués que creó Honda, empieza a preguntar de pa a atrás capa por capa cual fue la neurona que causo el error y en esa cadena de responsables se ajustan los pesos, como cuando en una empresa algo sale mal y empieza a escalar el problema capa por capa hasta encontrar el responsable

(obviamente no es tan simple, se calculan las derivadas parciales de las funciones de coste y de activación capa por capa)

```
# 1. Preparar el modelo
model = MLP(input_dim=10, hidden_dim=64, output_dim=1)
criterion = nn.MSELoss() # Mean Squared Error
optimizer = optim.Adam(model.parameters(), lr=0.01)

# 2. Preparar datos (ejemplo)
X_train = torch.randn(1000, 10)
y_train = torch.randn(1000, 1)

# 3. Ciclo de entrenamiento
model.train()
for epoch in range(100):
    # Forward pass
    predictions = model(X_train)
    loss = criterion(predictions, y_train)

    # Backward pass
    optimizer.zero_grad()
```

```
loss.backward()
optimizer.step()

if epoch % 10 == 0:
    print(f"Época {epoch}, Pérdida: {loss.item():.4f}")
```

Tensor in Pytorch

1	2	3
4	5	6
7	8	9

An n-dimensional array,
a generalization of matrices
and vectors

 PYTORCH EN RESUMEN

¿Qué es PyTorch?

- Framework creado por Meta (Facebook).
- Diseñado para Machine Learning y Deep Learning.
- Permite construir redes neuronales desde cero o con base previa.
- Ofrece librerías y herramientas para acelerar el desarrollo.

¿Qué es un Tensor?

- Estructura n-dimensional (como listas, vectores o matrices).
- Contiene datos numéricos organizados en filas y columnas.
- Es la **unidad básica** para representar datos en modelos.

¿Qué es Autograd?

- Sistema que realiza **derivación automática**.
- Calcula **gradientes** sin intervención manual.
- Esencial para el entrenamiento mediante **backpropagation**.
- Ahorra tiempo y evita errores al derivar funciones complejas.

Optimizer: Motor del Aprendizaje

- Ajusta pesos del modelo para mejorar su rendimiento.
- Determina la tasa de aprendizaje (*Learning rate*).
- Utiliza algoritmos como Adam para escapar de mínimos locales.

Tipos de optimizadores en PyTorch:

- Adam
- AdamX
- AdamW
- AdamZ

Modelo MLP (Multi Layer Perceptron): Pasos

1. Importar librerías: `import torch, torch.nn, torch.optim, torch.nn.functional, time`
2. Definir la arquitectura: `class MLP(nn.Module):`

```
def __init__(self, input_dim, hidden_dim, output_dim):  
    super(MLP, self).__init__()  
  
    self.layer1 = nn.Linear(input_dim, hidden_dim)  
    self.layer2 = nn.Linear(hidden_dim, output_dim)  
  
def forward(self, x):  
    x = F.relu(self.layer1(x))  
    return self.layer2(x)
```

1. Entrenar el modelo: `model = MLP(10, 100, 1)`

```
criterion = nn.MSELoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

Uso de GPU en PyTorch

¿Se puede usar GPU? ¡Sí!

- Mejora la velocidad de entrenamiento.
- Ideal para grandes volúmenes de datos y modelos complejos.

Cómo verificar y usar la GPU:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
  
x = torch.randn(2000000, 10).to(device)  
  
y = torch.randn(2000000, 1).to(device)
```

)

Resumen

Crear tu propio modelo GPT-2 desde cero puede parecer complejo, pero usando Python, PyTorch y Google Colab podrás realizarlo de forma práctica. El proceso incluye desde la configuración del entorno hasta la implementación del modelo, siempre enmarcado en la simplicidad de Google Colab con una GPU para mejorar el rendimiento.

¿Cómo configurar el entorno en Google Colab para construir GPT-2?

El primer paso para crear tu GPT-2 es estructurar adecuadamente Google Colab:

- Accede a la sección de recursos y selecciona GPU T4 para aprovechar su capacidad de procesamiento gráfico.
- Instala librerías esenciales como Einops, *Xformers* y NumPy (NP), además de PyTorch.

Esta configuración acelerará significativamente el desarrollo y la ejecución de tu proyecto.

¿Cuáles son los componentes clave del GPT-2?

GPT-2 integra varios módulos fundamentales que trabajarás en columnas separadas para claridad y organización:

- **Atención multi-cabezal (*multi-head attention*)**: biblioteca NN de PyTorch permite implementar consultas (*queries*), claves (*keys*) y valores (*values*) que permiten al modelo entender el contexto.
- **Feed Forward Network**: red neuronal básica que incluye capas convolucionales, activaciones (*function hello*) y *dropout* para evitar sobreaprendizaje.
- **Normalización de capas (*layer norm*)**: proporciona estabilidad al entrenamiento del modelo.

En la práctica, PyTorch automatiza procesos complejos, facilitando tareas de programación como el cálculo de gradientes (*backward function*).

¿Cómo ejecutar y validar tu modelo GPT-2?

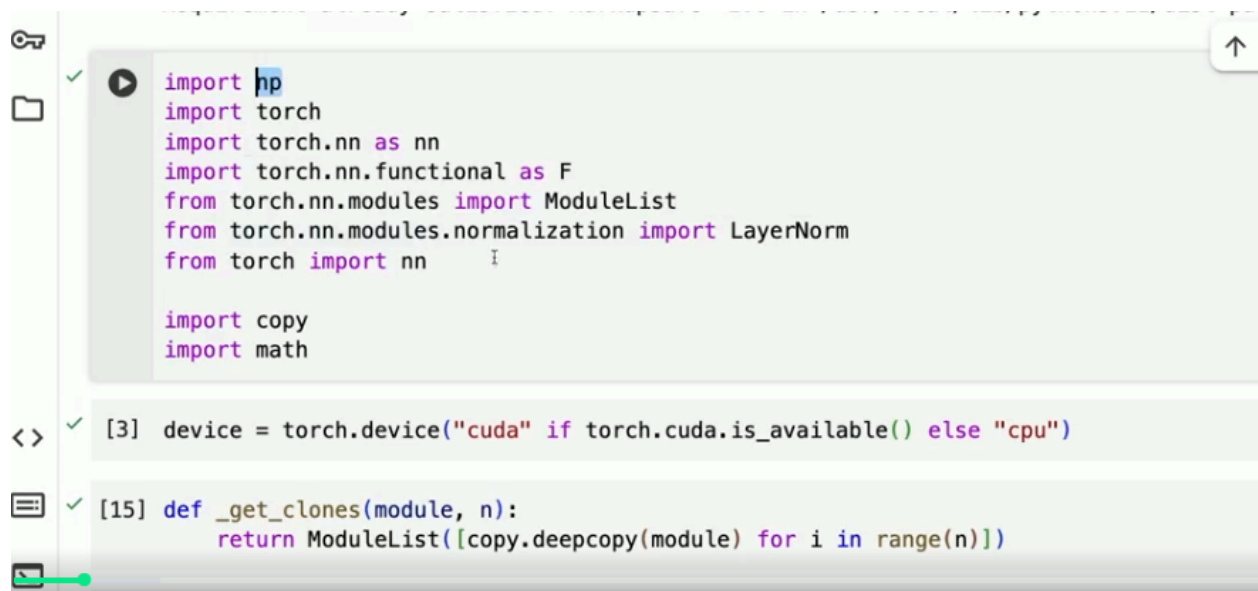
Una vez construido el GPT-2, verifica:

- La carga y ejecución de los parámetros pre-entrenados.
- Validación del ambiente GPU (CUDA).
- Uso de funciones auxiliares para duplicar módulos y realizar operaciones específicas como *split heads* y *merge heads*.

Puedes probar tu modelo dando textos básicos en inglés al *tokenizer*. El modelo generará nuevos tokens próximos, validando así tu implementación. Ten presente que GPT-2 fue entrenado para un contexto máximo de mil veinticuatro tokens y opera principalmente en inglés.

Anímate a explorar estas herramientas de inteligencia artificial y comparte en los comentarios tus experiencias o inquietudes en el proceso de construir tu propio GPT-2.

S



```

import hp
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.nn.modules import ModuleList
from torch.nn.modules.normalization import LayerNorm
from torch import nn

import copy
import math

[3] device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

[15] def _get_clones(module, n):
    return ModuleList([copy.deepcopy(module) for i in range(n)])
  
```

¡Vamos a ir “a las tripas” de la vectorización y los embeddings en LLMs! Aquí complemento lo que ya sabes con fundamentos matemáticos, propiedades geométricas y variantes modernas para oraciones/documentos y RAG. Mantengo el rigor, pero en pasos claros.

1) De tokens a vectores: la capa de embedding es una matriz aprendida

- Supón un vocabulario de tamaño V y dimensión de embedding d .
- Representa el token t como **one-hot** $x \in \{0,1\}^V \subset \{0,1\}^V$ (un 1 en el índice del token).
- La **matriz de embeddings** $E \in \mathbb{R}^{V \times d}$ contiene un vector por token (fila).
- El “lookup” es lineal: $h_t = x^T E = E[t] \in \mathbb{R}^d$. No hay magia: es una multiplicación por una matriz y una selección de fila.
- En modelos autoregresivos: a partir de h_{t-1} (más codificación posicional) pasan LL capas Transformer y se proyecta a logits $z = W_o L z = W_o h_L$ para **softmax**: $p(\text{próximo token}=i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$. La pérdida típica es entropía cruzada $-\log p(y) - \log p(y)$.
- Durante el backprop, los gradientes **ajustan directamente** las filas de E usadas: el embedding aprende a colocar tokens que ayudan a predecir el siguiente token más cerca/“alineados” con el objetivo.

Nota: en muchos LLMs se **ata** (weight tying) $W_o \approx W_{in} \approx E$ (o su traspuesta), ligando el espacio de entrada y salida para estabilidad y eficiencia de parámetros.

2) Por qué los embeddings “funcionan”: hipótesis distribucional y factorizar co-ocurrencias

Antes de Transformers, **Word2Vec**/SGNS y **GloVe** mostraron que poner juntos tokens que **co-ocurren** con contextos similares produce geometrías útiles. De hecho:

- **SGNS** (skip-gram + negative sampling) **equivale** a factorizar (aproximadamente) una matriz de **PMI desplazada**:
 $PMI(w,c) = \log p(w,c) - \log p(w) - \log p(c)$, y
 $SGNS \approx \text{factoriza } PMI(w,c) - \log p(w) - \log p(c)$. (papers.nips.cc, [ACM Digital Library](https://www.acm.org/publications/proceedings-neurips))

- GloVe optimiza a partir de **co-ocurrencias globales** para que diferencias de vectores modelen razones de probabilidades de co-ocurrencia (explica analogías lineales tipo $king - man + woman \approx queen$). (www-nlp.stanford.edu, [ACL Anthology](#), nlp.stanford.edu)

Intuición: si dos palabras ven contextos similares, sus vectores deben ser cercanos; analogías emergen porque **diferencias** de vectores codifican rasgos latentes (género, tiempo verbal, etc.).

3) Geometría real: anisotropía, “all-but-the-top” y normalizaciones

Los espacios de embeddings **no** son perfectamente isotrópicos: hay direcciones dominantes (un “vector medio” y unos pocos PCs) que “aspiran” los vectores → cosenos demasiado altos entre tokens no relacionados.

- Hallazgo empírico: **remover el componente medio** y las primeras **k componentes principales** (“All-but-the-Top”) mejora similitud/analogías y downstream. ([arXiv](#), [SciSpace](#), [Illinois Experts](#))
- En **embeddings contextualizados** (BERT/GPT), la **anisotropía** persiste pero **disminuye** en capas superiores: las capas altas son más específicas al contexto; el mismo token en contextos distintos se separa más. ([arXiv](#))

Buenas prácticas geométricas al usar embeddings para búsqueda/clustering:

- L2-normaliza (unit-norm) si vas a usar coseno; considera **centering** y, si hace falta, remover top-PCs.
- Recuerda que **rotaciones ortogonales** del espacio **no** cambian cosenos → los ejes absolutos no significan rasgos “interpretables” por sí solos.

4) “Embedding” ≠ “representación final”: estático vs contextual

- **Estático** (Word2Vec/GloVe): un vector por tipo de palabra, **sin contexto**. Ideal para analogías y recursos baratos.
- **Contextual** (BERT/GPT): parte de EE pero, tras atención multicabeza y MLPs, obtienes **un vector por ocurrencia** (token-en-contexto). Las analogías lineales clásicas son menos nítidas aquí, pero la **representación es mucho más informativa para tareas reales**. ([arXiv](#))

5) De palabra a oración/documento: pooling vs entrenamiento contrastivo

La media de tokens o el [CLS] no bastan para búsqueda semántica eficiente. Mejor:

- **SBERT**: redes siamesas con pérdidas tipo contraste/triplete para que sentencias semánticamente similares queden cercanas (acelera “semantic search” en órdenes de magnitud frente a BERT vanilla). ([arXiv](#), [ACL Anthology](#))
- Otras variantes modernas emplean **contraste** con grandes lotes, minado de negativos, etc., para obtener embeddings robustos de oraciones/párrafos.

6) Para RAG: single-vector vs multi-vector (ColBERT) y ANN a escala

- **DPR (bi-encoder)**: un vector denso por consulta y por pasaje; similitud por producto punto/coseno. Excelente velocidad, buena calidad. ([arXiv](#), [ACL Anthology](#))
- **ColBERT (late interaction)**: en lugar de colapsar a un único vector, mantiene **múltiples vectores por token** y compara con **MaxSim** (cada token de la query encuentra su mejor match en el documento). Sube calidad, coste moderado. Versiones nuevas (v2, video, etc.) siguen esta línea. ([arXiv](#), [people.eecs.berkeley.edu](#))
- **Índices ANN**: para millones-miles de millones de vectores, usa FAISS (IVF, PQ, HNSW, GPU). Es el “motor” de la mayoría de vectordbs serias. ([Engineering at Meta](#), [arXiv](#), [GitHub](#))

7) Tokenización de subpalabras: por qué BPE/Unigram importa al embedding

La calidad del embedding depende de cómo segmentas el texto:

- **BPE** (Sennrich et al.): combina pares frecuentes → maneja OOV, morfología y nombres propios razonablemente. ([arXiv](#), [ACL Anthology](#))
- **Unigram LM / SentencePiece** (Kudo): modelo probabilístico de subpalabras; permite **subword regularization** (muestrear segmentaciones) para robustez. Muy usado en LLMs modernos. ([arXiv](#))

8) Métricas y escalas: dot-product, coseno, temperatura y norma

- **Producto punto vs coseno**: si no normalizas, el producto punto mezcla ángulo y norma (palabras muy frecuentes tienden a mayor norma). El **coseno** aísla la dirección (semántica relativa).
- En generación, la **temperatura** se aplica a logits **antes** del softmax ($z/Tz/T$), no a embeddings. Pero su efecto geométrico es claro: reescala el “relieve” de la distribución; $T \downarrow$ agudiza picos (más determinismo), $T \uparrow$ aplana (más diversidad).
- En búsqueda, normalizar a norma 1 y usar coseno suele ser más estable; en recomendadores o retrieval neurales, el **dot product** con normas aprendidas también tiene ventajas (capta “confianza” vía la norma).

9) Variables y “controles” que realmente mueven la aguja

- **VV**: tamaño de vocabulario (y estrategia de subword).

- dd: dimensión del embedding (768, 1024, ...); más grande $\neq$ siempre mejor $\rightarrow$ trade-off señal/ruido y coste.
 - Pre-/post-procesado geométrico: centering, quitar top-PCs, L2-norm (mejoras gratis en similitud). ([arXiv](#))
 - Objetivo de entrenamiento: autoregresivo vs contraste/triplete (SBERT) según tarea. ([arXiv](#))
 - Índice ANN y parámetros (nlist, nprobe, PQ-m, HNSW-M/efSearch) $\rightarrow$ latencia/recall. ([arXiv](#))
 - Elección de **single-vector** (DPR) o **multi-vector** (ColBERT) según precisión/latencia/huella. ([arXiv](#))
-

Mini-ejercicios para “sentir” la geometría

1. **Isotropía:** toma embeddings preentrenados, calcula la media μ y cosenos antes/después de restarla; mide cómo cambia la distribución de cosenos (más “centrada” tras centrar). ([arXiv](#))
 2. **Pooling vs SBERT:** compara $[CLS]$, mean-pooling y un encoder tipo SBERT en STS; contrasta Spearman ρ . ([arXiv](#))
 3. **BPE vs Unigram:** entrena dos tokenizadores sobre el mismo corpus y mide OOV, longitud media de secuencia y calidad de retrieval a igual dd. ([arXiv](#))
-

Si quieres, lo llevo a código en PyTorch/HF con:

- (a) pipeline completo para *sentence embeddings* contrastivos (estilo SBERT),
- (b) construcción de un índice FAISS con búsquedas ANN,
- (c) comparación DPR vs ColBERT en un mini-corpus RAG.

O dime -xv y te doy las variables clave del sistema con sus rangos típicos y efectos.

En esta clase iremos un paso más allá y te mostraré cómo construir tu propio GPT dos desde cero en Python usando Torch y en

Google Colab. En este Google Colab vas a tener todo el código y lo primero que vamos a hacer en el Google en el Google

Colab es seleccionar nuestro tipo de sistema. Te vas en este en esta parte, le das clic, recursos, puedes ir hasta cambiar tipo de entorno

de ejecución y en este caso vamos a seleccionar GPU t cuatro. ¿Esto por qué? Porque queremos usar la GPU para ejecutar el código al

final, para que sea un poco más rápido. Lo primero que tenemos que instalar es A and Ops, Xformers y NP, estas son librerías para

programar transformers y programar otro tipo de estructuras. Esto puede tomar un rato, en este caso fue bastante raro porque tiene que instalar varias drivers

y varias librerías un poco pesadas. Luego, como mencioné, vamos a usar Torch y también vamos a usar un poco de NP o NumPy. En

este caso vamos a usar un par de módulos de Torch como el Si recuerdas en la arquitectura de GPT dos hablábamos del Norm, lo

ejecutamos y lo primero que vamos a hacer es validar en qué dispositivos está ejecutando nuestro código. Esto simplemente lo hacemos con Torch punto device,

recuerda que siempre que quieras entender o saber qué quiere, hace una función o a qué se refiere, acá en Google Colab puedes poner el

cursor y ver el código fuente o ver una pequeña documentación. Acá vamos a ejecutar y espero que nos diga CUDA, que es la tarjeta

gráfica que estamos usando. Ahí nos dice que el tipo de device es CUDA. Luego vamos, acá tenemos un par de funciones auxiliares, como mencioné

en la arquitectura de GPT, el transformer se multiplica por doce, entonces vamos a crear una función inicial auxiliar para ayudarnos a crear esos clones

de manera sencilla y no tener que estarlos poniendo a mano. Esto lo podemos hacer con el model list PyTorch, que simplemente lo que nos

ayuda a hacer es a crear esa copia, digamos, de manera segura. Esa es una función auxiliar. Para el primer bloque, hay un par de

cosas que quizás no vimos en la arquitectura puntual de GPT dos porque nos centramos en la parte más general, pero hay un par de

componentes como este, que es la convolución uno d, que nos ayuda en una parte de optimización, de optimización de la tarjeta gráfica y del

proceso. Es una parte más de ingeniería, no hay que entenderlo en su totalidad, pero te invito a usar un LLM, si quieres usarlo, de

igual forma, como todo está documentado en español, y vamos a dejar en el de este repositorio explicado punto por punto qué significa. Y si

quieres indagar un poco más, pues te invito aprender sobre Python. Luego de ejecutar este módulo de Torch con el módulo n n punto modul,

el conf uno d, vamos ir a la siguiente parte de nuestra arquitectura. Si recordamos, en la parte de la arquitectura hablamos de una feed

forward. Como mencioné, este es un tipo de red neuronal un poco más común y clásica que cuenta y consta de dos capas de convolución,

una función hello, que es una función de activación, y un dropout. El dropout normalmente se usa para que las redes neuronales no sobreaprendan y

no se queden solo con los conceptos que ven en el dataset. Es muy importante usar estas dropout de manera inteligente y siempre agregarlas en

nuestro código. De igual forma, en todos los tipos de módulos de PyTorch vamos a ver una función forward. Normalmente, esta función forward es la

que se ejecuta cuando llamamos a este módulo. Recordemos que hay una función forward y una función backward. Normalmente la backward es un proceso matemático

de diferenciación de derivadas, no vamos a entrar a detalle, pero también es un componente y en este caso PyTorch nos ayuda de manera automatizada,

no tenemos que hacer nada, simplemente con usar el n n module, él automáticamente crea ese bad word por nosotros teniendo en cuenta estas propiedades

y la función forward. Lo podríamos escribir a mano, pero pues usamos el poder de PyTorch en este caso. Este caso lo voy a ejecutar

y con eso ya tendríamos el bloque fit forward, así de sencillo. Ahora vamos a ir al componente que captó nuestra atención en la clase

anterior, que es el componente de atención. Vamos a ir un poco paso a paso en este componente puntual, porque es el que quiero que

entendamos y veamos cómo los conceptos que vimos en la clase anterior se aplica a código Python y cómo esa teoría se traduce a algo

más real. En este caso, tenemos la atención con, de nuevo, $n \times n$ model para tener toda la batería de aplicaciones y herramientas de PyTorch,

y en nuestra función de inicialización vamos a tener varios parámetros. Cada uno de estos parámetros son el d model que hace referencia a qué

dimensión va a ingresar en nuestro modelo. ¿Esa dimensión a qué hace referencia? Hace referencia a los embeddings. ¿Recuerdan que en nuestra en nuestras primeras

clases donde estábamos haciendo el ejemplo del rey, la reina y demás? Habíamos unos vectores. Esos vectores puntuales tenían una dimensión de uno por dos.

En este caso, dimensión es uno por setecientos sesenta y ocho. A eso hace referencia esa dimensión. Luego vamos a tener la cantidad de cabezas

o head de atención. Si recuerdan, la arquitectura de GPT decía multi head attention. Ese multi hace referencia porque no solo vamos a usar una,

en este caso vamos a usar de valor default doce. Luego vamos a tener un par parámetros más que son de ingeniería, digámoslo de esa

manera. Son parámetros que PyTorch necesita para su manejo interno, como lo es el contexto, que es básicamente un buffer donde se guarda información, y

luego vamos a tener d head, que es la dimensión de salida para las head de attention. ¿Cuánto queremos que salga de esas head? ¿Qué

dimensión? En este caso, sesenta y cuatro. Aparte vamos a tener dos valores booleanos que te invito luego de terminar esta clase a investigar. Uno

es el bias y el otro es el scale. El bias básicamente lo que nos dice es si queremos utilizar o no el sesgo y

el scale es si queremos utilizar la escala y la estabilidad numérica. ¿Recuerdas que en la función de atención teníamos una división de la raíz

cuadrada de d sub k ? Bueno, en este caso, como GPT dos no es un modelo tan grande, no es necesario utilizarlo en todos los

casos. En este ejemplo particular hay una forma de usarlo, pero te invito a investigar más qué pasa si lo usas, si no lo usas,

cuáles son las diferencias y demás. Acá en la inicialización de nuestro modelo, tenemos la asignación de cada una de las variables, vamos a tener

una función auxiliar del conf uno, esto era lo que te mencionaba que era más una parte de ingeniería para que todo funcione muy bien

con los logos que nos da PyTorch, luego guardamos la booleana de si queremos escalar o no, luego definimos la función softmax. Recordemos que la

función softmax es la que nos ayuda a bajar esos valores gigantes a valores entre menos uno y uno. Luego acá registramos el buffer, como

te mencioné algo de ingeniería, volvemos agregar el dropout, recordemos que el está ahí para que la red neuronal no sobreaprenda solo sobre los datos

de entrenamiento, entonces en este caso lo estamos inicializando con un cero punto uno, un valor bajo, y luego volvemos a tener un conf uno

d. Si eres un poco atento, te vas dar cuenta que tenemos uno al inicio y uno al final. Eso tiene un motivo de ser.

El del inicio lo que hace es convertir la matriz a una forma específica y el del final lo que hace es desconvertir esa matriz

en la matriz que nosotros teníamos inicialmente. Ahora vamos definir un par de funciones auxiliares, así como tuvimos la de get clones. La primera que

vamos a tener es el split heads. ¿Por qué es importante? Como te mostré, vamos a tener múltiples cabezas o head dentro de nuestra atención

por el multi head attention, Pero en algunos casos particulares, no en el caso de GPT dos, en su momento en el dos mil dieciocho,

de pronto sí, toda esa información no puede caer en una tarjeta gráfica. Entonces, lo que nosotros normalmente hacíamos era dividir el trabajo para poderse la

pasar a diferentes procesos de la tarjeta gráfica. En este caso dejé las funciones para que tú vayas, tienen un poco de ese concepto también

de paralelización que vamos a ver en clases posteriores. Básicamente, vamos a dividir cada una de estas cabezas en batches, o sea, si tenemos dos y lo queremos dividir en cuatro, pues vamos a tener cuatro batches de tres cabezas cada uno, eso es lo que hace esto. De pronto

puedes ver un poco acá de PyTorch avanzado, pero estos son simplemente funciones y pirotecnia que usamos acá para hacer lo que, para hacer un

split de las cabezas que tengamos. En este caso, al final hacemos un permute para tener la salida como la mencionamos acá. El primero será

el batch, luego será el head, luego será el sequence y luego será el feature. Ahora, sigamos a lo que nos atrae esta red de

atención y lo que vimos en nuestra clase anterior. Tenemos la función `raya_baja_attention` o `ATTN`, que recibe los tres valores que vimos en

la explicación, la `q` de query, la `k` de keys y la `v` de values. En esta función es donde pasa todo el proceso que

vimos en la clase anterior de manera visual. ¿Y cómo empieza? Recordemos, lo primero era multiplicar las queries por las keys, y eso lo vemos

acá. En este caso llamamos a la variable `scores` y utilizando `torch.matmul`, porque QYK son matrices, como lo mencionamos, `q` por KYK tiene algo

particular, que no lo mencionamos en la explicación visual, pero sí lo podías ver en la ecuación, que decía k a la t . Esa t

hace referencia a la transpuesta. Básicamente, eso lo que significa en términos prácticos es cambiar la forma de una matriz. Entonces, si nosotros teníamos, por

ejemplo, en el ejemplo de la reina y el rey, una matriz de uno por dos, la transpuesta sería una matriz de dos por uno

y nada más ya no sería en columnas, sino en fila. Eso es lo que quiere decir esta transpuesta. Hacemos la `matmul`, la multiplicación de

matrices, y luego en la ecuación tú recuerdas que estaba la división entre la raíz cuadrada de d sub k . En este caso, si recuerdas,

tenemos la variable `self.punto_scale`, que si es `true` lo aplica y sino no, en este caso no lo va aplicar porque lo tenemos

en `false` en `false`, pero acá lo puedes ver. Punto `SQRT` por la raíz cuadrada de `v` punto seis menos uno, que es la dimensión.

Size es la forma de llamar dentro de PyTorch las dimensiones, y menos uno es para obtener la última dimensión, similar a cómo funcionan las

listas en Python. Luego, vamos a tener los valores y vamos a definir un par valores más que son n_d , coma, n_s . Esos

valores puntualmente no se usan acá y te invito a investigar por qué no lo usamos. En el paper de GPT dos podrás ver el

motivo del por qué no lo agregué acá o lo podrías usar, depende de la arquitectura que quieras hacer o de los scores que quieras

obtener. Acá tenemos un IF con algo muy especial y es que normalmente en las redes neuronales y cuando uno se mete al nivel de

ingeniería, vamos a hablar de máscaras, de máscaras de las vistas, de máscaras de la tensión, de máscara de la matriz. Esos son conceptos avanzados.

En este caso, lo dejé acá como un easter egg para que las personas y ustedes en los comentarios me cuenten cómo lo implementarían o

para qué nos serviría esta función en particular. Luego de tener los scores, si recuerdan, nosotros aplicábamos la función softmax, que ya la teníamos definida

en la parte superior en nuestra inicialización acá n_n punto softmax. Entonces, simplemente se la aplicamos, scores es igual a self punto softmax scores.

Básicamente lo mismo. Luego, aplicamos el dropout, como ya te mencioné para qué sirve, es fundamental que esté ahí, y el último paso para saber

a dónde tengo que mover ese vector era multiplicar eso, es softmax de la división que ya hemos hablado por v , el valor de los

valores, el vector de los valores que se obtiene al multiplicar la matriz de pesos v por los embeddings de la transpuesta. Con esto ya

tendríamos la salida y ya tendríamos y sabríamos cómo tendríamos que modificar esos embeddings para guardar la información contextual en cada uno de ellos. Con

esto ya tendríamos definida nuestra función de atención. Es increíble ver cómo la matemática se traduce a código y cómo en algunos casos no simplemente

es copiar y pegar la fórmula. Hay que tener en cuenta las limitaciones técnicas de hardware u otras cosas que no se ven en la

matemática, sino son más aplicadas a la ingeniería. Luego de tener la función de atención, tenemos una función de merge heads, así como tenemos la

función de speed heads, en este caso las combinamos de nuevo, y luego tenemos nuestra función de forward, que es básicamente obtener los valores de

q por k por v y aplicar a estos la atención. Básicamente, ya con esto, cuando lo ejecutemos, ya tendríamos nuestra capa de atención, nuestra

multihead attention. Ahora vamos a construir el bloque de transformador. Tenemos nuestra feed forward, que la tenemos acá, y ya tenemos nuestro bloque de atención.

Nos falta simplemente añadir la slugger norm y añadir las self las atenciones en un solo bloque. Vámonos al transformer. El transformer es el más

sencillo de todos, ya que es básicamente la combinación de lo que estuvimos creando en los pasos anteriores. Entonces, el transformer block va a recibir

unos parámetros que son los que le vamos a pasar a, en este caso, a la atención y lo definimos sencillo. Simplemente decimos atención d

model siete seis seis ocho, le pasamos los parámetros. En este caso, yo le pasé que el bias sea true para que sí lo aplique.

Luego creamos la feed forward con los parámetros y luego creamos las layers de normalización, que serían estas de acá. Con esto ya prácticamente tenemos

todo, nada más nos falta crear nuestra forward, y acá es donde todo va a encajar, porque con este simple fragmento de código vamos a

poder ver de manera visual la el diagrama que mostramos en la clase anterior. Entonces, el el diagrama de la clase anterior, ¿qué nos decía?

El diagrama de la clase anterior nos decía que un GPT es, agarro los inputs, el input x, se lo sumo a, y ahí empezaba

el transformer, a la multi head attention, que ya la tenemos acá. Luego, esa multi attention, yo le aplicaba una layer norm, que la tenemos

acá, y luego a ese valor le aplicaba una feed forward, y luego le aplicaba una normalización. Acá lo podemos ver claro y sencillo. Ese

concepto y esos conceptos que vimos y esa arquitectura visual la estamos viendo clara acá. Ya con el bloque del transformador, lo único que nos

queda es crear la arquitectura completa de GPT. ¿Si recuerdan? Habíamos mencionado que GPT dos se componía de doce transformadores. Con eso, vamos a ver

cómo se construye eso. En este caso, tenemos el modelo GPT dos, que también es un módulo y que recibe un par de parámetros. El

primero de ellos es cuántos ledgers queremos tener. Recordemos cuántas veces queremos multiplicar el transformador, en este caso, doce. Podemos jugar con esto. Si recuerdas,

en una de nuestras clases anteriores hablamos del cambio de GPT dos a GPT tres y básicamente era que el modelo era mucho más grande

y aplicaba unas técnicas de normalización y demás, tú podrías intentarlo acá, podrías básicamente cambiarle el nombre a GPT tres y usar acá la cantidad

de layers que son, que son noventa y seis, y modificar el la ventana de contexto. No lo voy a hacer en este caso porque

necesitaríamos mucho más poder de cómputo que no tenemos disponible acá, pero te invito a probarlo. Aparte, vamos también a tener el contexto, la ventana

de contexto, la famosa ventana de contexto. La ventana de contexto hace referencia simplemente a cuántos tokens en el pasado yo puedo ver para saber

el contexto de una palabra. En este caso, modelo fue entrenado específicamente para mil veinticuatro toques, ese es el que le vamos a pasar. Luego

tenemos las dimensiones del modelo que ya te había mencionado son setecientas sesenta y ocho, y luego tenemos una variable que se llama vocabulary size.

Ese Vocabulary Size cincuenta mil doscientos cincuenta y siete hace referencia a la cantidad de palabras distintas que había en el dataset de entrenamiento de

GPT dos. En este caso, habían cincuenta mil doscientos cincuenta y siete palabras. Ya con esto, podemos empezar a crear nuestra arquitectura. Lo fundamental es

este bloque. Si te das cuenta, este bloque es el transformer, que por debajo crea sus atenciones, y luego tenemos que crear doce duplicaciones de

ese transformer. Si recuerdas, habíamos definido la función getclones, le pasamos block y le decimos doce, sencillo. Ya con eso diría, digamos, tenemos nuestra primera

capa. Luego, de acá para abajo, hay un par de cosas adicionales como el WTE y el WPE, otro dropout, otra layer normalization, otra linearización.

En este caso, estas embeddings lo que hacen es traducir la respuesta del de las atenciones, entonces es básicamente cumple esa función, traducir. Luego tenemos

el dropout, que ya te mencioné para qué es, luego tenemos el larger norm y luego tenemos una una capa lineal. Recordemos que al final

esto es una red neuronal, entonces necesitamos tener una capa de salida. ¿Esa capa de salida qué es? La capa de salida es, ¿cuál de

las cincuenta mil doscientos cincuenta y siete palabras que yo tengo es la que debo escoger? Por eso es una capa linear, porque es un

vector de una dimensión por el tamaño del vocabulario. Y al final tenemos una función de pérdida. La función de pérdida básicamente funciona para entrenar

el modelo. Si recuerdas, habíamos mencionado que las matrices de pesos, de valores, de quiz o de query se entrenaban dependiendo de la curada. Bueno,

la sí es calculada con base a la función de pérdida y eso actualiza esos valores. Al final tenemos un init weights, que esto básicamente

lo que hace es inicializar nuestras nuestras matrices de pesos y demás y de queries y de keys y de valores. Acá la podemos ver

y acá tenemos una función que lo que hace básicamente, si quieres indagar un poco más, es crearlo de manera uniforme, crear el con una

buena distribución esos pesos. Y al final vamos a tener la forward, que la forward es aplicar cada uno de los pasos similar a como

lo hicimos en la en el transformer. Entonces, inicialmente llega un position ID que hace referencia al ID de la posición que queremos empezar a

ver. Esto es importante y me gustaría que lo entiendan para las siguientes clases, y es hacer una verificación simplemente de si esos datos están

en el mismo device. El mismo device es la misma tarjeta gráfica o mismo chip. Recordemos que al crecer las redes neuronales, puede que necesitemos

muchísimo más cómputo, eso quiere decir que vamos a necesitar múltiples device. Entonces, este paso lo que hace es verificar y validar que al dato

que yo estoy tratando de acceder esté en la tarjeta gráfica que se está ejecutando este código. Luego vamos a hacer un un drop aplicando

la el drop out para de nuevo la el tema de el sobre entrenamiento y luego vamos a pasar nuestro input por cada uno de

los transformers y de acá para abajo ya es, digamos, todo el la traducción de lo que nos da el transformer a algo que nosotros

podamos entender que veremos más abajo. Ejecutando este bloque de código, ya tendrás literalmente programado tu GPT dos. Ahora veamos cómo lo podemos ejecutar. En

la parte inferior vas a ver que estamos importando un GPT tokenizer, recordemos que este fue el mismo que usamos en el ejemplo de las

clases anteriores, lo importamos, luego creamos nuestro modelo y luego con la siguiente línea lo que vamos a hacer es descargar los pesos entrenados de

GPT dos para PyTorch, que son de código abierto en este caso. Cuando ejecutes esto se va a demorar un poco, pero luego vas a

tener este archivo, que son los los pesos. Cuando los tengas descargados, simplemente ejecutas esta línea de código, y esto lo que va hacer es

cargar esos pesos en binario, traducirlos y cargarlos a la red neuronal que nosotros creamos. Entonces, ejecutamos la siguiente también, que se encarga de actualizar

los estados, y luego, ya en la siguiente línea vamos a poder ver cargado nuestro modelo. Entonces, acá vamos a poder ver que dice dos,

que se consiste entre un module list, que son doce transformadores. Luego, tenemos los embeddings que creamos, el dropout, la layer norm, la linear y

la cross entropy. Esto quiere decir que compiló y ejecutó correctamente nuestro modelo. Quiero que veamos un par de valores para que tengamos un poquito

de conceptualización de cuánto pesa este modelo en nuestra máquina. Para eso vamos traer el total de parámetros, lo podemos usar la función suma de

Python, y con num elements, esto es una función de PyTorch, básicamente nos traeremos la cantidad de parámetros. Luego vamos traducir esa cantidad de parámetros

a bytes. Acá vamos a asumir que estamos usando un float dieciséis, que tiene cuatro bytes, entonces lo multiplicamos por cuatro y, pues, hacemos la

operación para traducirlo a megabytes. Eso nos da que el tamaño total de GPT dos sin alteraciones es de cuatrocientas setenta y cuatro megabytes. No

es tan grande, lo podríamos ejecutar incluso en nuestro teléfono móvil. Luego vamos a traer el tokenizer y vamos hacer un pequeño contexto. En este

caso es importante tener claro que esta red neuronal se entrenó en el idioma inglés. Entonces, voy hacer un pequeño ejemplo en inglés porque no

está tan preparada para español. Entonces, me traigo el tokenizer de GPT dos en este caso y le agrego un contexto de el planeta tierra

es hermoso, y ya con eso tengo los invades necesarios para empezar a generar más tokens. Para eso vamos a usar usar la función generate.

Acá en la función generate básicamente lo que hacemos es un for loop de cuántos tokens queremos generar, acá yo le puse por default quinientos

cincuenta, pero en el ejemplo puntual que vamos a usar será de cuarenta. Esto lo que hace es un loop donde coge cada token, lo

ejecuta, lo agrega y lo vuelve a ejecutar, es como si se estuviera ejecutando así mismo, la creamos y ya podríamos ejecutar y crear nuevo

texto de la nada. Para esto vamos a ejecutar esta última celda y él va a empezar a ejecutar por debajo lo necesario para poder

obtener los tokens siguientes. Esto puede tomar un poco dependiendo de qué valor pongas en cuarenta, 10 en cuenta que si pones un valor muy

alto, vas a necesitar un poder de cómputo bastante alto. Con esto, ya en la última podemos ver cuánto tiempo le costó. Le tomó siete

punto seis nueve uno seis segundos y generó esto. The Planet Air is a beautiful place that, eso fue lo que nos generó. Cuarenta tokens.

¿Por qué cuarenta tokens? Cada token no es que sea una palabra, recordemos, como vimos, que a veces hacen subword, pero con esto ya puedes

experimentar y puedes hacer las modificaciones que quieras, puedes jugar con cualquier línea del código y experimentar. Recuerda que así es como las personas que

trabajan en OpenAI llegaron a este gran modelo.

Resumen

 **Contexto rápido:** En esta clase vas a ver los términos *Rotary Position Embedding* y *Rotary Position Encoding*. Queremos que sepas que ambos se refieren a la misma técnica: **RoPE**. No te preocupes por la diferencia en los nombres, es solo una cuestión de estilo. En la mayoría de los artículos y recursos técnicos se usa *Rotary Position Embedding*, pero el concepto es el mismo: una forma de incorporar la posición en los tokens a través de rotaciones en el espacio vectorial.

El **Rotary Position Encoding (RoPE)** es una técnica esencial para lograr que los modelos Transformers, como GPT-2, reconozcan la posición específica de cada token dentro de una secuencia. Implementar RoPE mejora notablemente la precisión del modelo reduciendo la carga computacional necesaria durante su entrenamiento.

¿Por qué los transformers necesitan RoPE?

Los Transformers no tienen una forma natural de interpretar la ubicación exacta de los tokens en una secuencia. Esto implica que para modelos sin encoding posicional podría resultar idéntica la relación entre tokens cercanos y aquellos muy separados, llevando a una mayor demanda de cómputo para el entrenamiento. RoPE soluciona esto almacenando información sobre la posición mediante el ángulo del embedding.

¿Cómo funciona RoPE en la codificación de posición?

En RoPE, cada palabra o token posee un embedding que puede visualizarse en un espacio vectorial bidimensional. Aquí es donde RoPE explota la idea del ángulo:

- Cada embedding tiene un ángulo asociado respecto al eje x.
- Este ángulo (*theta*) codifica la posición específica del token en la secuencia.
- A medida que avanzamos en la secuencia se rota el embedding según el ángulo *theta*, permitiendo al modelo retener con precisión la ubicación del token.

Además, RoPE asegura consistencia manteniendo invariantes las distancias relativas entre embeddings, siempre que la cantidad de tokens entre elementos objetivo permanezca constante.

¿Cuál es la matemática detrás de RoPE?

RoPE utiliza matrices de rotación del álgebra lineal, en particular empleando funciones trigonométricas:

- Coseno y menos seno en la primera fila.
- Seno y coseno en la segunda fila.

Esta matriz rota los vectores embeddings para codificar su posición relativa dentro de secuencias más grandes. Dado que los embeddings típicos tienen múltiples dimensiones, se dividen en pares dimensionales para aplicar matrices de rotación eficientemente.

Para reducir la carga computacional, RoPE no multiplica directamente grandes matrices, sino que ejecuta operaciones vectoriales:

- División del embedding en grupos de dos dimensiones.
- Aplicación individual de la matriz de rotación por pares.
- Operaciones vectoriales más simples (coseno, seno, suma y producto punto).

¿Cómo mejora RoPE el desempeño del modelo?

Integrar RoPE facilita que el modelo entienda con exactitud la distancia entre tokens, aumentando la calidad y velocidad del entrenamiento. La reducción rápida de la pérdida durante el entrenamiento con RoPE es un claro indicador de su eficiencia y de cómo favorece el aprendizaje más ágil del modelo.

Además, ayuda al modelo a discernir qué tan próximos están los tokens en la secuencia a través del producto punto entre vectores, facilitando interpretaciones más exactas durante el entrenamiento y la inferencia.

¿Has probado ya RoPE en tus modelos? Comparte tus resultados y dudas sobre este método efectivo para el escalamiento de Transformers.

Lo que buscamos con RoPE es codificar y guardar la información de en qué parte de la secuencia está la palabra perro, haciendo que el modelo ya no tenga que inferirlo o que lo haga de manera aleatoria.

Here's a concise yet technical explanation of Rotary Positional Embedding (RoPE), the elegant positional encoding method used in models like LLaMA and GPT-Neo!



1. What is RoPE?

RoPE encodes positional information directly into the **attention mechanism** by rotating query (Q) and key (K) vectors based on their positions. Unlike fixed sinusoidal embeddings (e.g., in original Transformers), RoPE:

- Preserves **relative positional relationships** (e.g., token at position m vs. n depends on $m-n$).
- Maintains **distance decay**: Nearby tokens attend more strongly than distant ones.

2. How It Works

For a token at position m , RoPE applies a **rotation matrix** to Q/K vectors in complex space:

- **Rotation Matrix**: $R_m = [\cos(m\theta), -\sin(m\theta); \sin(m\theta), \cos(m\theta)]$ (for each dimension pair).
- **Transformed Vectors**: $Q' = Q * R_m$, $K' = K * R_n$ (rotates by angles proportional to positions).
- **Attention Score**: $(Q' * K'^T)$ automatically encodes relative positions via rotational geometry.

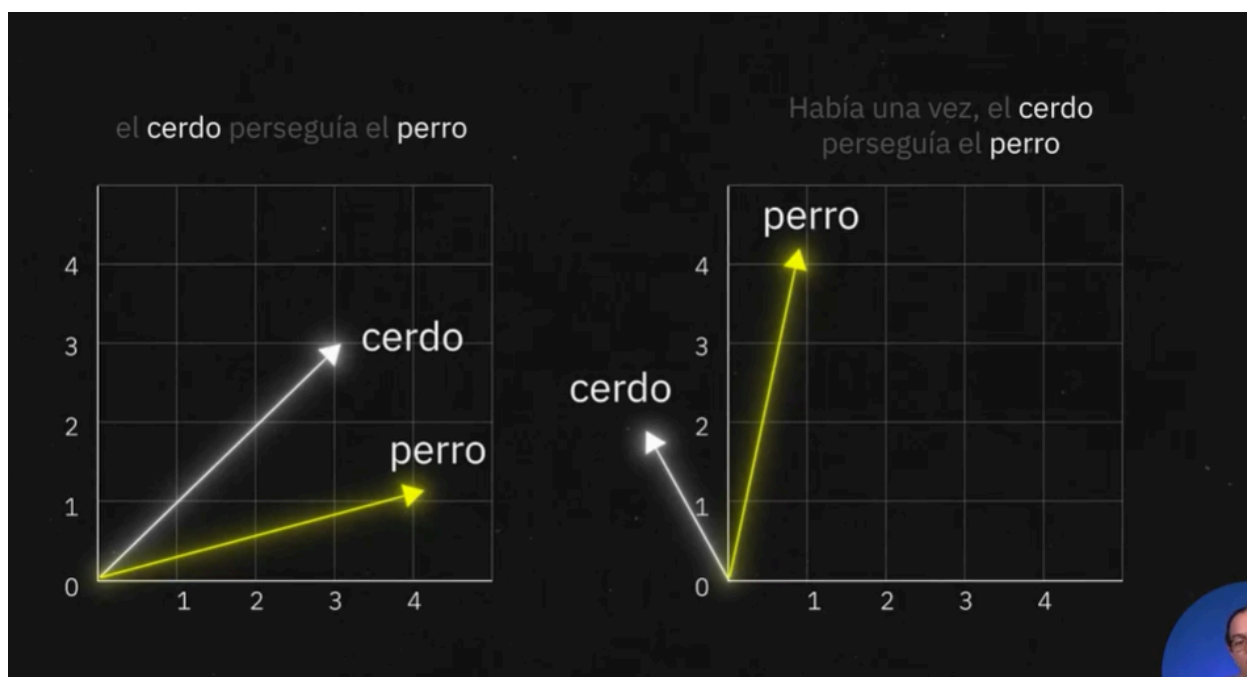
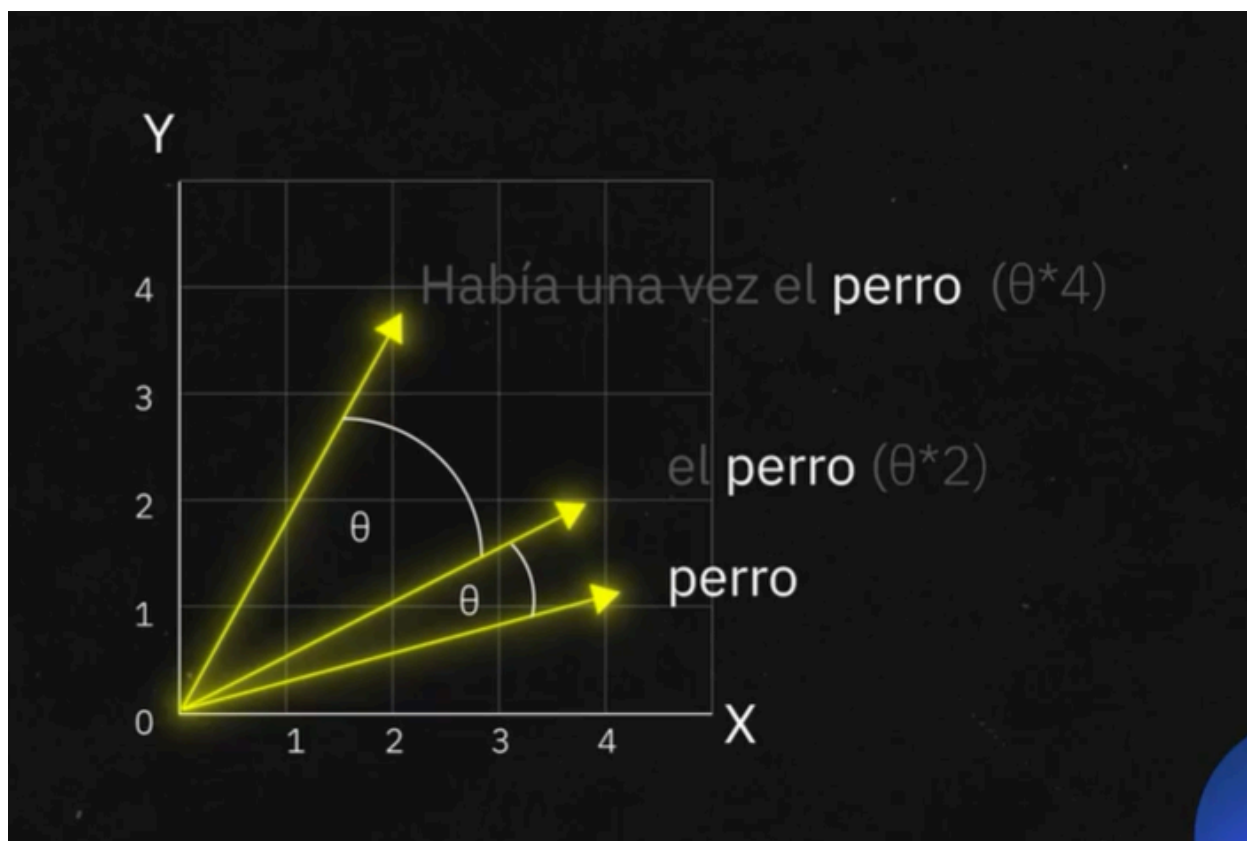
3. Key Advantages

- **Relative Position Awareness**: Captures $\text{token}_i \leftrightarrow \text{token}_{\{i+k\}}$ relationships naturally.
- **Long Sequences**: Scales better than absolute positional embeddings (e.g., no fixed max length).
- **Efficiency**: Adds minimal compute overhead vs. learned embeddings.

4. Impact on LLMs

- Used in LLaMA, PaLM, and GPT-J for better handling of long contexts.
- Enables **zero-shot length extrapolation** (training on short texts, inferring on longer ones).

Example: For $Q = [q_1, q_2]$, rotating by θ gives: $Q' = [q_1 \cos(m\theta) - q_2 \sin(m\theta), q_1 \sin(m\theta) + q_2 \cos(m\theta)] \rightarrow$ injects position *without* adding extra parameters!



🧠 RoPE (Rotary Position Embedding)

🔍 ¿Qué es RoPE?

- Técnica para codificar la **posición** de cada palabra (token) en una secuencia.
- También llamado **Rotary Position Encoding**.
- Se basa en **rotaciones** dentro de un espacio vectorial.

? ¿Por qué es necesaria?

- Los Transformers **no entienden el orden** natural de los tokens.
- Sin codificación posicional:
 - No distingue entre tokens cercanos o lejanos.
 - Mayor carga computacional.
- RoPE soluciona esto codificando la posición mediante **ángulos**.

⚙️ ¿Cómo funciona RoPE?

🧩 Paso a paso:

1. Cada token = un **vector (embedding)** en un espacio 2D.
2. Se le asigna un **ángulo (θ)** según su posición.
3. El vector se **rota** según θ para indicar su lugar en la secuencia.
4. Mantiene **distancias relativas** constantes entre tokens.

📐 Fundamento Matemático

🔄 Usa matrices de rotación:

- Fila 1: $\cos(\theta)$ y $-\sin(\theta)$
- Fila 2: $\sin(\theta)$ y $\cos(\theta)$

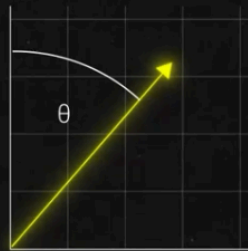
📊 Optimización:

- Divide el vector en **pares de dimensiones**.
- Aplica rotación a cada par.
- Usa operaciones simples: $\cos$, $\sin$, suma, producto punto.

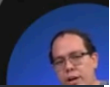
🚀 Beneficios de RoPE

- ✓ Aprendizaje más rápido
- ✓ Mejor comprensión de relaciones entre tokens
- ✓ Menor costo computacional
- ✓ Precisión en la **posición relativa**

Implementation: Matrix Multiplication



$$f_{\{q,k\}}(x_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_{\{q,k\}}^{(11)} & W_{\{q,k\}}^{(12)} \\ W_{\{q,k\}}^{(21)} & W_{\{q,k\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix}$$



Resumen

La integración del módulo Rope en GPT-2 mediante PyTorch es fundamental para mejorar el rendimiento del entrenamiento en modelos de procesamiento del lenguaje natural. En esta sesión, exploramos cómo implementar Rope paso a paso, analizando funciones auxiliares, la estructura del módulo y la adaptación necesaria en nuestro modelo GPT-2 original.

¿Qué funciones auxiliares necesitas para implementar Rope?

Para implementar Rope en GPT-2, es necesario añadir algunas funciones auxiliares en nuestro *notebook* original:

- **Exists:** Verifica la existencia de ciertos elementos.
- **Default:** Proporciona valores predeterminados en ausencia de entradas definidas.
- **Broadcast:** Moviliza tensores, matrices o vectores entre múltiples GPUs, optimizando recursos al usar más de una GPU.
- **Rotate Half:** Realiza rotaciones bidimensionales sobre los embeddings.

La función destacable es rotate half, cuyo objetivo es aplicar la rotación en dos dimensiones al embedding, conforme el ángulo theta.

¿Cómo funciona la rotación de embeddings en Rope?

El núcleo de Rope yace en aplicar rotaciones al embedding original utilizando el ángulo θ , aprendible durante el entrenamiento. Entre las funciones principales involucradas están:

- Aplicar rotación: Esta función rota embeddings usando valores θ definidos por las frecuencias o ángulos.
- Apply learner rotations: Actualiza el valor de θ durante el entrenamiento. Aquí interviene la notación de Einstein (einsum), recomendable revisar en recursos adicionales para entender su manejo algebraico y matricial.

Además, es crítico abordar el rescalado de θ , que evita que θ disminuya a cero durante el entrenamiento, asegurando estabilidad numérica.

¿Cómo implementar Rope en el mecanismo de atención de GPT-2?

Para lograr la integración efectiva de Rope en GPT-2 sigue estos pasos principales:

1. Definir dimensiones: Establece la dimensión de la rotación (por ejemplo, 32).
2. Integrar rotación en el módulo: Añadir el módulo de rotación en la función *forward*, específicamente sobre las queries (q) y keys (k), aplicando:

```
q = self.rotate.rotate_queries_or_keys(q)
```

```
k = self.rotate.rotate_queries_or_keys(k)
```

1. Actualizar pesos del modelo: Usa un archivo de pesos entrenado con Rope desde la fase inicial del modelo, garantizando compatibilidad y efectividad del método implementado.

Finalizar estos pasos permite integrar Rope eficazmente, incrementando significativamente el rendimiento de tu modelo GPT-2. Se recomienda consultar los recursos adicionales adjuntos para explorar otras variantes y optimizaciones posibles, invitándote a experimentar y enriquecer tu aprendizaje con estas herramientas.

GPT-2 + RoPE en PyTorch

1. Objetivo del Código

Implementar el modelo GPT-2 desde cero en PyTorch, integrando Rotary Positional Embeddings (RoPE) para el manejo posicional de los tokens, y generar texto usando pesos preentrenados.

2. Librerías Clave

- torch: construcción y entrenamiento del modelo.
- einops: manipulación avanzada de tensores.
- transformers: tokenización GPT-2.
- math, copy, time: funciones auxiliares.

3. Componentes del Modelo

♦ Conv1D

Simula capa densa Linear. Se usa para calcular Q, K, V, proyecciones y feedforward.

♦ FeedForward

Bloque clásico del Transformer:

- Capa lineal → GELU → Dropout → Capa lineal.

♦ RotaryEmbedding (RoPE)

Calcula posiciones rotacionales para Q y K:

- Aplica senos y cosenos escalados.
- Soporta cache e interpolación.

♦ Attention

Atención multi-head con:

- Cálculo Q, K, V.
- Aplicación de RoPE a Q y K.
- Producto escalar → Softmax → Valor ponderado.

♦ TransformerBlock

Un bloque Transformer con:

- LayerNorm → Atención → Residual → FeedForward → Residual.

♦ GPT2

Modelo completo:

- Embeddings: tokens (wte) + posiciones (wpe).
- 12 bloques Transformer.
- Salida Linear.
- Pérdida: CrossEntropy.

4. Integración de RoPE

- rotate_half, apply_rotary_emb: funciones matemáticas para aplicar RoPE.
- Aplicadas antes del cálculo de atención (Q, K).
- Se encapsula en clase RotaryEmbedding.

5. Descarga y Carga de Pesos

- Descarga pesos desde Hugging Face (curl).
- Ajuste de nombres de parámetros (mlp → feedforward).
- Carga selectiva de parámetros compatibles (state_dict).

6. Generación de Texto

♦ Tokenización

```
tokenizer.encode("The planet earth is a beautiful")
```

♦ Función generate(context, ntok=40)

- Genera texto token a token.

- Aplica top-k sampling (mantiene los 10 logits más altos).
- Muestra texto generado + tiempo de inferencia.



7. Resultado Final

- Imprime tamaño del modelo en MB.
- Muestra texto generado.
- Mide tiempo de inferencia (time.time()).



8. Ideas Clave

- RoPE reemplaza embeddings posicionales clásicos.
- Se integran directamente en el espacio de atención (Q, K).
- Código modular: cada componente refleja arquitectura GPT-2 estándar.
- Útil para entender modelos desde cero y modificaciones posicionales.

2

Responder

Reportar



Juan Mejía

student • hace un mes

¿Podrían los compiladores de deep-learning (p. ej. TVM, XLA) automatizar la generación de kernels *IO-aware* como FlashAttention para otras operaciones —batch-norm, convoluciones dilatadas, etc.— y cuáles son los principales desafíos (p. ej. modelado preciso de la jerarquía de memoria, portabilidad entre GPUs/TPUs, o la coordinación con recomputación en backward)?

Resumen

Las leyes de escalado y los modelos multimodales son esenciales en el ámbito actual de la inteligencia artificial. Aumentar los parámetros, la cantidad de datos y la capacidad computacional mejora el desempeño, aunque con retornos decrecientes tras cierto punto. A su vez, los modelos multimodales combinan diferentes tipos de información, permitiendo aplicaciones avanzadas en diversos contextos.

¿Qué son las leyes de escalado?

Las leyes de escalado revelan cómo mejora previsiblemente el rendimiento de un modelo al incrementar ciertos recursos clave. Estos recursos son:

- Parámetros del modelo.
- Datos disponibles para entrenamiento.
- Capacidad computacional.

Es clave equilibrarlos: un modelo amplio sin suficientes datos o al revés implica desperdicio de potencial. OpenAI comprobó esto con GPT-3: más parámetros y datos mejoraron consistentemente tareas de generación y comprensión textual, aunque

presentar retornos cada vez menores al aumentar repetidamente los recursos invertidos.

La relación entre el rendimiento y los recursos sigue una ley matemática conocida como ley de potencia, mostrando cómo al aumentar recursos específicos, los errores o pérdidas disminuyen con predictibilidad.

¿Por qué son importantes los modelos multimodales?

Combinar múltiples modalidades de datos, como textos, imágenes, audio y videos, permite a los modelos multimodales tener una comprensión más robusta y contextual del mundo. Algunos ejemplos actuales significativos incluyen:

- CLIP (OpenAI): asocia imágenes a descripciones textuales, permitiendo búsquedas visuales simples mediante lenguaje natural.
- DALL-E (OpenAI): genera imágenes coherentes a partir de descripciones claras, posibilitando aplicaciones creativas en diseño gráfico.
- Flamingo (DeepMind): destaca por su aprendizaje rápido y generación de conocimiento usando poquísimos ejemplos, integrando texto, imágenes, audio y video.

Entre sus ventajas destacan una comprensión contextual mejorada y calidad alta en contenido generado. Sin embargo, implican desafíos importantes como:

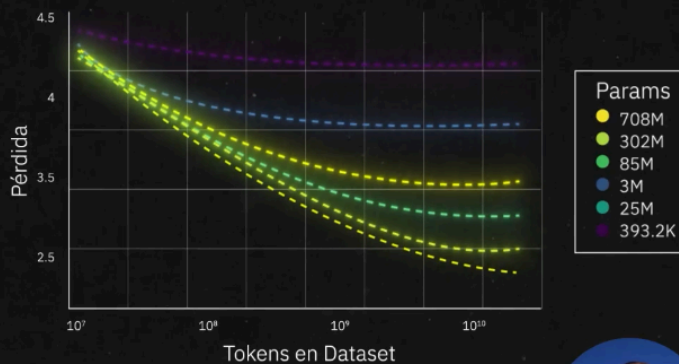
- Diseñar arquitecturas especializadas por tipo de datos.
- Alto costo computacional.
- Métodos específicos de preprocesamiento, fusión y alineación modal para representación coherente.

¿Cuáles son las aplicaciones prácticas de la multimodalidad?

Actualmente, existen aplicaciones reales muy diversas que ilustran la importancia de estos modelos. Ejemplo concreto es la aplicación móvil Chat LLM, con modelos como O1, que permite cargar diferentes tipos de datos (texto, imágenes, videos, audios) y procesarlos integrado con aprendizaje reforzado para dar respuestas contextualizadas y precisas. Otros modelos, como O3 mini, no poseen capacidades multimodales, lo cual es fundamental considerar al seleccionar tecnologías para proyectos específicos.

Experimentar personalmente con estas tecnologías puede aportar experiencia valiosa sobre su potencial y limitaciones en distintos escenarios prácticos. Por ello, se recomienda explorar diferentes LLM, observando cómo procesan modalidades variadas de información.

Leyes de Escalado



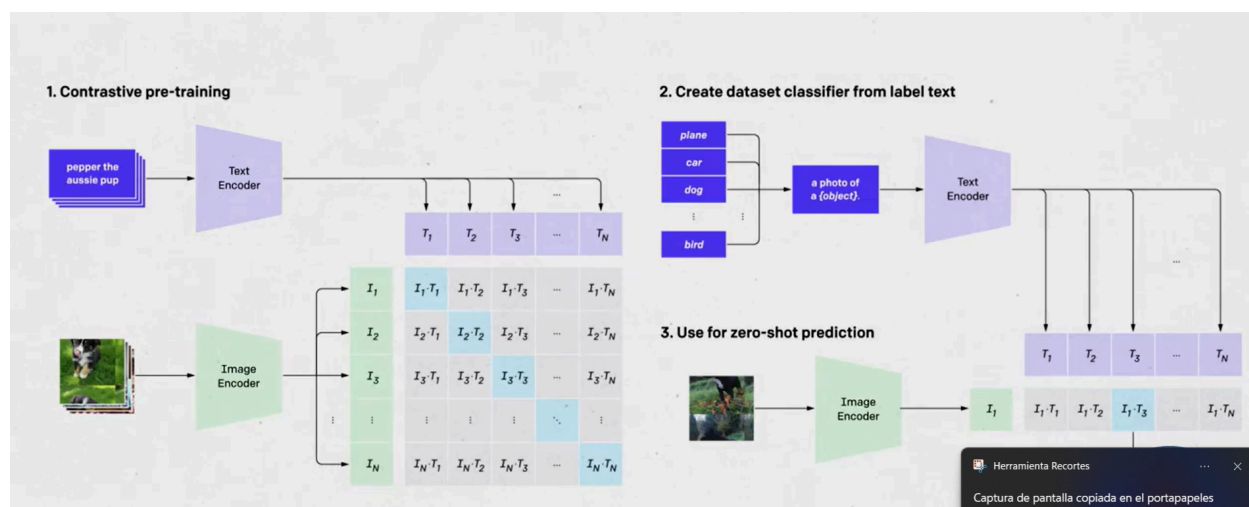
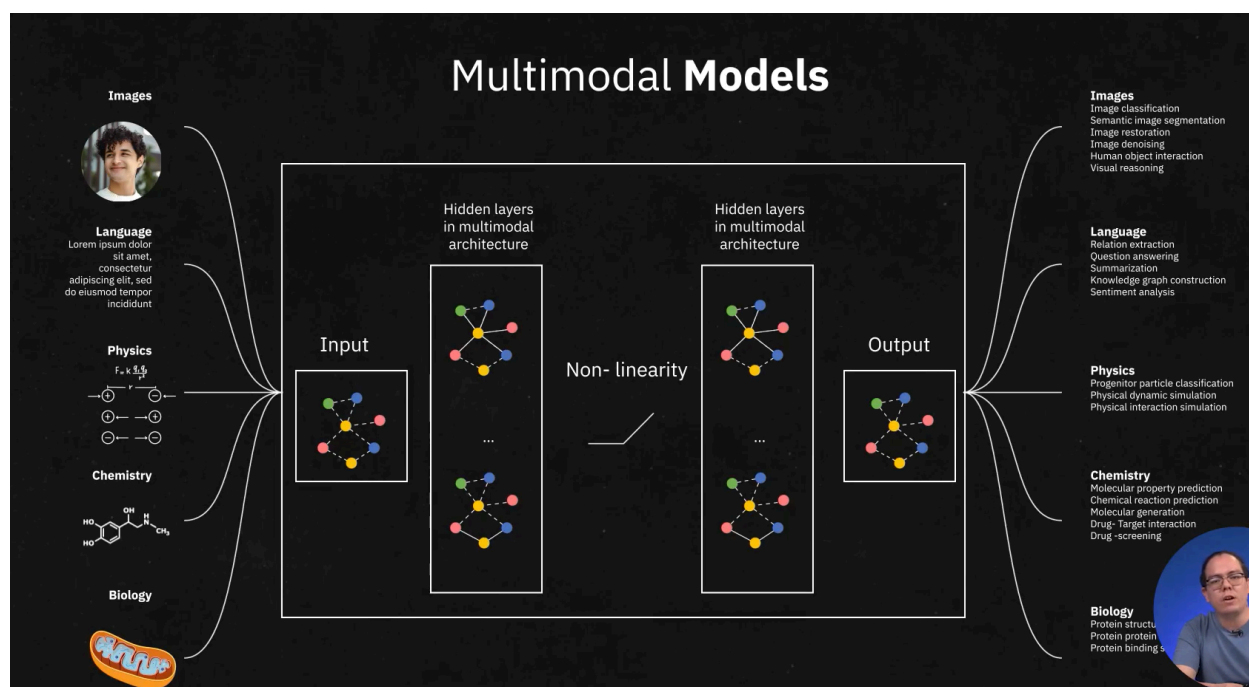
Investigando un poco sobre un model multimodal como Flamingo, resulta que en vez de usar un Transformer (como GPT) utiliza algo llamado Perceiver Resampler, que básicamente resume entradas largas (como una imagen) en un conjunto fijo de tokens útiles (latent tokens), usando cross-attention, que —a diferencia del self-attention que utiliza el Transformer, donde Q, K y V provienen del mismo conjunto de vectores—, Q viene de un conjunto (esos latent tokens) y K,V vienen de otro (la imagen de entrada).

Que son las leyes de escalado

La ley de escalado dice que el modelo mejora al aumentar los parámetros, datos o costo de computo de un modelo. Pero para una eficiencia optima es mejor escalar los tres coordinadamente

Que son los modelos multimodales

Son modelos entrenados con varios tipos de datos como texto, imágenes, audio, video, etc. Para lograr esto en su arquitectura se necesita unificar cada tipo de dato con codificadores, después ya codificados se ponen en el embedding y aquí el modelo puede entender todos los tipos de dato. Esto les permite tener mejores resultados y poder ser mas sofisticados y robustos para usos reales



¡Vamos a implementarlo “de verdad” en PyTorch sobre GPT-2 (Hugging Face), con mates, detalles de ingeniería y un parche limpio al módulo de atención!

1) Qué es RoPE (rotary positional embeddings), en 2 fórmulas

Para un head_dim par d_{hd} , emparejamos las dimensiones $(2i, 2i+1)$ y rotamos cada pareja con un ángulo que depende de la posición p y de una frecuencia ω_i .

- Frecuencias (base típica 10,000):

$$\omega_i = \text{base}^{2i/d_h} \Rightarrow \theta_{p,i} = p \cdot \omega_i \quad ; = \quad ; \quad \frac{1}{\text{base}^{2i/d_h}} \quad \Leftrightarrow \quad \theta_{p,i} \quad ; = \quad ; \quad p \cdot \omega_i$$

- Rotación de un vector 2D (x, y) por θ :

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

- Aplicamos la misma rotación a Q y K por-posición, por-par de dims y por-cabeza:

$$Q' = Q \odot \cos\Theta + \text{rot}(Q) \odot \sin\Theta, K' = K \odot \cos\Theta + \text{rot}(K) \odot \sin\Theta$$

$$Q' = Q \odot \cos\Theta + \text{rot}(Q) \odot \sin\Theta \quad ; + \quad ; \quad K' = K \odot \cos\Theta + \text{rot}(K) \odot \sin\Theta$$

donde $\text{rot}(x_{2i}, x_{2i+1}) = (-x_{2i+1}, x_{2i})$ y Θ es el tensor de ángulos expandido a la forma de Q, K, V .

Claves:

- RoPE codifica posición multiplicativamente en el espacio de Q/K (no suma un vector posicional).
- Se elimina la necesidad de wpe (positional embeddings aprendidos de GPT-2) o se deja a cero.

2) Plan de integración en GPT-2 (HF Transformers)

1. Cargamos GPT-2.

2. Anulamos la contribución posicional wpe (poner pesos a 0 y congelar).
 3. Insertamos RoPE justo después de proyectar a Q, K y antes del producto QK^T .
 4. Mantener shape estándar: $(batch, seq, n_heads, head_dim)$ o $(batch, n_heads, seq, head_dim)$ según convenga.
 5. Cache KV y máscara causal siguen igual; RoPE sólo toca Q,K.
-

3) Código completo (parche seguro)



Requiere: `pip install transformers torch`

```
import math
import torch
import torch.nn as nn
from transformers import AutoModelForCausalLM, AutoTokenizer
import types

# ----- Utilidades RoPE -----
class RotaryEmbedding(nn.Module):
    """
    Implementación estándar de RoPE:
    - Precalcula inv_freq (frecuencias) en función de head_dim y base.
    - Genera cos/sin por posición a demanda (con cache simple).
    Soporta secuencias > max_seq_len inicial via recálculo on-the-fly.
    """
    def __init__(self, head_dim: int, base: float = 10000.0, max_seq_len: int =
2048, device=None, dtype=None):
        super().__init__()
        assert head_dim % 2 == 0, "head_dim de RoPE debe ser par."
        self.head_dim = head_dim
        self.base = base
        inv_freq = 1.0 / (base ** (torch.arange(0, head_dim, 2, device=device,
dtype=torch.float32) / head_dim))
        self.register_buffer("inv_freq", inv_freq, persistent=False)
```

```

self._seq_cached = 0
self.register_buffer("cos_cached", None, persistent=False)
self.register_buffer("sin_cached", None, persistent=False)

def _build_cache(self, seq_len: int, device, dtype):
    # posiciones [0..seq_len-1]
    t = torch.arange(seq_len, device=device, dtype=self.inv_freq.dtype)
    # ángulos: [seq_len, head_dim/2]
    freqs = torch.einsum("n,d->nd", t, self.inv_freq) # (seq, head_dim/2)
    # Interleave para construir [seq, head_dim] duplicando cos/sin en
    (2i,2i+1)
    emb = torch.cat([freqs, freqs], dim=-1) # (seq, head_dim)
    cos = emb.cos().to(dtype=dtype)
    sin = emb.sin().to(dtype=dtype)
    self.cos_cached = cos[:, None, None, :] # (seq, 1, 1, head_dim)
    self.sin_cached = sin[:, None, None, :] # (seq, 1, 1, head_dim)
    self._seq_cached = seq_len

def forward(self, q, k, seq_start=0):
    """
    q, k: (batch, seq, n_heads, head_dim) o (batch, n_heads, seq, head_dim)
    Soportamos ambas: detectamos dónde está seq.
    """
    if q.dim() != 4:
        raise ValueError("Esperaba q,k con 4D")

    # Acomodar shapes para indexar por posición
    # Detectamos si la dimensión seq es 1 (batch, seq, heads, dim) o 2
    (batch, heads, seq, dim)
    if q.shape[1] == k.shape[1] and q.shape[1] != q.shape[2]:
        # (B, S, H, D)
        seq_dim = 1
        B, S, H, D = q.shape
        q_ = q
        k_ = k
    else:
        # (B, H, S, D) -> transponer a (B, S, H, D)
        seq_dim = 2
        B, H, S, D = q.shape
        q_ = q.transpose(1, 2) # (B, S, H, D)
        k_ = k.transpose(1, 2)

    device, dtype = q_.device, q_.dtype
    if (self.cos_cached is None) or (self._seq_cached < (seq_start +
q_.size(1))):

```

```

        self._build_cache(seq_len=seq_start + q_.size(1), device=device,
dtype=dtype)

```

```

cos = self.cos_cached[seq_start:seq_start + q_.size(1)] # (S,1,1,D)
sin = self.sin_cached[seq_start:seq_start + q_.size(1)] # (S,1,1,D)

```

```

q_rot = apply_rotary(q_, cos, sin)
k_rot = apply_rotary(k_, cos, sin)

```

```

if seq_dim == 2:
    # devolver a (B, H, S, D)
    q_rot = q_rot.transpose(1, 2)
    k_rot = k_rot.transpose(1, 2)
    return q_rot, k_rot

```

```

def rotate_half(x: torch.Tensor) -> torch.Tensor:
    # (x1, x2) -> (-x2, x1) por parejas de dims
    x1, x2 = x[..., ::2], x[..., 1::2]
    return torch.stack((-x2, x1), dim=-1).flatten(-2)

```

```

def apply_rotary(x: torch.Tensor, cos: torch.Tensor, sin: torch.Tensor) ->
torch.Tensor:
    # x: (B,S,H,D), cos/sin: (S,1,1,D) broadcast por batch y heads
    return (x * cos) + (rotate_half(x) * sin)

```

----- Parche de atención GPT-2 para insertar RoPE -----

```

def patch_gpt2_with_rope(model, rope_base=10000.0, max_seq_len=2048):
    """

```

```

    Reemplaza el forward de cada GPT2Attention para:
    - Sustituir pos embeddings aprendidos por rotatorios (RoPE) en Q,K.
    - Mantener todo lo demás igual.
    Además, pone wpe.weight a cero y lo congela.
    """

```

```

    # 1) Anular wpe
    if hasattr(model.transformer, "wpe"):
        with torch.no_grad():
            model.transformer.wpe.weight.zero_()
            model.transformer.wpe.requires_grad_(False)

```

```

    # 2) Para cada bloque, añadimos RotaryEmbedding y redefinimos forward de
    atención
    for block in model.transformer.h:

```

```

    attn = block.attn
    n_heads = attn.num_heads
    head_dim = attn.head_dim # usualmente embed_dim // n_heads
    rope = RotaryEmbedding(head_dim=head_dim, base=rope_base,
max_seq_len=max_seq_len)
    attn.rope = rope # adjuntamos el módulo

    # Guardamos el forward original por si se quiere consultar
    attn._orig_forward = attn.forward

    def rope_forward(self, hidden_states, layer_past=None,
attention_mask=None, head_mask=None,
                        use_cache=False, output_attentions=False):
        # Este forward replica el de GPT2Attention (transformers) pero
insertando RoPE en Q,K
        # Referencia: HF GPT2Attention -> c_attn (proyección), split heads,
attn, c_proj
        # 1) Proyección a Q,K,V
        query, key, value = self.c_attn(hidden_states).split(self.split_size,
dim=2)

    def shape(x):
        new_x_shape = x.size()[:-1] + (self.num_heads, self.head_dim)
        return x.view(*new_x_shape)

    def unshape(x):
        new_x_shape = x.size()[:-2] + (self.num_heads * self.head_dim,)
        return x.view(*new_x_shape)

    query = shape(query) # (B,S,H,D)
    key = shape(key) # (B,S,H,D)
    value = shape(value) # (B,S,H,D)

    # 2) Aplicar RoPE a Q y K (seq_start para cache incremental)
    seq_start = 0
    if layer_past is not None:
        # layer_past: ((past_key), (past_value)) en forma (B,H,S_past,D)
        # como key/value actuales están en (B,S,H,D), al concatenar, la
posición de inicio es S_past
        seq_start = layer_past[0].size(-2) # past_key: (B,H,S_past,D)

    # GPT2 usa (B,S,H,D) -> nuestra RotaryEmbedding lo soporta
    query, key = self.rope(query, key, seq_start=seq_start)

    # 3) Transponer a (B,H,S,D) para cálculo de atención como en HF

```

```

query = query.permute(0, 2, 1, 3)
key   = key.permute(0, 2, 1, 3)
value = value.permute(0, 2, 1, 3)

if layer_past is not None:
    past_key, past_value = layer_past
    key   = torch.cat((past_key, key), dim=-2)
    value = torch.cat((past_value, value), dim=-2)

if use_cache:
    present = (key, value)
else:
    present = None

# 4) Atención escalada
attn_scores = torch.matmul(query, key.transpose(-1, -2))
attn_scores = attn_scores / math.sqrt(self.head_dim)

if attention_mask is not None:
    attn_scores = attn_scores + attention_mask # máscara causal ya
precomputada en HF

attn_probs = nn.functional.softmax(attn_scores, dim=-1)
attn_probs = self.attn_dropout(attn_probs)

attn_output = torch.matmul(attn_probs, value) # (B,H,S,D)
attn_output = attn_output.permute(0, 2, 1, 3).contiguous() #
(B,S,H,D)
attn_output = unshape(attn_output) # (B,S,E)
attn_output = self.c_proj(attn_output)
attn_output = self.resid_dropout(attn_output)

if output_attentions:
    return attn_output, present, attn_probs
else:
    return attn_output, present

# bind del nuevo forward
attn.forward = types.MethodType(rope_forward, attn)

return model

# ----- Ejemplo de uso -----
if __name__ == "__main__":

```

```

tok = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")
model = patch_gpt2_with_rope(model, rope_base=10000.0, max_seq_len=4096)
model.eval()

prompt = "En un futuro cercano,"
inputs = tok(prompt, return_tensors="pt")
with torch.no_grad():
    out = model.generate(**inputs, max_length=40, do_sample=False)
print(tok.decode(out[0], skip_special_tokens=True))

```

Notas prácticas

- `head_dim` par: RoPE rota por parejas; si cambias `n_heads/hidden_size`, asegúrate de que `head_dim` sea par.
- `wpe` a cero: así evitas “doble” posicional (`learned` + RoPE).
- KV-cache: usamos `seq_start = past_len` para que los ángulos continúen correctamente en generación incremental.
- Más contexto: aumenta `max_seq_len` (y considera técnicas como NTK scaling, YaRN, xPos) si vas a 8k-32k tokens.

4) Comprobaciones y debugging

- Sanity check (sincrónico): con `do_sample=False` y un `seed` fijo, la salida debe ser determinista.
- Gradientes: en entrenamiento, RoPE no añade parámetros; sólo afecta el flujo hacia Q/K.
- Velocidad: el costo de `cos/sin` es mínimo (cacheado). Si cambias `seq_len` frecuentemente, deja que `_build_cache` recalcule.

5) Variantes y mejoras

- LongRoPE / NTK-aware RoPE: reescala $w_i \omega_i$ con un factor dependiente de la longitud objetivo para mantener estabilidad en contextos largos.
- Partial RoPE: aplicar rotación sólo a una fracción de dims del head (útil en mezclas con ALiBi, etc.).
- ALiBi: sesgo lineal por distancia en lugar de embeddings posicionales; combinarlo con RoPE no es estándar pero hay trabajos híbridos.

Si quieres, te dejo una versión entrenable mínima (LM head + causal loss) o lo adapto a GPT-J/GPT-NeoX. También puedo convertir esto en un parche reutilizable para cualquier `AutoModelForCausalLM` con atención multi-cabeza estándar.

Resumen

Entrenar un modelo de lenguaje grande (*Large Language Model*, LLM) implica un proceso extenso que combina grandes volúmenes de información, potente infraestructura tecnológica y variadas técnicas de aprendizaje automático. Comprender cómo se llevan a cabo estos pasos ayuda a aprovechar al máximo el potencial de un LLM para resolver distintos problemas o tareas específicas.

¿Qué es y por qué importa el corpus de entrenamiento?

El conjunto de entrenamiento o corpus es la base esencial del desarrollo de un LLM. Este dataset determina directamente qué problemas puede ayudar a resolver nuestro modelo.

- El corpus está compuesto por textos recolectados, generalmente vía *web scraping*.
- Gigantes tecnológicos como Google recolectan datos por años para crear datasets internos cuidadosamente curados.
- Para tener datasets de calidad, se requiere infraestructura tecnológica robusta y habilidades humanas para revisión y limpieza.

Este corpus puede llegar a tener tamaños inmensos. Por ejemplo, el modelo Llama cuatro de Meta utiliza cerca de treinta trillones de tokens, equivalentes aproximadamente a ciento nueve terabytes de información textual.

¿Cómo se calcula el tamaño de un corpus?

Para estimar el tamaño, se considera un token promedio de cuatro bytes. Treinta trillones de tokens resultan así en alrededor de 109 terabytes de datos textuales.

- Es importante eliminar duplicados y contenido nocivo del corpus, asegurando la calidad necesaria para entrenar eficazmente el modelo.

¿Qué es la tokenización y para qué sirve?

La tokenización convierte el corpus en unidades mínimas (tokens) que el modelo pueda procesar. Esta transformación es crucial:

- Tokens podrían ser palabras completas, sílabas o letras individuales.
- Se utilizan herramientas como TikToker de Hugging Face.

De esta forma, el corpus de entrenamiento es interpretado adecuadamente por el modelo.

¿Cómo se construye la arquitectura del LLM?

Una vez tokenizados los datos, seleccionamos o desarrollamos la arquitectura a usar:

- La arquitectura varía según el uso específico deseado.
- Ejemplo: GPT-2 consiste principalmente en capas de atención y capas de *multilayer perceptron* apiladas.
- Empresas como Meta hacen pública su arquitectura (*open source*), facilitando su replicación y adaptación.

¿En qué consiste el entrenamiento intensivo del modelo?

El entrenamiento computacional es la fase más costosa y prolongada:

- Requiere miles de GPUs o TPUs distribuidos en centros especializados de procesamiento.
- Cada GPU procesa fragmentos del corpus y reporta resultados, errores y ajustes al servidor central.
- Este proceso es altamente repetitivo y dura mucho tiempo, dependiendo del volumen de datos y tamaño del LLM.

¿Qué técnicas se utilizan tras el entrenamiento inicial?

Al finalizar el entrenamiento básico, el LLM aún es general. Para tareas específicas, se ejecuta una fase adicional conocida como *fine tuning*:

- Consiste en reentrenar parcialmente el modelo con datos específicos.
- Ejemplos prácticos: adaptación con imágenes médicas o consultas financieras.
- El tiempo de ajuste es considerablemente menor al entrenamiento inicial.

¿Qué es el RLHF y cómo mejora el modelo?

El *Reinforcement Learning with Human Feedback* (RLHF) introduce retroalimentación humana para ajustar el modelo según nuestras necesidades y expectativas:

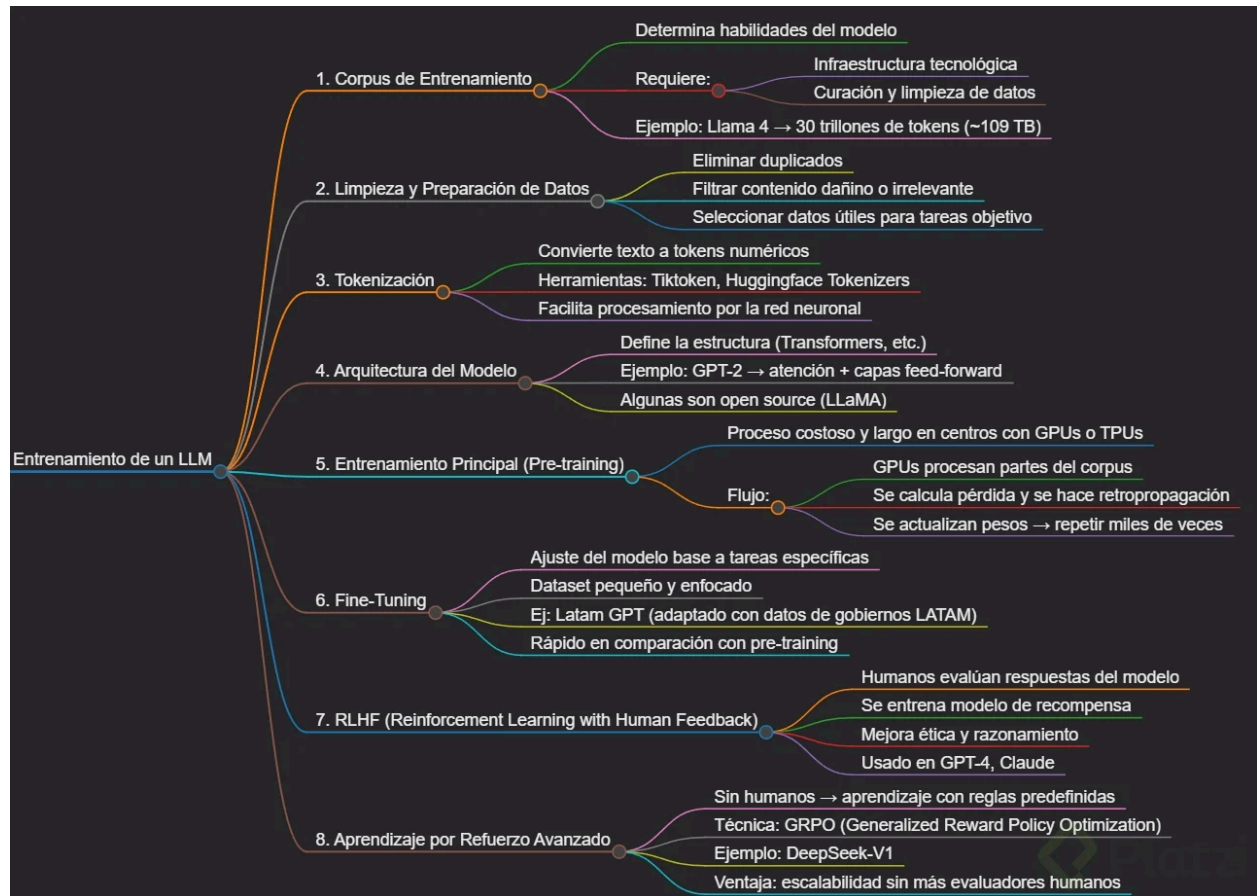
- Humanos evalúan respuestas del modelo, otorgando feedback y premiando conductas correctas.
- Transformación notable: salto de GPT-3 a GPT-4 gracias a la retroalimentación humana implementada.

¿Qué ventajas ofrece el aprendizaje por refuerzo no supervisado?

Para reducir el esfuerzo humano, se emplean técnicas donde el modelo aprende por sí mismo, basándose en reglas predefinidas:

- Aprendizaje offline: El modelo adquiere comportamientos sin la intervención directa humana.
- Ejemplo de éxito: DeepSeek R1, haciendo que el modelo aprenda de manera autónoma, mediante técnicas avanzadas como GRPO.

¿Tienes dudas específicas sobre alguna etapa de este proceso tan interesante?
¡Déjanos tus comentarios y conversemos al respecto!



Dejo mis apuntes! Lo trate de hacer lo mas fácil y entendible posible.

Pasos de un Modelo de Lenguaje Grande (Large Language Modal):

1. Crear el corpus.

El corpus es toda la información con el que el modelo va a ser entrenado. Para tener esta información se necesitan varios procesos para tener datos de entrenamiento de calidad.

En Gemini por ejemplo se hizo web scraping para obtener información de internet.

Para que sea de calidad por ejemplo se limpia la información potencialmente peligrosa, se eliminan duplicados e información que no tenga que ver con el propósito del LLM, filtrados de sesgos, eliminación de información personal o formateo de datos.

2. Tokenizar todo el Corpus.

Convertir en tokens todas las palabras del Corpus para convertirla a vectores (números) que el modelo pueda entender y procesar.

3. Armar la arquitectura del LLM.

Aquí podemos usar diferentes tipos de arquitecturas acorde a el objetivo del LLM.

Como la de Codificador-Decodificador usado para transformar texto, Codificador para solo analizar y comprender el texto y la de Decodificador usada para producir texto prediciendo la palabra siguiente.

4. Entrenar el modelo.

Ahora con nuestra arquitectura de LLM entrenamos el modelo con todos los tokens del corpus. Esto se hace en datacenters en varias partes del mundo con el poder de computo necesario.

Se usan técnicas como el Reinforcement Learning from Human Feedback o Group Relative Policy Optimization para entrenar al modelo a dar respuestas correctas.

Con RLHF se usan humanos para decirle al modelo que respuestas están correctas y cuales no y un modelo de recompensa usa esta información para calificar al modelo.

Con GRPO, el modelo se entrena al comparar su desempeño (calificado con criterios como la precisión o la relevancia) con el desempeño promedio de todo un grupo de modelos. Si es mejor que el promedio, recibe un incentivo para seguir mejorando.

5. Fine-Tuning.

El resultado hasta aquí es un modelo ya entrenado con mucha información pero de forma general. Si se requiere especializar el modelo se entrena de nuevo el modelo con una base de datos enfocada en ese tema. Así logramos que el LLM tenga alguna especialización.

- Requiere revisión y limpieza humana.
- Empresas como Google y Meta crean sus propios datasets curados.

Ejemplo real: LLaMA 4 (Meta) usa 30 billones de tokens = 109 terabytes de texto.



3. TAMAÑO DEL CORPUS

Cómo se mide:

- 1 token $\approx$ 4 bytes
- 30 billones de tokens $\approx$ 109 terabytes

Objetivo: Eliminar contenido duplicado o dañino para mejorar calidad.



4. TOKENIZACIÓN

¿Qué es? Proceso de dividir el texto en unidades mínimas (*tokens*) para que el modelo pueda interpretarlo.

Tipos de tokens:

- Palabras completas
- Sílabas
- Letras individuales

Herramientas comunes: TikToken (Hugging Face)



5. ARQUITECTURA DEL MODELO

¿Qué implica? Diseñar la estructura del modelo según su uso.

Ejemplo: GPT-2 usa:

- Capas de atención
- Redes neuronales MLP (Multilayer Perceptron)

Dato útil: Algunas arquitecturas son open source (como las de Meta).



6. ENTRENAMIENTO INTENSIVO

Fase más costosa y prolongada.

Requiere:

- Miles de GPUs o TPUs
- Centros de datos especializados

Proceso:

- Cada GPU procesa una parte del corpus
- Se envían ajustes al servidor central
- Dura semanas o meses



7. FINE TUNING

¿Qué es? Ajuste del modelo para tareas específicas.

Ejemplos de aplicación:

- Diagnóstico médico
- Finanzas

Ventaja: Más rápido que el entrenamiento inicial.



8. RLHF (Reinforcement Learning with Human Feedback)

¿Qué hace? Humanos corrigen respuestas del modelo y dan retroalimentación.

Beneficio: Mejora la calidad del modelo.

Ejemplo notable: Transición de GPT-3 a GPT-4.



9. APRENDIZAJE POR REFUERZO NO SUPERVISADO

¿Cómo funciona? El modelo aprende solo, sin intervención humana directa.

Técnicas:

- Reglas predefinidas
- Aprendizaje offline

****Ejemplo destacado:****DeepSeek R1 usando técnicas como GRPO.

G

Resumen

Imagina poder optimizar el rendimiento de los modelos de lenguaje, reduciendo significativamente el tiempo y dinero invertidos en su entrenamiento. Esto es posible gracias al enfoque conocido como Mixture of Experts (MOE), una técnica innovadora que permite crear modelos especializados en tareas concretas, tal como sucede en nuestro cerebro con el aprendizaje especializado.

¿qué es exactamente un modelo Mixture of Experts (MOE)?

Los modelos Mixture of Experts (MOE) son versiones optimizadas de los conocidos transformadores, cuya principal característica es dividir el aprendizaje en diversas especializaciones pequeñas (expertos). En vez de entrenar una inmensa red general que responda todo tipo de preguntas, se entrenan múltiples modelos más pequeños, cada uno especializándose en áreas específicas como programación, matemáticas o idiomas.

Cada uno de estos expertos actúa de forma independiente durante el entrenamiento inicial y posteriormente se combinan. El resultado es una arquitectura eficiente que utiliza solo una pequeña porción de parámetros durante la inferencia.

¿cuáles son los beneficios clave de usar MOE?

El principal beneficio de utilizar una estructura MOE radica en la eficiencia en cómputo. Mientras que los modelos convencionales requieren activar toda su enorme red neuronal para responder cada consulta, los modelos MOE solo activan un grupo limitado de expertos especializados. Esto permite:

- **Menor uso de recursos:** Los MOE ejecutan un cómputo significativamente menor, requiriendo menos GPUs y menos energía, logrando hasta 30 veces de ahorro computacional en comparación con los modelos densos tradicionales.
- **Reducción del tiempo de entrenamiento:** Gracias al entrenamiento paralelo de los expertos, el tiempo requerido baja considerablemente, pasando de meses a semanas y reduciendo así los costos operativos.
- **Democratización tecnológica:** Al reducir la necesidad de grandes infraestructuras y costosos chips, estos modelos hacen más accesible la tecnología de vanguardia a laboratorios y empresas más pequeñas, fomentando una distribución más equitativa del conocimiento tecnológico.

¿cómo selecciona el modelo qué expertos usar?

Aquí entra en juego otra pieza fundamental del enfoque MOE: la denominada gate network. Esta "red gate" funciona como un guía intuitivo, reconocido por identificar rápidamente qué expertos deben activarse según la consulta específica realizada por el usuario. Para compararlo con un ejemplo sencillo: la gate network es como ese vigilante en un parque temático que nos orienta hacia la atracción que buscamos rápidamente.

Adicionalmente, es importante mencionar dos términos relevantes:

- Sparsity (esparcidad): técnica que asegura que la gate network dirija únicamente hacia expertos relevantes, evitando activar innecesariamente áreas de la red no útiles.
- Load balancing: en entrenamiento e inferencia asegura que los expertos se activen adecuadamente en función de la tarea requerida.

¿cuáles son los retos actuales del uso de modelos MOE?

Aunque los modelos MOE presentan grandes ventajas, también implican consideraciones prácticas importantes. Una de ellas es que, aunque la inferencia activa sólo ciertos expertos, todos los parámetros del modelo deben estar previamente cargados en la memoria VRAM de las GPUs, demandando significativamente más memoria video incluso de la que usan en ejecución.

Por otra parte, asegurar que la red distribuya tareas equitativamente entre expertos para evitar sobrecargas (load balancing) y optimizar la respuesta es uno de los retos actuales más importantes en esta tecnología. Además, la investigación actual explora cómo los modelos pueden utilizar simultáneamente varios expertos para resolver preguntas complejas, optimizando así la calidad de las respuestas.

¿cuál es el impacto potencial de los modelos MOE?

En definitiva, los modelos MOE ofrecen una solución atractiva y eficiente en términos de costos y rendimiento. No solo permiten desarrollar modelos potentes utilizando menos recursos, sino que además abren la puerta a nuevas oportunidades para laboratorios e investigadores independientes alrededor del mundo, incluidos aquellos de Latinoamérica. Estos modelos facilitan la participación competitiva a nivel global, sin depender obligatoriamente del acceso a la última tecnología del mercado.

¿Tienes preguntas o comentarios sobre cómo implementar o trabajar con los modelos MOE? ¡Comparte tus ideas y experiencias a continuación, estaremos encantados de conversar contigo!

Here's a clear, code-free explanation of Mixture of Experts (MoE)—the breakthrough architecture behind efficient AI giants like GPT-4 and Mistral! 🌟

1. Concept: Smarter, Not Harder

MoE mimics a team of specialists:

- Instead of using the *entire* neural network for every input, it selects a few "experts" (small sub-models) per task.
- A gating mechanism (like a smart router) decides which experts to activate based on the input.
- Only the chosen experts process the data—saving computation while maintaining model power.

Analogy: Think of MoE like a hospital:

- For a *heart issue*, only cardiologists (experts) are called, not every doctor on staff.
- The receptionist (gating router) directs patients to the right specialists.

2. Key Components

(A) Experts

- Small, specialized neural networks (e.g., feedforward layers).
- Each expert excels at specific tasks (e.g., grammar, math, facts).

(B) Gating Router

- Analyzes the input (e.g., a word or image patch).
- Assigns weights to experts (e.g., *Expert 3: 80%, Expert 7: 20%*).
- Only the top-k experts (usually 1-2) are activated.

(C) Dynamic Computation

- Inputs take different paths through the model.
- Unused experts stay "off," reducing compute costs.

3. Benefits

(A) Efficiency

- Massive capacity: Models can have trillions of parameters (e.g., Switch Transformer's 1.6T), but only billions are active per input.
- Faster inference: Less computation per token → lower latency.

(B) Specialization

- Experts auto-learn domains (e.g., some handle code, others chemistry).
- Improves accuracy for diverse tasks.

(C) Scalability

- Experts can be distributed across many GPUs/TPUs.

4. Real-World Impact

- GPT-4 (rumored): Uses MoE for efficiency in text/image tasks.
- Mistral 7B: Open-weight MoE model rivaling larger dense models.
- Google's Switch Transformer: 1.6T params but ~3x cheaper to run than dense equivalents.

Trade-offs:

- Higher memory (all experts stored, but not all active).
- Complex to train (requires balancing expert usage).

5. Example Workflow

For the input "The mitochondria are the [MASK] of the cell":

1. Router detects a *biology* context.
2. Activates:
 - Expert 5 (biology terms) → "powerhouse"
 - Expert 2 (grammar) → ensures verb agreement.
3. Ignores irrelevant experts (e.g., *math*, *legal jargon*).

Why It Matters: MoE unlocks large-model capabilities with small-model costs—pushing the boundaries of what's practical in AI! 🚀



Mixture of Experts (MOE)

Optimización inteligente de modelos de lenguaje



¿Qué es MOE?

- Técnica avanzada para modelos de lenguaje.
- Divide el aprendizaje en "expertos" especializados.
- Cada experto se entrena en una tarea específica (como programación, matemáticas o idiomas).
- Solo unos pocos expertos se activan por consulta → mayor eficiencia.



¿Cómo funciona?

- Usa una gate network (red guía) que selecciona los expertos adecuados según la tarea.
- Evita activar toda la red, solo lo necesario.



Ventajas de MOE

- 💡 Eficiencia computacional: menos procesamiento, menos consumo energético.
- ⌚ Entrenamiento más rápido: expertos se entrenan en paralelo.
- 🌍 Democratización tecnológica: accesible para laboratorios pequeños y regiones con menos recursos.



Conceptos clave

- Gate Network: guía que elige qué expertos usar.
- Sparsity (esparcidad): activa solo lo relevante.
- Load Balancing: distribuye bien el trabajo entre expertos.

⚠ Retos actuales

- 📦 Alta demanda de memoria (VRAM): todos los parámetros deben estar cargados.
- ⚖ Equilibrar la carga: evitar que unos expertos trabajen más que otros.
- 🧩 Resolver tareas complejas: investigar cómo combinar varios expertos al mismo tiempo.

☀ Impacto

- Reducción de costos y tiempos.
- Acceso a IA avanzada sin grandes recursos.

Oportunidades globales para desarrolladores independientes.

2

Responder

Reportar



Tadeo Juarez

student•hace un día

Hey que tal, dejo mis apuntes redactados de la forma mas sencilla posible! Mixture of Experts es una arquitectura de redes neurales que permiten usar menor cantidad de computo a la hora de entrenar y hacer inferencia (generar una respuesta al prompt) en el modelo

Mixture of Experts (MOE) define "expertos" como partes de la red neuronal que se especializan en procesar diferentes tipos de información. Al momento de "pensar" la respuesta del usuario, Gating Network, que es una pequeña red neuronal, decide que expertos que nos ayudan a contestar mejor acorde al campo de conocimiento del prompt. Así, en vez de activar todas las redes neuronales con todo el conocimiento de entrenamiento para contestar al usuario usamos solo las partes que podrían dar la mejor respuesta al usuario

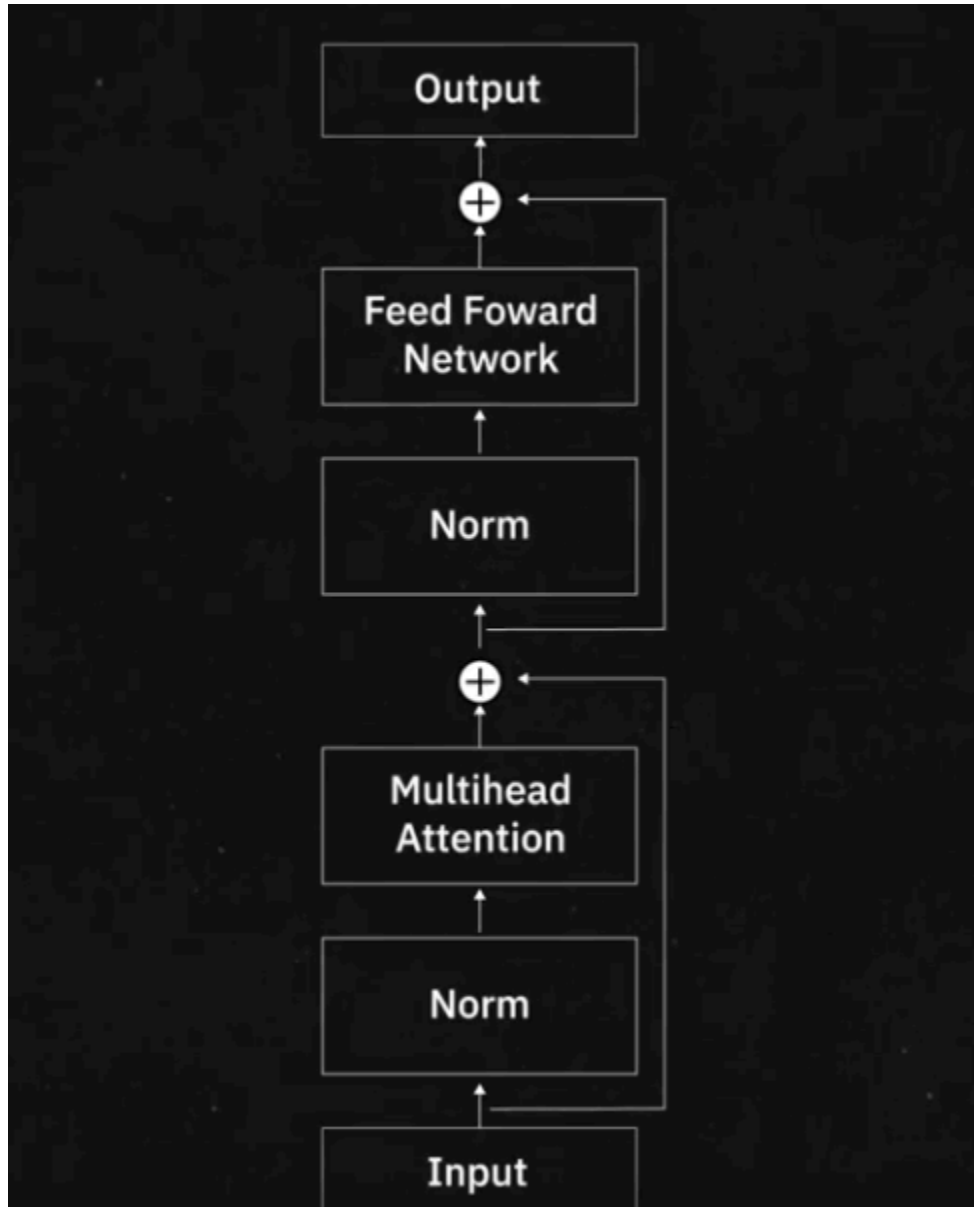
Lo contrario de estos son los modelos densos, modelos que usan toda la información que tienen de entrenamiento (corpus) para contestar al usuario, esto requiere mayor capacidad de computo, mas tiempo de entrenamiento que es igual a mas costo de entrenamiento

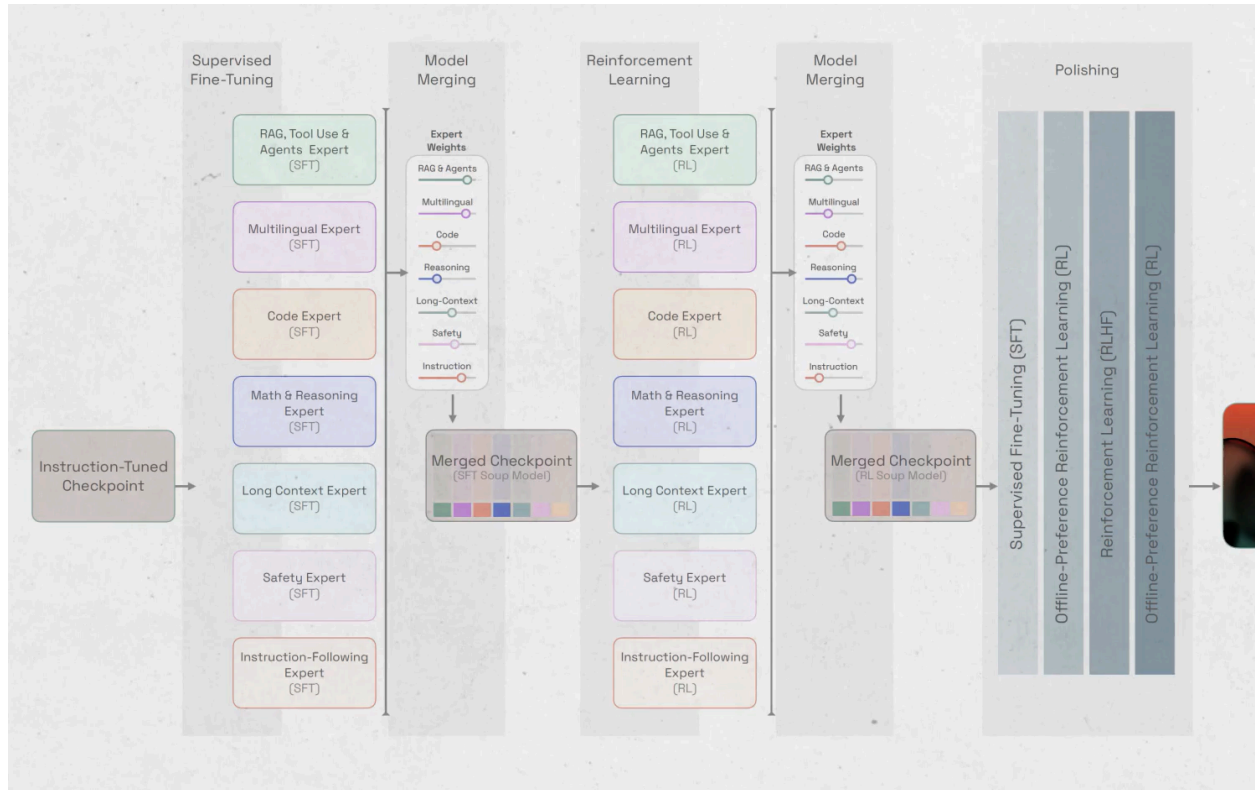
Deepseek es el ejemplo mas claro del poder de esta arquitectura al crear DeepSeek r-1 con menor poder de computo y mejor calidad que modelos de empresas como openai y claude.

Esto es un gran avance porque para poder entrenar estos modelos se necesitan chips, chips que no son infinitos y que tarde o temprano no serán suficientes para estos avances. Desde entonces se han estado desarrollado técnicas de optimización en el entrenamiento de modelos para poder entrenar mejores modelos con cada vez menor capacidad de computo

La "sparsity" o esparcidad se refiere a la característica de un modelo donde la mayoría de los parámetros están en cero o no se utilizan en un momento dado. En el contexto de los modelos de lenguaje, como los Mixture of Experts (MOE), la

esparcidad permite que solo se activen ciertos grupos de parámetros (expertos) durante la inferencia, lo que reduce la carga computacional y mejora la eficiencia. Esto contrasta con los modelos densos, donde todos los parámetros se utilizan para cada predicción, aumentando así el costo y el tiempo de procesamiento.





Resumen

¿Te gustaría ejecutar modelos LLM (*Large Language Models*) directamente desde tu computadora personal? Antes de iniciar debes dominar algunos conceptos importantes como CPU, GPU, RAM, VRAM y cuantización.

¿Cuál es la diferencia entre CPU y GPU?

La CPU es un procesador general encargado de múltiples tareas en la computadora. Por otro lado, la GPU está especializada en operaciones matemáticas específicas como "m por x más b", esenciales en tareas como los modelos LLM.

¿Por qué son importantes la RAM y la VRAM?

¿Qué función tiene la memoria RAM?

La memoria RAM almacena temporalmente archivos e información activa, como cuando editas una imagen en Photoshop o navegas por múltiples pestañas en Internet. Para ejecutar modelos LLM localmente, se recomienda:

- Un mínimo de 16 GB.
- Preferentemente hasta 64 GB de RAM para un rendimiento óptimo.

¿Cuál es el papel de la memoria VRAM?

La VRAM (*Video Random Access Memory*) está directamente integrada con la GPU. Al estar físicamente cercana, permite una menor latencia y mayor rendimiento. Para ejecutar modelos LLM pequeños o cuantizados, lo ideal es contar con entre 12 y 16 GB de VRAM. Las tarjetas gráficas de consumo alcanzan hasta 36 GB en su gama más alta.

¿Qué es y qué beneficios ofrece la cuantización de modelos?

La cuantización permite reducir el peso de los modelos de machine learning, bajando la precisión numérica empleada. Los modelos originalmente entrenados con 16 bits pueden cuantizarse a 8 o 4 bits, disminuyendo su tamaño significativamente. Por ejemplo:

- Un modelo de 100 GB (16 bits) podría reducirse a 25 GB (4 bits).

Estas reducciones facilitan ejecutar modelos grandes en hardware más modesto y accesible.

Ahora que entiendes estos conceptos, en próximas sesiones podrás verificar qué modelos son compatibles con tu equipo utilizando herramientas como Hugging Face y Olama.



Ejecutar Modelos LLM en tu PC

Conceptos clave que debes dominar



CPU vs. GPU

♦ CPU (Procesador Central)

- Realiza tareas generales del sistema.
- Encargado de múltiples tareas.

♦ GPU (Procesador Gráfico)

- Especializada en cálculos matemáticos específicos ($y = mx + b$).
- Esencial para tareas de machine learning y modelos LLM.



RAM y VRAM

♦ RAM (Memoria de Acceso Aleatorio)

- Almacena temporalmente lo que estás usando.
- Ejemplos: editar imágenes, tener muchas pestañas abiertas.

Recomendado para LLM:

- Mínimo: 16 GB
- Ideal: hasta 64 GB

♦ VRAM (Memoria de Video)

- Integrada con la GPU.
- Velocidad alta gracias a su cercanía física con el procesador gráfico.

Recomendado para LLM:

-  De 12 a 16 GB para modelos pequeños o cuantizados
-  Hasta 36 GB en tarjetas gráficas de gama alta a nivel personal



¿Qué es la Cuantización?



Reducción de tamaño del modelo

- Baja la precisión numérica: de 16 bits a 8 o 4 bits.
- Menor tamaño, mismo propósito.

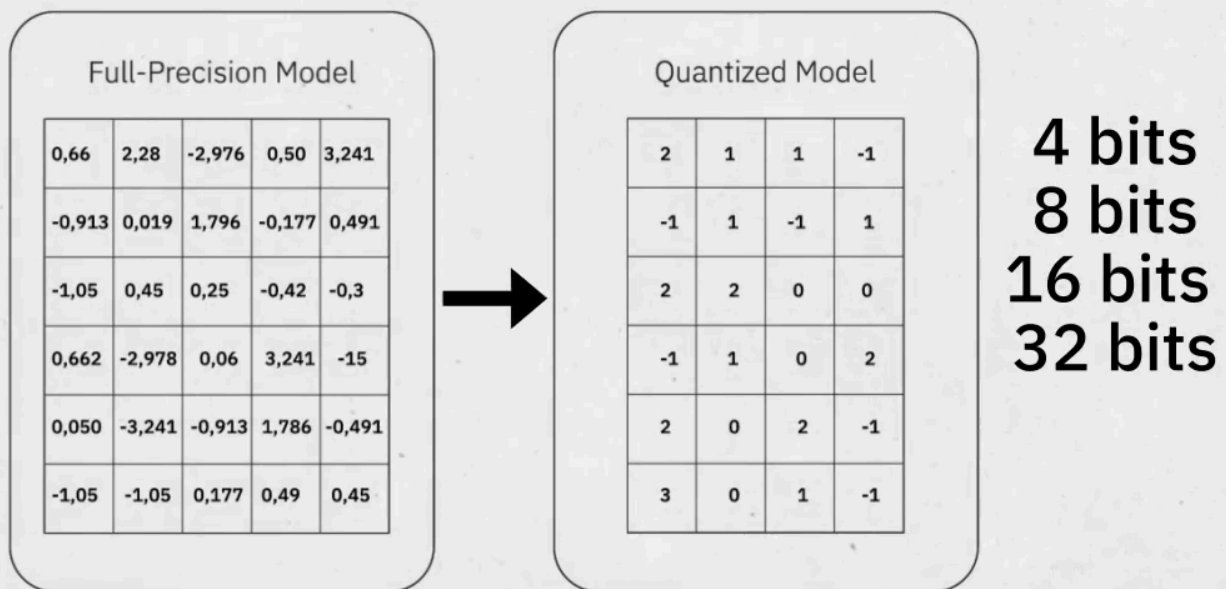


Ejemplo: Modelo de 100 GB (16 bits) → 25 GB (4 bits)



Permite ejecutar modelos grandes en computadoras modestas.

La cuantización ofrece una manera de reducir el almacenamiento y el uso de memoria de los modelos.



Aprender a cuantizar para abarca mas aunque se pierda precision

Resumen

¿Te interesa aprender a ejecutar modelos LLM (Large Language Models) de forma local en tu equipo? A continuación, aprenderás cómo descargar Olama, configurar modelos como DeepSig y utilizar ExoLabs para conectar múltiples computadoras potenciando tu capacidad de procesamiento local para LLMs.

¿Cómo se instala y ejecuta Olama de manera local?

Olama es una herramienta sencilla para ejecutar modelos LLM de forma local:

1. Descarga Olama: Ve a Google, busca Olama, y descarga la versión adecuada para tu sistema operativo (macOS, Windows o Linux).
2. Instalación: Una vez descargado el archivo binario, ejecútalo haciendo doble clic y siguiendo los pasos que indica tu sistema operativo (por ejemplo, en macOS selecciona mover a Aplicaciones).
3. Uso desde terminal: Abre la terminal y ejecuta Olama para ver los comandos disponibles. Copia desde la web de Olama el comando específico del modelo que quieres ejecutar (por ejemplo, DeepSig r1-7b).

¿Cómo interactúa Olama con los modelos LLM?

Al ejecutar un modelo en Olama, este descargará los archivos necesarios (puede tomar varios minutos):

- Interacción básica: Realiza preguntas o comandos como "¿cuánto es dos más dos?" para empezar a trabajar.
- Consumo de recursos: La descarga y uso del modelo consumen recursos de CPU y memoria, por ejemplo, durante una consulta puede subir el consumo a más de dos gigabytes temporalmente.

¿Qué es ExoLabs y cómo potencia el uso local de LLMs?

ExoLabs permite conectar múltiples computadoras creando un clúster local de potencia computacional compartida:

- Concepto principal: Usa redes peer-to-peer (P2P) como la tecnología detrás de plataformas como Torrent para compartir datos y tareas entre máquinas conectadas.
- Ventajas: Reduce costos al utilizar varias computadoras económicas, mejora la privacidad (importante en sectores bancarios o jurídicos) y permite controlar específicamente las configuraciones y costos operacionales.

¿Cómo montar un clúster local con ExoLabs?

Estos pasos específicos son clave para conectar varias máquinas con ExoLabs:

- Equipamiento: necesitas diversas computadoras conectadas mediante switch y router, pudiendo combinar conexiones Ethernet y WiFi.
- Configuración y ejecución: instala ExoLabs en cada máquina; al ejecutar, crea automáticamente una red entre ellas mostrando información como teraflops o memoria disponible.
- Funcionamiento: Cada computadora aporta sus capacidades (CPU, GPU, memoria) a la red, haciendo posible ejecutar modelos de gran tamaño que de otro modo requerirían hardware costoso.

¿Qué modelos LLM puedes ejecutar localmente con ExoLabs?

Puedes ejecutar modelos de distintos tamaños según la capacidad de memoria acumulada en tu red:

- Modelos como DeepSig r1-70b requieren 70 gigabytes de memoria, completamente alcanzables al conectar varios equipos que sumen dicha capacidad.
- Esta forma de trabajo localiza la informática, especialmente útil si tienes requisitos estrictos sobre información privada.

¿Cuáles son las ventajas concretas de ejecutar LLM en local?

- Privacidad total: Los datos sensibles permanecen en tu entorno, clave para industrias sensibles al manejo de información.
- Control absoluto: Define exactamente qué quieres ejecutar, cuánto gastar, y cómo ajustar o optimizar tu modelo específicamente para tu caso de uso con técnicas como el fine tuning.

Ahora que conoces cómo instalar Olama, manejar modelos locales e integrar ExoLabs para incrementar capacidad local, puedes aprovechar al máximo el potencial de los modelos LLM de manera segura y controlada.

Objetivo principal

Ejecutar modelos LLM (Large Language Models) de forma local, usando herramientas como Olama, DeepSig y ExoLabs, sin depender de la nube.

Herramienta 1: Olama *Permite correr modelos LLM en tu propia computadora*

Cómo instalar Olama

- ♦ Busca “Olama” en Google
- ♦ Descarga según tu sistema operativo (macOS, Windows, Linux)
- ♦ Ejecuta el archivo descargado y sigue las instrucciones del sistema
- ♦ Abre la terminal → Escribe olama → Verifica comandos disponibles

- ◆ Copia desde la web de Olama el comando del modelo que quieras usar (ej. deepsig r1-7b)

Cómo funciona Olama

- Descarga automática del modelo al ejecutarlo
- Puedes hacer preguntas simples (“¿cuánto es dos más dos?”)
- ⚠ Usa recursos del equipo (CPU y memoria >2 GB durante el uso)

🌐 Herramienta 2: ExoLabs *Crea una red local de computadoras para sumar potencia de procesamiento*

¿Qué hace ExoLabs?

- ◆ Conecta varias computadoras en red (tipo P2P como Torrent)
- ◆ Comparte tareas y datos entre equipos
- ◆ Ideal para mantener privacidad y reducir costos

Ventajas

- ✓ Menor costo: puedes usar equipos económicos
- ✓ Más privacidad: los datos no salen de tu red
- ✓ Control técnico: decides cómo y con qué recursos ejecutar modelos

🔧 ¿Cómo montar una red local con ExoLabs?

- 📌 Conecta varias computadoras mediante router y switch (Ethernet o WiFi)
- 📌 Instala ExoLabs en cada una
- 📌 Al ejecutarse, se conectan automáticamente y muestran:
 - ▢ Memoria disponible
 - ▢ Teraflops
 - ▢ Componentes activos (CPU/GPU)

🧠 Modelos que puedes usar con ExoLabs

- 📌 Dependen de la memoria total de la red
- 📌 Ejemplo: DeepSig r1-70b → requiere 70 GB

✓ Al conectar suficientes equipos, puedes ejecutar modelos muy grandes localmente

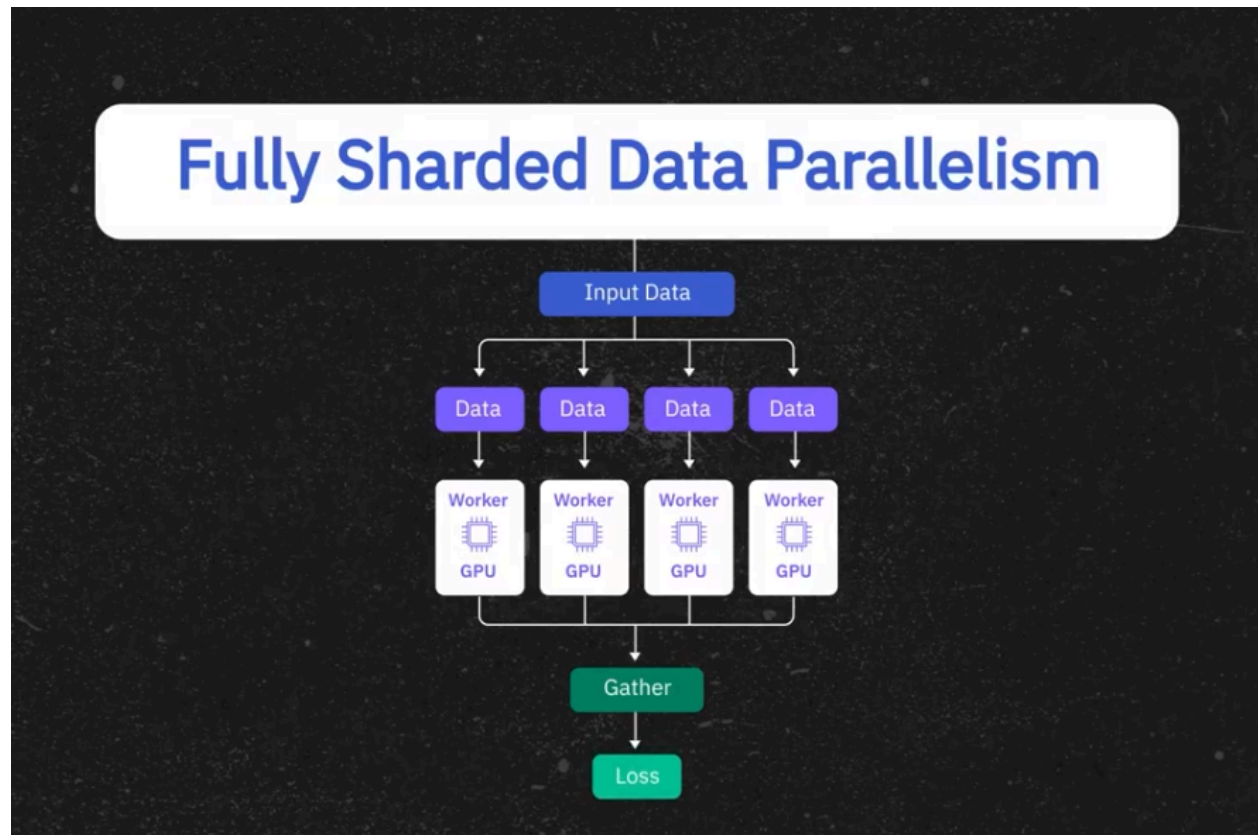
🔒 Ventajas de ejecutar LLMs en local

🔒 Privacidad: datos sensibles permanecen en tu red

⚙️ Control total: eliges modelo, recursos y configuración

Una de las cosas que menciona Jhenner en el min 10:22 de que todo está unificado en el chip es una de las razones por las cuáles ni el mejor soldador del mundo puede deshacer esto sin dañar la máquina, y por ende, tampoco se puede incrementar la RAM como se puede hacer con otras computadoras.

Redes p2p , conectar multiples maquinas



Resumen

La técnica supervised fine tuning permite ajustar modelos preentrenados como GPT-4 usando conjuntos específicos de datos. Esto es clave cuando modelos generales como ChatGPT carecen de respuestas adecuadas por limitaciones en su entrenamiento inicial.

¿Qué es el fine tuning supervisado en la inteligencia artificial?

El fine tuning supervisado significa entrenar un modelo preexistente suministrando claramente tanto el dato como la respuesta esperada. El modelo ajusta sus pesos mediante procesos llamados *forward pass* y *backward pass*,

procedimientos explicados previamente y esenciales para adaptar el modelo al nuevo contexto.

¿Para qué sirve aplicar fine tuning con GPT-4 en OpenAI?

Esta técnica permite darle al modelo la habilidad de responder preguntas específicas que inicialmente no podría contestar bien debido a la limitación de su entrenamiento original:

- Adaptar respuestas a contextos especializados como medicina.
- Incrementar la precisión y calidad de respuestas.
- Controlar extensiones de textos generados para eficiencias mayores y costos operacionales menores.

¿Cómo preparar un dataset para fine tuning de GPT-4?

El proceso anterior a la ejecución del fine tuning implica una serie de etapas esenciales para asegurar la calidad del modelo entrenado:

1. Análisis inicial del conjunto de datos identificando duplicados, respuestas truncadas, o contenido inapropiado.
2. Eliminación de registros no óptimos como entradas duplicadas, información incompleta o expresiones problemáticas mediante herramientas como expresiones regulares con Pandas.
3. Transformación y formateo del dataset según la estructura admitida por OpenAI, con campos claramente definidos para roles de asistencia, usuario y sistema.

¿Qué tipo de ajustes se requieren para un entrenamiento exitoso?

Algunos aspectos críticos durante la limpieza y filtrado del dataset pueden incluir:

- Comprobación exhaustiva del contenido duplicado o incorrecto.
- Asegurar respuestas completas y evitar desviaciones o datos irrelevantes.
- Mantener un estándar mínimo definido (p.e., respuestas mayores a cincuenta caracteres).

Usar librerías Python como Pandas facilita enormemente estos procesos.

¿Qué parámetros configurar durante el fine tuning en OpenAI?

El usuario debe establecer hiperparámetros vitales que optimizan cómo el modelo aprende del nuevo dataset:

- Batch size: define el número de iteraciones antes de actualizar los pesos (menor número implica procesos más lentos).
- Learning rate multiplier: ajusta el nivel del aprendizaje para evitar mínimos locales.
- Número de épocas: automatizable por OpenAI o manualmente controlable para diferentes niveles de detalle en el entrenamiento.

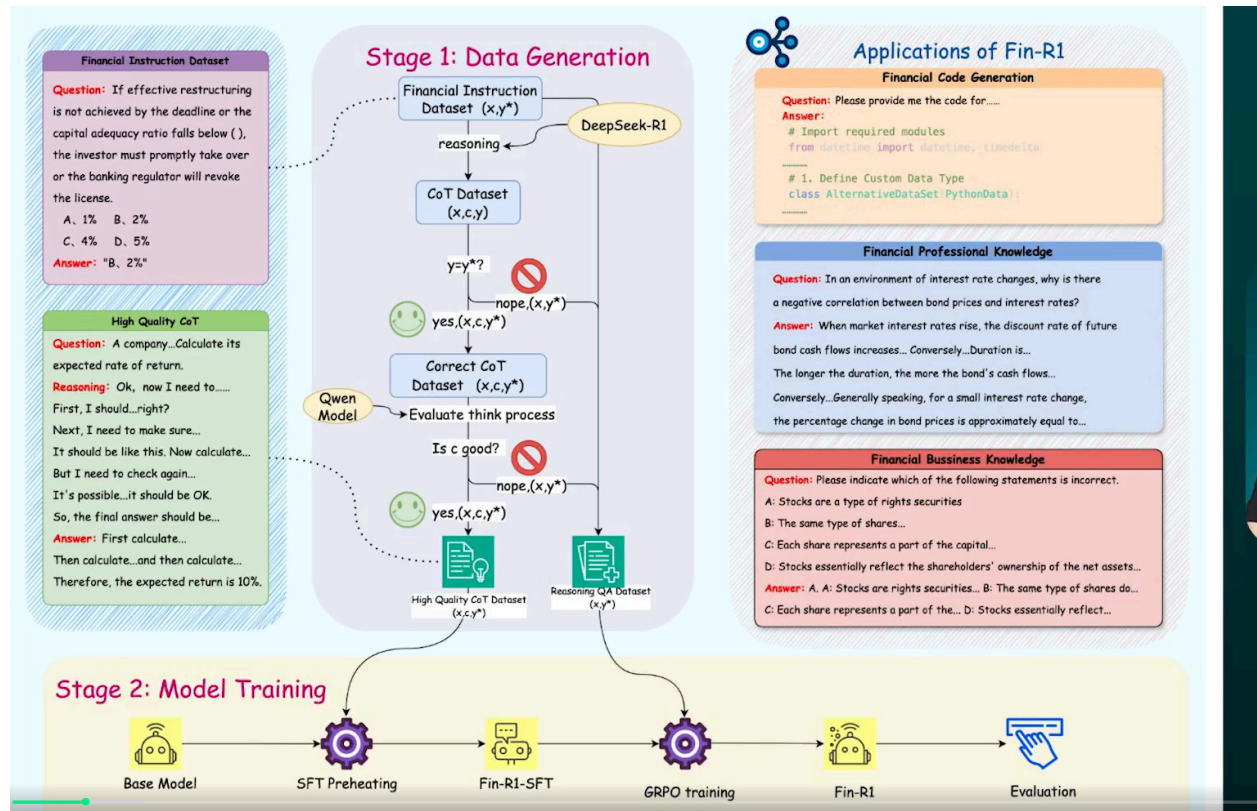
¿Qué resultados esperar tras el proceso de fine tuning?

La plataforma OpenAI proporciona múltiples métricas relevantes:

- Cantidad de tokens utilizados, directamente vinculados al costo económico.
- Gráficas detalladas mostrando comportamiento de métricas como pérdida (*loss*) en entrenamiento y validación.
- Precisión (*accuracy*) del modelo con el conjunto de pruebas (*test dataset*).

Además, OpenAI ofrece distintas etapas del modelo que permiten comparar rendimiento versus el modelo original mediante la interfaz del Playground, facilitando actualizaciones incrementales para optimizar resultados con iteraciones adicionales sobre el dataset original.

Se anima a quienes estén realizando este procedimiento a compartir sus experiencias analizando dataset, considerando técnicas estadísticas simples para descubrir información adicional o mejoras posibles en los datos utilizados.



Fine-tuning supervisado de GPT-4 con datasets médicos (SCRIPT)

1. Análisis del dataset

Función: `analyze_dataset()`

- Entrada: JSON con campos `instruction`, `input`, `output`
- Detecta:
 - Duplicados
 - Respuestas truncadas (... o < 50 caracteres)
 - Frases inútiles o lenguaje inapropiado

2. Corrección del dataset

Función: `correct_dataset()`

- ✓ Elimina duplicados
- ✂ Filtra respuestas muy cortas o truncadas
- ✗ Elimina respuestas con contenido inadecuado
- 📁 Guarda nuevo JSON como `corrected_<original>.json`

3. Formateo para fine-tuning

- 📄 Filtra respuestas con `output < 500` caracteres

Estructura como conversación tipo ChatGPT: [

```
{"role": "system", "content": "instrucción"},
```

```
{ "role": "user", "content": "input"},
{ "role": "assistant", "content": "output"}
]
```

-
-  Divide el dataset:
 - `train_formatted_dataset.jsonl` ← primeros 500
 - `test_formatted_dataset.jsonl` ← siguientes 499

4. Carga en OpenAI + Entrenamiento

-  Usa `dotenv` o `userdata` para obtener `OPENAI_API_KEY`
-  Sube los archivos `.jsonl` con `purpose="fine-tune"`
-  Fine-tuning supervisado sobre `gpt-4o-2024-08-06`:
 - `batch_size = 5`
 - `learning_rate_multiplier = 0.001`
 - `n_epochs = auto`

5. Monitoreo del entrenamiento

-  Lista los últimos trabajos: `client.fine_tuning.jobs.list()`

 Visualiza eventos: `client.fine_tuning.jobs.list_events(...)`

Fuentes:

https://huggingface.co/datasets/medalpaca/medical_meadow_wikidoc/blob/main/medical_meadow_wikidoc.json

Bien, ya hemos entendido cómo funcionan los LLMs, cómo podemos construir nuestros propios modelos y cómo funcionan algunos pipelines de datos.

Pero, ¿qué pasa si

un modelo de LLM como ChatGPT, GPT cuatro o Cloud no pueden resolver las preguntas que nosotros tenemos porque en su conjunto de entrenamiento no

tienen la información suficiente para poder responder este tipo de preguntas. Recordemos que al final un LLM usa la distribución de la información con la

que se entrenó para poder dar respuestas acertadas a las preguntas que nosotros tengamos. Para esto, se crearon técnicas de fine tuning o ajuste fino.

Estas técnicas de fine tuning son principalmente hechas y desarrolladas para que nosotros no tengamos que entrenar un modelo desde cero con vastas cantidades de

información, sino que tomemos un modelo preentrenado, así se le llama la industria, y cojamos este modelo y lo entrenemos con un conjunto de datos

que nosotros tengamos o hayamos desarrollado según nuestro caso de uso. En esta clase vamos a hacer un fine tuning de GPT cuatro o, haciendo

uso de la técnica supervised fine tuning. ¿Qué significa este supervised fine tuning? Esto básicamente lo que nos quiere decir es que de manera supervisada,

es decir, que nosotros le vamos a explicar explícitamente al modelo cuál es el dato y cuál es la respuesta que nosotros quisiéramos para cada

pregunta, y él va a aprender, va a modificar sus pesos haciendo un paso forward y un backward pass, que es lo que vimos en

nuestras primeras clases, para modificar los pesos del modelo. Este proceso puede tomar unos minutos, así que vamos al código, veamos este proceso más a

detalle desde el conjunto de entrenamiento y los pasos intermedios que hay que hacer antes de hacer el fine tuning y vamos a lanzar este

proceso de tuning en la plataforma de OpenAI. En este collab, en la sección de recursos específicamente, voy dejar un dataset. Este dataset yo le

puse como nombre Bad Medical Medium Wikidoc, y básicamente es un dataset que encontré en Cable, la plataforma de datasets y de competición en data

science, que lo que nos da es varias datos. Primero nos da ahí una instrucción, que es básicamente el system prompt, o el prompt, digamos,

del sistema. Luego tenemos el input, que es la pregunta y el output. Este dataset en particular es de preguntas médicas. Yo este dataset, si

se dan cuenta, le puse bat porque le hice un un par de cosas malas. Le apliqué duplicados, le agregué malas palabras, tronqué respuestas, cambié

respuestas cosas que no tenían sentido. Entonces, esta parte lo primero que tenemos que hacer, siempre que queremos hacer fine tuning, es limpiar nuestro conjunto

de datos. No podemos simplemente hacer un dump o traernos toda la información de una base de datos y subirla a un modelo de tuning,

Tenemos que analizarlo. El proceso de análisis también se lo dejo como una tarea para que lo compartan dentro de los comentarios, porque acá les

voy mostrar un par de cositas que yo le agregué al dataset, pero no son todas. No son todas las reglas, ni preguntas, ni palabras

malas que yo agregué dentro del dataset. Yo hice un script personalizado para esto, pero dejé acá un par de cosas. Voy a cerrar un

momento los archivos, vamos a empezar ejecutando la primera celda para que nos importe todas las librerías que necesitamos, y acá yo tengo dos variables.

La primera es una lista de frases que no son de ayuda, como este tipo de frases de, no sé, le preguntele a otra persona,

vaya a un doctor inmediatamente, sino que le le puse yo en el en el momento crear el dataset este tipo de respuestas que realmente

lo que van hacer es confundir al modelo. Y adicionalmente, puse unas palabras malas o unas course words, que digamos que serían groserías, en este

caso le puse damn, y otras. Hay muchas más, te animo a que las busques, va ser un poco difícil porque vas a tener que

hacer un análisis de las palabras que están en cada uno de los de los campos, vamos a ejecutar esta celda y luego yo creo

una función analyze dataset. En esa función analyze dataset trato de replicar los pasos que normalmente uno en la vida real lleva cuando está limpiando

este tipo de datasets. Primero, analizamos si hay duplicados, ¿por qué? Este fue una de las cosas que agregué cuando estaba haciendo el dataset. Entonces,

primero miramos cuántos duplicados hay, luego miramos si hay respuestas truncadas, respuestas que terminen en tres puntos o que tengan menos de cincuenta caracteres. ¿Por

qué? Porque cuando tú te vas al dataset original, al Wikidoc, vas a darte cuenta de que no hay ninguna respuesta que tenga menos de

cincuenta caracteres. Yo creé respuestas y modifiqué respuestas y las tronqué para que tuvieran menos de cincuenta caracteres. Luego lo que hago es un donde

empiezo a analizar si alguna de las respuestas o dentro dato hay alguna de las frases que no son de ayuda o hay groserías, y

las voy guardando en una variable. Vamos a ejecutar esta función para que quede definida y yo retorno finalmente cuántas son las entradas, cuántos son

los duplicados, cuántos son los truncados y cuántos tienen contenido problemático. Ya con esa información yo sé cuánto tengo de cada cosa y qué tan

preciso y qué tan bueno es mi calidad de los datos. Luego, en la función correct dataset es donde aplico efectivamente esas mejoras. Entonces, volvemos

cargar la información, cargamos el dataset, eliminamos los duplicados, filtramos las respuestas truncadas, eliminamos el contenido potencialmente dañino, que en este caso serían las frases

que no son de ayuda o las frases que son o las palabras groserías particular. Y esto es muy similar a lo que uno hace

cuando está limpiando un dataset. 10 en cuenta de que uno de los principios fundamentales del machine learning, de deep learning y de la AI

en general es que garbage in o basura adentro, garbage out, basura afuera. Es decir, si yo entré en un modelo con datos basura, el

modelo lo único que va ser capaz de hacer es responderme con datos basura. Entonces, los los filtramos acá con una expresión regular. Pandas particularmente

contiene funciones que no se permiten ejecutar expresiones regulares como este contains, las eliminamos y guardamos los datos correctos en un archivo aparte. Vamos definir

esta función de nuevo y acá la vamos a ejecutar. Entonces inicialmente yo cargo mi dataset, lo analizo, obtengo las respuestas de qué tanto tenemos

de cada uno de los problemas y luego lo corrijo. Entonces vamos a ejecutarlo, no debe tomarle tanto tiempo, Listo, acá lo tenemos. Entonces, tenemos

de total de entradas, o sea, total de puntos de información, tenemos once mil quinientos. De esos once mil quinientos, tenemos mil ciento trece que

son duplicados, es decir, que lo quitamos de nuestro dataset original. Tenemos setecientos sesenta y tres respuestas truncadas y doscientas ochenta y tres respuestas con

malas palabras o información que realmente no nos va ayudar a responder nuestras preguntas. Y ya finalmente, cuando aplicamos todos estos filtros, lo que tenemos

es que pasamos de un dataset de once mil quinientos registros a un dataset de nueve mil seiscientos cuarenta y uno. Es importante tener en

cuenta que esto es un ejemplo, pero uno puede encontrarse en la vida real que tener un cien por ciento de datos y tener hasta

el final un veinte, es decir, que todo el proceso de limpieza nos dejó solo un veinte por ciento de los datos. Por eso el

activo más valioso de estas empresas normalmente no es solo el modelo, el modelo es importante, pero lo más valioso siempre van a ser los

datos, porque sin datos de buena calidad realmente no vas a poder hacer absolutamente nada. Con esto ya tendríamos finalizada nuestra fase de limpieza de

la información. Acá es importante que tengas en cuenta que este paso y estos pasos que te mostré son unos ejemplos, pero son los ejemplos

que tú puedes usar para guiarte en el mundo real cuando estés haciendo tratamiento de información. Yo te insto a que agarres este dataset y

le apliques estadística, poquito de estadística, cuentes cuántas palabras hay, cuál es la palabra que más se repite, a ver si así eres capaz de

encontrar los otros easter eggs que yo dejé dentro del dataset y que también deben ser removidos para que el fine tuning tenga mejor resultado.

Ahora, te voy mostrar cómo podemos modificar este dataset que ya tenemos, que tiene una estructura, a la estructura que OpenAI recibe en sus servicios

de fine tuning. Ahora vamos a cargar nuestro dataset ya limpiado de estos errores que introducimos. Entonces, esta casilla lo que estoy haciendo también es

filtrando por las puntos de datos que tengan menos de quinientas palabras en su salida, su respuesta. ¿Por qué lo estoy haciendo? Porque no quiero

que el modelo dé respuestas tan extensas en las preguntas que nosotros le hagamos, es como algo que quiero hacer, nada más acá para que

el proceso de entrenamiento también dure un poco menos, pero pueden quitar esta línea, esta línea es totalmente opcional, si quieren la pueden eliminar. Luego

vamos a modificar el dataset para que reciba y tenga la estructura que OpenAI recomienda y es la que ellos aceptan. Eso ustedes lo van

a poder ver en la API de OpenAI, si buscan en Google, punto docs open AI fine tuning, ahí van ver la documentación, y se

van dar cuenta de que la forma en que él lo recibe es que para cada uno de los mensajes o los chats vas a

tener que pasar tres cosas. La primera es el system prompt, que en el caso de nuestro dataset es la instrucción. Luego, lo que el

usuario va a preguntar, que en nuestro caso es el input, con el rol usuario. Y luego lo que la AI o el asistente, como

lo llama OpenAI, responde, que sería el output. Y con esto vamos a crear una lista que va ser lo que finalmente nosotros le vamos

a dar a OpenAI. Nosotros le damos un a este result, vamos a ver que tiene seis mil seiscientos ochenta. Seis mil seiscientos ochenta datos

son demasiados, son bastantes, para hacer un fine tuning y puede tomar este proceso horas. Para efectos de la clase, también lo puedes eliminar. Yo

lo que estoy haciendo es tomando solo quinientos registros de esos seis mil seiscientos ochenta, que lo estoy haciendo con esta instrucción, con un list

comprehension de dos puntos quinientos, esto le dice básicamente que vaya de la posición cero a la quinientos y me dé toda esa información. Vamos

a guardar en este caso con este nombre que es train formatted dataset punto json l. JSON l es una forma específica de guardar archivos

json que sean listas, que hagan referencia a cada línea es un registro único. Es la forma en la que OpenAI también lo recomienda. Y

luego vamos a guardar conjunto de entrenamiento. Recuerden que siempre tenemos que tener un conjunto de test y un conjunto de entrenamiento. Ahora voy a

instalar la librería que me va a ayudar a cargar las variables de entorno que tengo que cargar en mi sistema. Entonces, eso lo podemos

hacer con exclamación pip install. Esa exclamación lo que hace es que pueda ejecutar comandos de la consola dentro de Google Colab. Luego vamos a

instanciarlo y vamos a cargar OpenAI API Key. Eso donde lo podemos ver acá en la llavecita, vamos a ver que yo le puse el

nombre OpenAI API Key y le puse un valor. Ya con eso, en teoría, la tendríamos que tener carga de nuestro sistema. Entonces, a ejecutar acá

esta línea. Bueno, acá me da un error, me dice que no tiene un API Key, se lo voy a pasar. API Key es igual a API Key

y me sigue dando un error. Seguramente el API Key está mal cargado. Vamos a ver qué valor tiene. Ok, no tiene ningún valor. Entonces,

esto quiere decir que el sistema no está cargando correctamente nuestra API Key. Vamos a ir un momento acá a la llavecita. Ah, mira. Acá

abajo me está diciendo cómo yo la tendría que cargar. Entonces, a copiar este código específicamente y voy a decir API key es igual a

user data punto get y el nombre de mi secreto, que sería este. Como ya confío más en el código que me está dando Google

Colab, no voy a mostrar acá específicamente qué valor tiene, vamos a confiar. Entonces, ahí ya la ejecutamos correctamente, se la estamos pasando al cliente,

estamos subiendo los archivos de train y de test a OpenAI, algo importante es que le tenemos que poner como propósito en el campo purpose

fine tune. ¿Por qué? Porque esto le va permitir a la API saber que este archivo va ser usado para un proceso de supervised fine

tuning. Vamos a ejecutarlo, se puede tomar un poquito de tiempo, porque está subiendo archivos, bueno, lo subió. Esto lo que nos va dar es

un objeto que va tener un ID. Vamos a ejecutarlo acá, entonces tenemos que el train ID es file y el test ID es este

de acá. Con eso tendríamos nuestros dos archivos cargados en la API de OpenAI para poder proceder con el fine tuning. ¿Cómo hacemos un job

de fine tuning? ¿Cómo lo creamos? Acá es donde viene la parte más sencilla de todos realmente. Cuando yo estaba aprendiendo a hacer AI, esto

era muy complicado, eran cientos de miles de líneas de código para hacer un proceso de fine tuning. Hoy en día es muy sencillo. Con

client punto fine tuning, de nuevo esta palabra la vamos a ver siempre que queramos hacer un fine tuning, punto job, ¿por qué punto jobs?

Porque lo que vamos a es un trabajo que va ejecutarse asincrónicamente, del cual no tenemos que estar esperando acá en el notebook de fine

tuning con los datos que nosotros le damos. Le damos punto create. Lo primero que le tenemos que pasar es el archivo, el ID del

archivo de entrenamiento. Entonces, voy a cogerlo, train, este de acá, y lo voy a poner. Y luego le tenemos que pasar uno de validación

para que el modelo sepa si está diciendo bien su trabajo o no, que sería en este caso este. Lo voy poner acá. Luego tenemos

que seleccionar un modelo que queremos nosotros hacer fine En este caso, los modelos son poco limitados, en los recursos de la clase te va

bajar la liga, la documentación, donde están todos los modelos que tú le puedes hacer fine tuning. En este caso, vamos a hacer un fine

tuning de GPT cuatro o, entonces nada más ponemos GPT cuatro o y la versión del modelo que queremos fine tuner, que en este caso

específicamente es la de dos mil veinticuatro cero ocho cero seis. Luego en el método, ahí ponemos el método específico que nosotros queremos usar para

el fine tuning. Recordemos que esta clase es sobre supervised fine tuning, entonces el tipo específicamente es supervised, supervisado. Y este método tiene un par

de parámetros, un par de cosas que nosotros le podemos decir al modelo que use diferente dependiendo nuestro caso de Estos son llamados hiperparámetros. Hyperparámetros,

¿por qué? Y esto lo vamos a mucho en los modelos de machine learning o de AI en general. Son parámetros que nosotros como personas

seleccionamos a mano y a punta de prueba y error buscamos cuál es el mejor parámetro para nuestro caso de uso. En este caso, el

batch acá le tengo un valor de cinco, el batch size hace referencia a cuánta, cada cuánto se van a actualizar los pasos en el

back propagation. Entre menor sea este número significa que cada paso, por ejemplo, este caso con cinco es cada cinco ejecuciones de nuestro de nuestro

dataset de entrenamiento, va ser un backward propagation, un back propagation, ¿ya? Si nosotros bajamos este número, el proceso va ser mucho más demorado. Si

le ponemos uno, que es el mínimo que se permite, el proceso va ser el más demorado de todos. Podemos poner cuando cuando quisamos. En

este caso, yo le estoy poniendo cinco, pero lo podemos poner ciento veintiocho o mil, depende de la cantidad de información que tenemos. Como en

este caso tenemos quinientos, yo le estoy poniendo cinco, que es como el uno por ciento. Entonces, con cinco cada cinco pasos va a actualizar

los pesos. Luego vamos a tener un learning rate multiplier. ¿Recuerdan que en clases anteriores hablamos de que el learning rate era como esto que

usamos para darle una patada al balón para sacarlo de mínimos locales y que el modelo pueda aprender mejor. En este caso, yo estoy usando

cero coma cero cero uno, pero lo recomendado puede ser desde cero coma cero cero cero uno hasta diez incluso. Normalmente, en papers y demás,

van a ver que el learning rate siempre es muy bajito, porque si bien queremos que el modelo se salga de esos mínimos locales, también

si ponemos un número muy alto, puede que volvamos loco el modelo y nunca va aprender. Y luego en el número de épocas o en

epochs vamos a ponerlo automático. ¿Por qué? Porque OpenAI es muy inteligente, tiene algoritmos que ya detectan con base a los otros hiper parámetros cuánto

serían las números de iteraciones posibles y necesarias para alcanzar el mejor modelo posible con nuestro conjunto de datos. Pero acá también puedes poner el

valor que quieras. Recuerda, entre más alto sea este número, tiempo va tomar el entrenamiento. Ya con esto, cuando le demos crear, ahí se demoraron

un poco, ya lo creó. En teoría, si nosotros nos vamos a la plataforma de OpenAI, ya lo vamos a poder ver. Entonces, yo acá

tengo ya abierto OpenAI, acá yo tengo un par de ejemplos que estuve haciendo, voy a recargar y vamos a ver que ya se está

creando nuestro proceso. Entonces, ahí va a empezar validando los archivos que subimos, nos va dar un job ID, nos va decir qué tipo de

método es, el modelo, cuándo se creó, si los datos se comparten o no. En este caso, la cuenta que yo estoy usando en Platzi,

tenemos que no se compartan los datos con OpenAI y todos los hiperparámetros que nosotros le pasamos, adicional de los archivos, y un par de

mensajes que nos van decir en qué estado está. Esto va tomar un tiempo, Entonces, te recomiendo que vayas leyendo algunos papers, te voy dejar

algunas recomendaciones buenas sobre supervisors fine tuning en la recursos de la clase mientras esto se ejecuta. Puede tomar minutos, decenas de minutos o horas.

Va depender mucho de si le quitaste esa línea de código donde yo solo estaba trayendo quinientos registros para hacer el fine tuning. Entonces, con

esto nada más tenemos que esperar a que termine el modelo para poder ya usarlo. Luego de un par de horas o días, dependiendo de

la cantidad de información que le hayas dado el LLM para hacer fine tuning, vas a ver que el entrenamiento ya terminó. Sé paciente, pueden

pasar varias cosas durante el entrenamiento que te hagan dudar de si tu LLM realmente está aprendiendo o no. Vamos a ver un par de

ellas ahorita en la interfaz de OpenAI. En este momento estoy otra vez en la interfaz de OpenAI y vas ver mandé un job adicional

al último que mandé antes de esperar a que terminara. Y es que el primer job que mandamos, este que dice GPT cuatro dos mil

veinticuatro cero ocho cero seis, que lo hice el cinco de mayo a las diez y veintidós UTC, tuve que cancelarlo. ¿Por qué? Si nosotros

vamos al final de esta parte de de la plataforma, vamos a ver que hay varios mensajes. El primero me va decir que se creó

el trabajo de fine tuning, luego me va decir que estaba validando el archivo de entrenamiento y luego me va decir que los archivos fueron

validados y que se movió a un queue state. ¿Qué significa esto? Significa que OpenAI, al mismo tiempo que nosotros estamos mandando nuestro proceso de

fine tuning, tiene cientos de miles más, no somos los únicos utilizando la plataforma de OpenAI, o puede ser que ChatGPT, la plataforma donde nosotros

interactuamos con GPT, esté siendo muy usada y los recursos computacionales de OpenAI se estén empleando ahí, y por eso entra una cola donde va

esperar hasta que haya los recursos suficientes para poder procesarse. Yo, en este caso, lo cancelé de manera manual, pero puede pasar de que el

trabajo también se cancele de manera automática, porque OpenAI realmente no es capaz de procesar la información en un período de tiempo incluso largo. Pero

no pasa nada, usando el Notebook que ya hemos visto en esta clase, podemos mandar a crear otro, que sería este que yo creé un

par de horas después. Entonces, nos vamos de nuevo a la parte de mensajes, ya vamos a ver las gráficas con métricas, vamos a ver

qué hizo lo mismo, creó el trabajo, validó el archivo, fue movido una cola, y si se dan cuenta acá dice que fue a las

tres y cuatro, pero luego a las diez de la mañana empezó el trabajo. ¿Y qué pasa? Yo lo dejé a las tres de la

tarde del cinco de mayo y el trabajo realmente empezó hasta el seis de mayo las diez y cincuenta y cinco, o sea, una cola

casi casi de diecinueve horas, es decir, tuve que esperar bastante para que pudiera empezar el proceso. Entonces, acá vemos que el trabajo de fine

tuning empezó, luego se crean varios checkpoints en una cantidad de repeticiones, acá lo ven en el step cien y en el step doscientos, y

luego, pues ya el trabajo realmente terminó de las diez y cincuenta y cinco a las once y veinticinco, aproximadamente unos treinta minutos. Luego de

que el entrenamiento termine, OpenAI nos va proveer de diferentes métricas. Vamos a revisar estas métricas una a una. La primera y, de pronto, la

más importante para algunos de ustedes es cuántos tokens de entrenamiento gastamos, ya que, pues, toques de entrenamiento es realmente con lo que OpenAI nos

cobra. Ese número, en este caso particular, para ese dataset son ciento dieciséis mil doscientos ochenta y tres. En su caso, como ustedes van hacer

un análisis adicional del dataset que yo dejé, puede que suba o disminuya esa cantidad de toques, depende del preprocesamiento que ustedes le hagan al

dataset. Luego vamos a ver los hiperparámetros. En este caso, que nosotros dejamos que las épocas fueran de manera automática, o la cantidad de repeticiones

y OpenAI escogió tres. Luego el batch size, que ese sí lo definimos, y luego el learning rate multiplier, que fue este el que pusimos.

Luego van a ver un Este hace referencia a la semilla con la que van a empezar la generación de números aleatorios que OpenAI va

a usar para el estado inicial del modelo. Recordemos que realmente en las computadoras no existen números aleatorios, números pseudo aleatorios, y esta semilla es

la base para que se creen estos números aleatorios. De hecho, así funciona Minecraft, por ejemplo, o muchos videojuegos con semillas procedurales, donde lo que

hacen es con una misma semilla se genera el mismo mundo. En este caso, nosotros en el notebook podríamos pasarle una semilla, ustedes pueden usar

esta misma y se van dar cuenta de que las métricas que vamos a ver más adelante van a ser muy similares, ya que aparten

del mismo punto inicial. Luego vamos a ver los checkpoints. En este caso se crearon tres, el primero que fue cuando creó el modelo, luego

en el step cien y en el step doscientos. Acá vas ver que puedes ir a ver y usar cada uno de estos steps, estos

checkpoints en el playground para que veas la diferencia del modelo base, del modelo con cien pasos de entrenamiento y con doscientos haciéndole diferentes preguntas.

Ya vamos a ir al playground. Luego vamos a ver los archivos. Tenemos primero el de entrenamiento y luego el de test. Y luego vamos

a ver la métrica final que OpenAI usa en el momento de el entrenamiento. La primera va ser el train loss, luego el valid loss

y luego el full valid loss. Estas métricas, si recuerdas nuestras clases donde estábamos explicando cómo funciona un LLM y una inteligencia artificial en general,

pues es la métrica que nos dice realmente si nuestro modelo está aprendiendo o no. Y ya finalmente vamos a ver unas gráficas para cada

una de estas métricas. Entonces, por ejemplo, acá podemos ver el valid y el train loss, y vamos a ver que en este caso se

ve muy sharp. Entonces, vamos a voy a agregarle un smoothie para que veamos una gráfica un poquito más fácil a nuestros ojos de ver,

y vamos a ver cómo el valid loss, que es la línea azul, va bajando y llega a un punto en que ya no sigue

bajando, sino que empieza subir. ¿Esto qué quiere decir? Recordemos que no le pasamos toda la información, estamos haciendo un entrenamiento más sencillo para que

no tome tantas horas. Pero si ustedes quieren probar diferentes hiper parámetros y pasarle más información, van a ver que la línea va a entender

a bajar. Lo mismo pasa con la de training y la de valid. En este caso vemos que se estabilizan un punto y no se

sigue moviendo. Luego vamos a ver el accuracy. Acá vamos a ver tanto el train como el valid. Este accuracy se calcula con el conjunto

de test, con el archivo de test que nosotros le pasamos, usando la pregunta y la respuesta que está en ese archivo. Acá vemos que

nuestro modelo en entrenamiento al final obtuvo un sesenta por ciento de accuracy y en el valid obtuvo un poquito menos del cincuenta y seis

por ciento. No es nada bueno, recordemos también no le pasamos toda la información, usamos unas épocas bastante poquitas para que el entrenamiento no tomara

tanto tiempo, si ustedes quieren pueden hacer pruebas pasándole toda la información, por ejemplo, y aumentando la cantidad de épocas para que el modelo aprenda

y van a ver cómo ese accuracy va tender hacia arriba. Y ya, esas son las tres gráficas que nos muestran y las tres métricas

que nos da principalmente OpenAI. Como ellos son los que tienen el control de cómo se entrena el modelo, realmente no podemos hacer un análisis

más a profundidad. Eso ya lo podemos hacer cuando estemos haciendo fine tuning de manera más manual con Hue again face, como lo vamos a

en la siguiente clase. Pero bueno, ya tengo entrenado mi modelo. ¿Cómo lo pruebo ahora? Vamos a usar el playground de OpenAI. Para usar nuestro

modelo fine tunable, le vamos a dar playground. Acá OpenEd nos da una interfaz muy buena para probar realmente la diferencia entre el modelo base

que usamos para hacer fine tuning y nuestro modelo. Yo lo que voy a hacer es voy a ir a mi notebook, voy a buscar en mis

archivos el dataset y voy a seleccionar cualquiera de los ejemplos que nosotros tenemos acá, por ejemplo. Vamos a usar esta pregunta. ¿Qué qué

encuentros en un laboratorio están asociados con la hemofilia? Entonces, vamos a ir, voy a pegar esa respuesta y le voy a preguntar. A la izquierda está

el modelo de base Y a la derecha está el modelo de nosotros, el que acabamos de entrenar. Entonces, nosotros vemos la respuesta, acá en este

caso el modelo base nos está diciendo que la hemofilia es un desorden de la sangre y nos da varias varias puntos que hacen referencia

a él. Si nosotros vemos la respuesta de nuestro modelo veintinueve es mucho más concisa, no se extiende tanto. Y si nosotros, por ejemplo, vamos

y buscamos acá la respuesta que nos da el dataset, vamos a ver que puede ser muy similar a lo que nos está dando nuestro modelo.

Voy a pegarlo acá nada más para poder compararlo, pero miren, la respuesta es concisa y acá vemos la explicación de cada uno de ellos,

la APTT, la PT y otros factores que hacen parte de los de lo que podemos encontrar en un estudio de hemofilia en un laboratorio.

Entonces, un poco más, algo que creo que veamos en este ejemplo, que se demoró un poco más, se demoró casi dos segundos más se

dan la respuesta. Tengan en cuenta de que el modelo base de p t cuatro, seguramente en los data centers donde se ejecuta esto ya

está cargado en memoria. Es decir, tiene que esperar a que el modelo se cargue en memoria para usarse, en cambio el nuestro sí. En

ese entonces en las primeras preguntas podemos tener un poco menor de latencia, pero algo que quiero que vean también es la cantidad de que

nos dio respuesta, fueron ciento ochenta y un tokens versus doscientos ochenta y nueve tokens. Y es que esto es lo poderoso. No sé si

les ha pasado, pero cuando usan modelos de lenguaje grandes, puede que la respuesta a veces sea muy extensa y tenga muchas cosas que realmente

no le dan sustancia a lo que nosotros realmente queremos saber.

Necesitamos a veces respuestas más concisas y rápidas. Ahí es donde el fine tuning

es poderoso, porque nosotros podemos modificar al modelo para que a respuestas, preguntas puntuales nos dé respuestas más claras, sin que dé tantas vueltas y

sin que alucine. Entonces, es el poder, por ejemplo, acá pasamos de doscientos noventa y ocho a ciento ochenta y un tokens. Eso, si tú

lo multiplicas por una cantidad grande de preguntas que le estés haciendo, puedes estarte ahorrando cientos de dólares o miles de dólares dependiendo de cuántos

clientes tengas y cuántas personas estén usando tu modelo, o si tú eres el que lo usa de manera personal y lo estás usando en

tu día a día. Entonces, esto ya podemos ir comparando, y eso también es bueno, porque de esta manera yo puedo saber dónde es que

mi modelo está flaqueando y mejorar mi dataset, es un proceso iterativo. Recordemos cuando estábamos explicando cómo funcionaba el entrenamiento de un modelo de lenguaje

natural, y es que al final nosotros necesitamos darle buena información, tener un dataset curado, y esto nos ayuda porque podemos ver qué nos está

dando el modelo base y cuáles son las mejoras que nosotros necesitamos introducir en nuestro dataset para tener un mejor modelo.

Resumen

¿Qué es LoRa y cómo facilita el fine tuning?

¿Quieres adaptar modelos enormes como GPT-4, pero no tienes los recursos de grandes empresas? LoRa, o *Low-Rank Adaptation*, permite realizar fine tuning sobre modelos gigantes con menos recursos computacionales. En lugar de entrenar la totalidad de un modelo con millones o incluso billones de parámetros, LoRa entrena solamente pequeñas matrices, disminuyendo enormemente el uso de memoria y tiempo del proceso.

¿Por qué entrenar con LoRa y no con métodos tradicionales?

Cada nueva versión de grandes modelos lingüísticos (LLMs) aumenta significativamente la cantidad de parámetros. Entrenar todos estos parámetros

sería caro y poco escalable para individuos o pequeñas organizaciones. Por ejemplo, GPT-4 tiene alrededor de un trillón de parámetros y su entrenamiento completo requeriría cientos o miles de GPUs. Aquí es donde LoRa destaca:

- Permite entrenar matrices de pesos considerablemente más reducidas (r por d en lugar de d por d).
- Reduce significativamente las operaciones por segundo necesarias.
- Facilita que cualquier usuario pueda realizar procedimientos efectivos de fine tuning.

¿Cómo funciona exactamente LoRa?

LoRa aprovecha la llamada hipótesis del rango intrínseco, según la cual con pocos ajustes puntuales en pequeñas matrices es posible adaptar el comportamiento del modelo más grande. Esto implica:

- Congelar la gran matriz de pesos original.
- Crear matrices reducidas que sí serán entrenadas.
- Integrar, al final, estas matrices con la matriz original para adaptar el modelo al nuevo caso de uso.

Si originalmente se necesitaba manejar matrices de pesos gigantes (e.g., $100,000 \times 100,000$), con LoRa estas matrices pueden reducirse sustancialmente (e.g., $8 \times 100,000$).

Implementando LoRa con recursos gratuitos

Usando una cuenta gratuita de Google Colab, y empleando librerías como Parameter Efficient Fine Tuning (PEFT) y transformers de Hugging Face, se puede realizar de manera sencilla:

- Instalar librerías necesarias como PyTorch, PEFT y CUDA.
- Seleccionar instancias gratuitas como la GPU T4.
- Definir claramente hiperparámetros clave para LoRa (e.g., rango de reducción, alpha, dropout).
- Preparar los datos específicos para entrenamiento supervisado.
- Formatear correctamente la entrada usando *tokenizers* específicos de cada modelo.

Es posible usar modelos open source indicados especialmente para fine tuning, por ejemplo, aquellos identificados con etiquetas como *Instruct* en Hugging Face. Estos modelos, además de ya estar diseñados para facilitar el fine tuning, permiten realizarlo eficazmente con un menor número de parámetros entrenados.

Entrena tu modelo con LoRa paso a paso

La ejecución del fine tuning con LoRa en Colab incluye pasos concretos y simples:

1. Configurar y cargar tu modelo cuantizado.
2. Reducir dimensionalidad con LoRa en matrices puntuales del modelo.
3. Preparar y formatear tu dataset correctamente para el entrenamiento supervisado.
4. Ejecutar el entrenamiento supervisado, validando constantemente la reducción del loss.

Como resultado del proceso, puedes tener adaptadores específicos para distintos casos de uso, reduciendo el espacio requerido de gigabytes a megabytes.

¿Te gustaría experimentar personalmente con LoRa? Déjanos tus comentarios sobre tus experiencias con este proceso o tus preguntas para que sigamos aprendiendo juntos.



Fine-tuning eficiente con LoRA sobre modelos LLM cuantizados



¿QUÉ HACE ESTE SCRIPT?

Realiza el fine-tuning eficiente de un modelo LLM (Phi-3 Mini de Microsoft) usando:

- Cuantización 4-bit (reduce memoria)
- LoRA (Low-Rank Adaptation) (adapta solo una fracción del modelo)
- Dataset conversacional estilo Yoda
- Entrenamiento optimizado en GPU con bitsandbytes, peft, trl y transformers



1. IMPORTACIÓN DE LIBRERÍAS

¿Por qué se usan?

- transformers: carga modelos y tokenizadores de Hugging Face
- peft: aplica técnicas de fine-tuning eficientes como LoRA
- trl: entrena modelos para tareas conversacionales (SFTTrainer)
- bitsandbytes: permite cargar modelos en 4-bit
- datasets: acceso fácil a datasets listos para NLP
- safetensors, pandas, matplotlib, numpy: formatos de peso y análisis auxiliar
- huggingface_hub: subir el modelo entrenado al Hub



2. CARGA DE MODELO CUANTIZADO

Modelo base: microsoft/Phi-3-mini-4k-instruct

Cuantización:

- 4-bit con BitsAndBytes

-  Reduce uso de VRAM
-  Computación en bfloat16
-  Ideal para entrenar en GPU con poca memoria

Resultado: Carga rápida y eficiente de un LLM listo para ser adaptado.

3. ADAPTACIÓN CON LoRA

¿Qué es LoRA?

- Técnica de entrenamiento que congela el modelo base y entrena solo pequeños adaptadores.

Ventajas:

-  Mucho más rápido
-  Consume menos memoria
-  Solo adapta capas específicas (como qkv_proj, down_proj, etc.)

Configuración usada:

- r=8, alpha=16, dropout=0.05

4. DATASET CONVERSACIONAL (YODA)

Origen: dvgodoy/yoda_sentences (Hugging Face)

Proceso:

- sentence → prompt
- translation_extra → completion
- Se elimina la columna translation
- Se convierte a estructura tipo chat (messages)

Ejemplo de formato final:

```
[
  {"role": "user", "content": "Do or do not."},
  {"role": "assistant", "content": "There is no try."}
]
```

5. ENTRENAMIENTO

Tokenizador: Adaptado a formato de conversación (chat_template) y con pad_token

Configuración SFT:

-  num_train_epochs = 10
-  max_seq_length = 64
-  paged_adamw_8bit como optimizador
-  gradient_checkpointing activado para ahorro de memoria

Trainer usado: SFTTrainer (de trl)

6. INFERENCIA

Función: Genera respuestas del modelo fine-tuneado usando plantilla de chat

Ejemplo:

```
sentence = 'The Force is strong in you!'
```

→ respuesta adaptada estilo Yoda

7. GUARDADO Y PUBLICACIÓN EN HUGGING FACE

Pasos:

1. `save_model()` guarda el modelo localmente
2. `login()` autentica con tu token
3. `push_to_hub()` publica el adaptador entrenado en Hugging Face

Here's a no-code-friendly breakdown of LoRA (Low-Rank Adaptation) for fine-tuning giant models efficiently! 🎯✨

1. What is LoRA?

LoRA is a technique to fine-tune massive models (like GPT-3) without modifying all their weights—saving 90%+ compute! 🚀

- Instead of updating all 175B+ parameters, LoRA adds tiny *adapters* (low-rank matrices) to specific layers (e.g., attention heads).
- Think of it as "model stickers" that tweak behavior without rewriting the whole brain! 🧠🔧

2. How LoRA Works (Visually)

Imagine a pretrained model as a frozen pizza 🍕:

- Traditional Fine-Tuning: Re-bakes the whole pizza (expensive!).
- LoRA: Adds lightweight toppings (e.g., pepperoni = task-specific adapters). Only the toppings are trained!

Key steps:

1. Freeze the original model weights.
2. Inject small matrices (e.g., 100x100) into transformer layers.
3. Train only these matrices on your data (e.g., medical reports 🩺 or legal docs ⚖️).

3. Why LoRA is a Game-Changer

- GPU Memory: Needs ~3x less memory than full fine-tuning.
- Speed: 2-5x faster training (adapters are lightweight!).
- Portability: Share/patch adapters without distributing the whole model.

4. Using LoRA Without Coding

Tools like Hugging Face PEFT (Parameter-Efficient Fine-Tuning) or RunPod offer LoRA fine-tuning via:

- GUI Platforms: Upload your dataset, select LoRA, and click "Train" (e.g., Lamini, Modal).
- Cloud Services: AWS SageMaker or Google Vertex AI with LoRA presets.

Want a specific tool recommendation? 😊

Example: Fine-tune GPT-3 for customer support with LoRA—just upload chatlogs and pick layers to adapt!

Visual Recap: Original Weights 📦 + LoRA Adapters 🧩 = Task-Specialized Model 🏆





$$W = \mathbb{R}^{D \times d}$$

🧠 Fine-tuning eficiente con LoRA sobre modelos LLM cuantizados

🔧 ¿QUÉ HACE ESTE SCRIPT?

Realiza el fine-tuning eficiente de un modelo LLM (Phi-3 Mini de Microsoft) usando:

- ♦ Cuantización 4-bit (reduce memoria)
- ♦ LoRA (Low-Rank Adaptation) (adapta solo una fracción del modelo)
- ♦ Dataset conversacional estilo Yoda
- ♦ Entrenamiento optimizado en GPU con bitsandbytes, peft, trl y transformers

📦 1. IMPORTACIÓN DE LIBRERÍAS

¿Por qué se usan?

- transformers: carga modelos y tokenizadores de Hugging Face
- peft: aplica técnicas de fine-tuning eficientes como LoRA
- trl: entrena modelos para tareas conversacionales (SFTTrainer)
- bitsandbytes: permite cargar modelos en 4-bit
- datasets: acceso fácil a datasets listos para NLP
- safetensors, pandas, matplotlib, numpy: formatos de peso y análisis auxiliar

- `huggingface_hub`: subir el modelo entrenado al Hub

2. CARGA DE MODELO CUANTIZADO

Modelo base: `microsoft/Phi-3-mini-4k-instruct`

Cuantización:

-  4-bit con `BitsAndBytes`
-  Reduce uso de VRAM
-  Computación en `bfloat16`
-  Ideal para entrenar en GPU con poca memoria

Resultado: Carga rápida y eficiente de un LLM listo para ser adaptado.

3. ADAPTACIÓN CON LoRA

¿Qué es LoRA?

- Técnica de entrenamiento que congela el modelo base y entrena solo pequeños adaptadores.

Ventajas:

-  Mucho más rápido
-  Consume menos memoria
-  Solo adapta capas específicas (como `qkv_proj`, `down_proj`, etc.)

Configuración usada:

- `r=8`, `alpha=16`, `dropout=0.05`

4. DATASET CONVERSACIONAL (YODA)

Origen: `dvgodoy/yoda_sentences` (Hugging Face)

Proceso:

- `sentence` → `prompt`
- `translation_extra` → `completion`
- Se elimina la columna `translation`
- Se convierte a estructura tipo chat (`messages`)

Ejemplo de formato final:

```
[
  {"role": "user", "content": "Do or do not."},
  {"role": "assistant", "content": "There is no try."}
]
```

5. ENTRENAMIENTO

Tokenizador: Adaptado a formato de conversación (chat_template) y con pad_token

Configuración SFT:

- 🔄 num_train_epochs = 10
- 📏 max_seq_length = 64
- 🚀 paged_adamw_8bit como optimizador
- 📊 gradient_checkpointing activado para ahorro de memoria

Trainer usado: SFTTrainer (de trl)

🔍 6. INFERENCIA

Función: Genera respuestas del modelo fine-tuneado usando plantilla de chat

Ejemplo:

```
sentence = 'The Force is strong in you!'
```

→ respuesta adaptada estilo Yoda

☁️ 7. GUARDADO Y PUBLICACIÓN EN HUGGING FACE

Pasos:

1. save_model() guarda el modelo localmente
2. login() autentica con tu token
3. push_to_hub() publica el adaptador entrenado en Hugging Face

Perfecto, ya tienes tu primer modelo usando el GPT cuatro o. Como viste, tomó bastante tiempo, depende mucho, porque la carga de OpenAI varía dependiendo

del día y entras en una cola donde tienes que esperar tiempo. Pero, ¿qué pasa si yo quiero en particular usar un modelo totalmente distinto

a los de OpenAI? Porque en este caso los modelos de OpenAI son de código cerrado, realmente no podemos saber ni cuánto cómputo usan, ni

cuánto tiempo ni dinero nos está costando realmente fine tunear ese modelo. Para esto, existen formas de entrenar modelos bastantes grandes y open source de

manera local o incluso con recursos más moderados. Acá es donde entran técnicas más avanzadas de fine tuning específicas. La que hoy vamos a ver

se llama LoRa, Load Rank Adaptation. Pero, ¿qué es LoRa y qué soluciona en particular? Particularmente soluciona el problema de tener que entrenar modelos gigantes.

En los últimos años hemos visto cómo cada uno de los modelos nuevos tiene decenas de miles de millones de parámetros adicionales versus su versión

anterior, y eso es un problema, porque si nosotros queremos hacer un fine tuning supervisado como el que vimos en la clase anterior, tendríamos que

reentrenar el modelo completo. Es decir, el caso de GPT cuatro, por ejemplo, tendríamos que reentrenar cerca de un trillón de parámetros. Eso es mucho,

requerirías de cientos o miles de GPUs para poder hacer un fine tuning a tu medida en tu casa, por ejemplo, y eso no escala.

Por esto, LoRa lo que soluciona es que vamos a reentrenar partes mucho más chiquitas de los parámetros, no vamos a reentrenar el modelo completo,

sino vamos a hacer de una manera más inteligente el entrenamiento en partes puntuales del modelo para que esto requiera mucho menos cómputo y mucho

menos información a la hora de hacer el entrenamiento. Pero, ¿cómo funciona LoRa? Se basa en un principio fundamental, y es que ellos plantearon que

uno no necesita reentrenar todo el modelo completo para que el modelo pueda captar cambios significativos basados en los datos. Esto se llama la hipótesis

del rango intrínseco, donde básicamente lo que nos dice es que nosotros podemos tener una matriz mucho más chiquita donde entrenemos el modelo y luego

combinarla con el modelo más grande. Recuerda que cuando nosotros vimos los fundamentos de la arquitectura de un LLM, hablábamos de la matriz de pesos.

Esta matriz de pesos es la que realmente tiene la carnita, lo importante del modelo donde se aprenden los conceptos, y esa matriz de pesos

normalmente tiene un tamaño d por d . Ese tamaño d por d depende de la cantidad de tokens y de la cantidad de parámetros, pero

por ejemplo, si nosotros tenemos modelo con diez trillones de parámetros, probablemente esa matriz de pesos d por d sea de diez trillones, o sea,

va ser cerca de cien mil por cien mil. LoRa lo que dice es que nosotros podemos coger el modelo preentrenado, congelarlo y crear una

matriz mucho más chiquita, en rangos. Recordemos que el ahora viene de low rank adaptation. Ese rank y ese low hace referencia a que nosotros

vamos a reducir la dimensionalidad de esa matriz. Entonces, en vez de ser d por d , ahora va a ser r por d . Ese r es

el rango que nosotros le queremos dar a la matriz de Lora para su entrenamiento, y van a existir muchas matrices más pequeñas r por

d en cada una de estas capas. Esto hace que no solo no tengamos que entrenar todos los parámetros, sino que computacionalmente tengamos que ejecutar

muchas menos operaciones por segundo, porque ya no vamos a tener que hacer una matriz cien mil por cien mil, sino puede ser ocho por

cien mil. Esto lo vamos a ver mucho más claro en el código, lo que quiero decirte es que Lora lo que va a hacer es,

a entrenar esta matriz particular, va a aplicar el back propagation sobre esta matriz en particular y al final lo único que va a hacer es combinar

la matriz de pesos original, esta matriz nueva, en una sola, y eso va a hacer que el modelo aprenda según nuestros datos. Pero, ¿cuál es

la ventaja principal de esto? Tomemos el ejemplo que hay, una matriz de pesos cien mil por cien mil. Esto significa que vamos a tener

diez billones de parámetros, si multiplicamos cien mil por cien mil, este es el valor que nos da diez billones. Y si aplicamos LoRa y

en vez de entrenar esta matriz de cien mil por cien mil, aplicamos un rango, un r de tres, esta matriz que vamos a entrenar

de nuevo va a ser tres por cien mil. Eso va a querer decir que vamos a bajar de diez billones a trescientos mil parámetros.

Esto baja significativamente la cantidad de memoria de GPU que yo necesito para entrenar el modelo, y no solo eso, también baja la cantidad de

tiempo que voy a tener que gastar en cada una de estas operaciones. Esto es muy eficiente y LoRa lo que nos permite es entrenar diferentes

matrices para cada caso de uso. Eso quiere decir que podemos tener un modelo preentrenado base y diferentes adaptadores. Esto lo vamos a ver mejor

en el código, pero cada uno de estos adaptadores puede ser entrenado con un dataset diferente y vamos a poder tenerlos e irlos cargando dependiendo

de nuestro caso de uso. Y eso hace que bajemos de gigas de peso de los modelos a incluso megabytes. Eso hace muy eficiente el

poder ejecutar estos modelos incluso en dispositivos que no tengan las capacidades de memoria que tienen los grandes centros de datos de OpenAI, por ejemplo.

Ahora viene la parte más divertida de todo, programar y entrenar nuestro primer modelo haciendo fine tuning con LoRa. Para esto, a usar la librería

Parameter Efficient Fine Tuning de Hugging Face, que nos va a ayudar a ahorrarnos mucho trabajo en líneas de código y no tener que hacerlo

desde cero. Yo soy un afán de hacerlo desde cero, pero para este curso te voy mostrar cómo usar esta librería que voy llamar de

ahora en adelante PEF en nuestro notebook de Collaf. Este notebook lo puedes encontrar en los recursos del material. Ahora, modelo que vamos a usar

para hacer nuestro fine tuning con Lora va ser uno chiquito para poder usarlo en Collaf con la cuenta gratuita. Si ustedes quieren pagar por

Collaf y tienen acceso a las máquinas más grandes, van a poder hacer este fine tuning con modelos mucho más grandes. En este caso, vamos

a usar Microsoft Fit tres mini cuatro k Algo importante es que los modelos que tienen la etiqueta en el nombre Instruat son los modelos

que son los mejores para usar en fine tuning como el caso que vamos a hacer, porque son modelos que son pretrained desde su

base, es decir, no tienen muchos más datos de entrenamiento, sino que están especializados para tomarlos como base y entrenar un modelo nuevo. En este

caso, este modelo en particular es de tres punto ochenta y dos billones de parámetros, casi cuatro billones de parámetros. Recordemos las clases anteriores donde

les expliqué más o menos cómo funcionaba esta interfaz de Hugging Face, acá van a encontrar mucha más información, incluso van a encontrar acá adaptadores,

que es el que vamos a entrar en este momento, fine tunes y cuantizaciones. Entonces, con esto ya tenemos nuestro modelo base. Ahora, en nuestro

collab, lo primero que tenemos que verificar es que estemos instanciando una instancia t cuatro. T cuatro es una instancia que viene ya con memoria

de GPU, en este caso quince gigabytes, y es la mejor instancia que podemos tener con la cuenta gratuita de Colab. Recordemos que si pagamos,

pues vamos a tener acceso a máquinas mucho más poderosas. Lo primero que tenemos que hacer es instalar Hugging Face, Transformers y PEF, y un

par librerías más que van a hacer uso de la GPU. No es necesario que lo entiendas, esas librerías específicamente están hechas para sacarle todo

el jugo a la GPU de manera automática, sin que nosotros tengamos que meternos muy profundamente en el funcionamiento. Entonces acá esto lo que va

empezar a hacer es instalar un montón de librerías de CUDA, de NVIDIA, puede tomar un tiempo, tranquilízate, porque está descargando mucha información, pero mientras

hace eso, vamos un poco más abajo en el código. Entonces, lo siguiente que vamos a hacer es instalar Torch, recordemos nuestra clase de PyTorch

introductoria, donde les expliqué cómo funcionaba Torch de una manera básica, acá lo vamos a usar de nuevo. Luego, de PEF, vamos a PEF, recordemos

que hace referencia a la librería de Hugging Face para poder hacer fine tuning con LoRa, vamos a traer que nos traiga el modelo vamos

a traernos la configuración de LoRa y vamos a traernos una función que lo que va hacer es ayudar a preparar al modelo para un

entrenamiento óptimo en nuestro sistema. Luego de transformers, lo que vamos a traer son las cosas que nos van a ayudar a traernos el modelo

de Microsoft, el que definimos anteriormente. Entonces, a traer automóvil for causal m l l m, que básicamente lo que va hacer es cargar el

modelo, el tokenizer, el auto tokenizer, que lo que nos va hacer es dar un modelo, traernos el tokenizer que se usó para ese modelo,

así nos lo tenemos que ir a buscar y traerlo uno, no, de manera manual. Luego tenemos bytes and bytes config, que lo que va

hacer es cargar el modelo cuantizado. En la siguiente clase vamos a ver qué es un modelo cuantizado, ya vamos a llegar hasta allá. Entonces,

primero que tenemos que hacer es definir la configuración de este modelo cuantizado. Acá lo que le estoy diciendo es que lo cargue en cuatro

bits, que use doble cuantización y el tipo de cuantización o el tipo de método que se usó para cuantizar y el tipo dato que

está usando el modelo originalmente. Bueno, acá me aparece un error que me dice que tengo que reiniciar la sesión. ¿Por qué me pasa esto?

Yo particularmente, pues, ya he ejecutado este notebook y ya tenía la instancia previamente usada. Entonces, como estamos instalando nuevas versiones de MapLopLip y MPL

Toolkits, nos pide reiniciar la sesión para, pues, usar esas nuevas versiones que acabo instalar. Entonces, no hay ningún problema, simplemente lo reiniciamos, va tomar

un tiempo y volvemos a ejecutar esa parte para que ejecute de nuevo la instalación. Ahí lo que nos va decir es que todo está

satisfied porque literalmente lo acabo instalar. Entonces, miren, lo hizo muchísimo más rápido, en este caso ya terminó, podríamos ejecutar las siguientes celdas. Entonces, vamos

a ejecutarlas y continuamos acá en la cuantización. ¿Cómo yo sé el tipo de dato que tengo que decirle que es la base del modelo?

Y es que si nosotros volvemos al model card en Hugging Face, lo vamos a ver acá en Tensor Type, que dice BF dieciséis. Entonces,

Storch tiene un tipo de dato específicamente para BF dieciséis, que es BFloat dieciséis. Tenemos que especificarle muy bien el tipo de dato principal y

al cual queremos llevar. Le estamos diciendo queremos cuatro bits de dieciséis. Luego, ponemos el Repo ID. El es muy fácil, si volvemos a al

viewing face, vamos a ver este icono de copiar, le damos clic y ya lo tendríamos en nuestra papelera, simplemente lo ponemos acá. Ya con

eso tendríamos el repo ID para poderlo traer. Luego, para cargarlo usamos AutoModel for Causal LM, que lo que va a hacer es literalmente buscar

ese Repo ID, cargarlo dentro de nuestro device. Recordemos nuestra clase de PyTorch, donde hablábamos de los dispositivos que podíamos usar, en este caso, tenemos

una tarjeta gráfica NVIDIA con CUDA, le decimos CUDA cero. Cero, ¿por qué? Porque solo tenemos una, si tuviéramos múltiples, pasarle una lista de dispositivos

y le pasamos la configuración de la cuantización. Entonces, cuando ejecutemos este en particular, lo que él va empezar a hacer es a descargar todos

los safe tensors, que son, digamos, los pesos del modelo. Esto puede tomar un tiempo, vamos a dejarlo ahí, sobre todo porque es bastante grande

el modelo. Entonces apenas descargue, volvemos con la siguiente parte del código. Listo, con esto ya tenemos nuestro modelo cargado de memoria, duró cerca de

un minuto. Algo importante es que estamos descargando cosas de Face, para eso necesitamos estar autenticados. ¿Cómo se autentica uno en Hugging Es muy sencillo,

solo hay que traer el token de nuestra cuenta y ponerlo en nuestras llaves de entorno, en este caso HF Token y acá yo ya

puse mi valor. Luego de esto, digamos que quiero saber cuánta memoria está ocupando mi modelo. Con la función get memory footprint, él me va

devolver la cantidad de bytes que está usando el modelo. Para saber cuántas gigas son omegas, vamos a usar la división sobre un millón. Este

uno e seis hace referencia a un millón, porque lo podemos escribir de esta manera. Es equivalente, es como una anotación que tiene Python. Entonces,

ejecutamos esta función y nos va decir que pesa dos mil doscientas megas, más o menos unas dos gigas, es bastante pesado el modelo. Pero

ahorita con Lora vamos a ver cómo el modelo entrenado con nuestros datos va a pesar significativamente menos que esas dos gigas. Acá podemos ya

ver la estructura del modelo, vamos a ver que tiene un embedding, un dropout, que un dropout lo que hace es desactivar cierta neurona en

el entrenamiento para que el modelo no memorice. Luego vamos a ver que tiene una decodificación que tiene la la atención, que fue lo que

vimos cuando vimos la estructura de GPT dos. Y acá podemos ver todo. Aparte, también vemos el MLP, que es el multilayer perceptron, y vemos

ya la parte final, que es la que selecciona los tokens basados en la probabilidad. Ya con el modelo cargado, podemos empezar a hacer la

configuración de nuestro Lora para hacer lo que realmente nos importa. Entonces, lo primero que tenemos que hacer es preparar nuestro modelo para un entrenamiento.

De verdad que Hugging Face con sus librerías nos hace la vida muy fácil, porque con la función prepare model for COVID training, y le

paso el modelo, él va a preparar todo el entorno, la GPU, las operaciones y demás, para que se pueda entrenar el modelo cuantizado de

una manera más eficiente. Por eso tenemos el cabbit. Luego vamos a configurar el LoRa. Eso es muy sencillo, con la clase LoRa config a

esta clase le pasamos los hiperparámetros. Recordemos que los hiperparámetros son parámetros que nosotros seleccionamos y que realmente no existe una fórmula mágica para nosotros

seleccionarlo. La fórmula mágica es prueba y error. Lo primero que tenemos que definir es la r del rank, del rango, es decir, qué tanto

queremos que esa matriz gigante se reduzca. En este caso, yo le pongo ocho. Recordemos nuestro ejemplo de cien mil por cien mil. En este

caso, estoy convirtiendo esa matriz cien mil por cien mil, estoy creando otra parte que es ocho por cien mil y la de cien mil

por cien mil la dejamos congelada, es decir, no hacemos ninguna operación sobre ella. Luego modificamos el Lora alfa. Este es un multiplicador, este es

similar al Learning Rate que lo que nos va ayudar a mantener un entrenamiento más eficiente. Normalmente este Lora alfa debe ser dos por r ,

en este caso como nuestro r es ocho, pues nuestro Lora alfa va a ser dieciséis. Luego configuramos el bias. Si queremos agregarlo, realmente no

es necesario, pero te invito a que pruebes qué pasa si tú le agregas el bias con las métricas de validación y de pérdida. En

este caso lo configuró como non. Luego ponemos un LoRa dropout. El dropout es una función que lo que hace es apagar ciertas neuronas o

decirle al modelo, oiga, no las use, en un cierto porcentaje. ¿Qué porcentaje? En este caso le estoy poniendo un cinco por ciento de dropout.

Entonces, cada vez que se haga una inferencia y se esté entrenando el modelo, el cinco de el cinco por ciento de neuronas se va

pagar de manera aleatoria para tener un entrenamiento mucho más eficiente. Luego vamos a ponerle el tipo de tarea. En este caso, modelos LLMs, la

gran mayoría son causales. Causales, ¿qué quiere decir? Que las funciones estadísticas por debajo son causales. Entonces, usamos causal LM. Y luego, para los modelos

más nuevos, esto antes no era así, pero ahora es muy recomendado, vamos poner un target modules. Este target modules va ser referencia a qué

módulos dentro del modelo queremos entrenar. Entonces, por ejemplo, tenemos o Proj. Si nosotros buscamos ese nombre en la definición que tenemos acá del modelo,

vamos a ver qué está dentro de de atención, acá lo vemos, o Proj. También tenemos $k \times k \times v$ Proj y rotatoria embedded, que

son algunos de los que tenemos acá agregados. También tenemos $q \times k \times v$ proc, tenemos gate up y down, que son partes del modelo. Eso

lo que le queremos decir es que eso van a ser los targets donde vamos a coger esa matriz de pesos, la matriz cien mil

por cien mil, y le vamos a bajar la dimensionalidad a ocho por cien mil, en este caso. Entonces, vamos a tener cuatro matrices más

pequeñas, que son las que vamos a entrenar. Y simplemente, para ya tener nuestro modelo con LoRa, lo que vamos a decir es con la

función getPathModel, le pasamos el modelo original y le pasamos la configuración que hay que aplicarle a ese modelo. Cuando nosotros ejecutemos esto y vayamos

a ver realmente de nuevo cómo es la configuración de nuestro modelo, vamos a ver que se van agregar un par de capas adicionales, que

son las capas de lora. Entonces, acá vamos a ver lora a y lora b, que son esas matrices reducidas. Acá lo podemos ver el

ocho y el ocho acá. Eso lo que quiere decir es que redujo la dimensionalidad de nuestras matrices que antes eran tres mil setenta y

dos por tres mil setenta y dos. Ahí es donde vemos lo que realmente ha sido cien dólares, está bajando la complejidad y la cantidad

de información. Lo vamos a ver en o proc, $q \times k \times v$ proc, en cada uno de los modelos donde lo definimos y eso es

lo que realmente se va a entrenar. Acá nosotros podemos pensar, pero el modelo creció en tamaño y la realidad es que no. Le agregamos

un par de capas y esas capas son las que se van a entrenar, el resto de cosas no se van a modificar. Ya con

esto, cuando tengamos definido nuestro Lora, vamos a ver cuánto le aumentamos al memory footprint. En este caso, le aumentamos unas cuatrocientas megas aproximadamente, pasamos

de dos punto, de dos mil doscientas megas a dos mil seiscientas. Esas cuatrocientas megas son las que realmente se van a entrenar y lo

que realmente va tener la información del del aprendizaje sobre nuestro conjunto de datos que vamos a usar. También tenemos un par métricas adicionales, en

este caso es cuántos parámetros vamos a entrenar, cuántos parámetros totales tenemos y qué porcentaje de parámetros son los entrenables. Esto lo podemos hacer usando

la función `getNV trainable parameters`, nos devuelve cuántos son los parámetros que se van a entrenar y cuánto es el total. Entonces, este caso me

dice que los parámetros totales son tres mil tres mil ochocientos treinta y tres millones o tres punto ocho billones, que es lo que veíamos

en el card de Hugging Face, y los que realmente vamos a entrenar son doce millones de parámetros, que es el cero punto treinta y

tres. O sea, estamos dejando por fuera el noventa y nueve punto siete por ciento de los parámetros, porque los queremos mantener así, solo queremos

entrar el modelo para una tarea específica. Ahora, para el entrenamiento, ¿qué dataset vamos a usar? Vamos a usar uno muy divertido que se llama

Yoda sentences, que lo único que hace es tomar la frase que nosotros queramos a traducirlo a una manera en la que Yoda la diría.

Yoda de Star Wars. Entonces, este es el dataset, es de DV Godoy, Yoda sentences, copiamos el el ID, volvemos a nuestro notebook y lo

vamos a ver acá. Entonces, acá es el que ya tengo puesto, lo volveré a copiar y pegar por si acaso, pero acá les dejo

una línea comentada que es básicamente si quieren usar el mismo dataset con el que entrenamos el modelo de GPT cuatro o. Lo importante es

que le tenemos que decir que nos dé la parte del entrenamiento. Entonces con esto él lo que va empezar a hacer es a descargarlo,

lo descargo súper fácil, son muy pocos ejemplos, la verdad, son setecientos veinte. Luego vamos a hacer un poquito de renaming de las columnas, ¿por

qué? Porque la forma en la que p funciona, también igual que OpenAI, recibe la estructura de los datos de una manera particular, entonces tenemos

que cambiar la pregunta o lo que nosotros queremos que se use como prompt a prompt, y el translation extra, en este caso sería el

completion, es similar a cómo funcionan muchas APIs. Y luego vamos a eliminar la columna de traducción. Ya con esto tendríamos nuestro dataset con las

dos columnas que son obligatorias, la cantidad parámetros puntos de datos serían setecientos veinte y podemos ver un ejemplo. Entonces acá, por ejemplo, está traduciendo

esta frase en un inglés perfecto a una forma en la que la diría Yoda. Es increíble, realmente el modelo va aprender a traducir como

si fuera Yoda. Ahora vamos a hacer un format del dataset, similar a como lo hicimos para la API de OpenAI con FineTune. En este

caso, no necesita un system prompt directamente con user e assistant, podemos hacerlo. Entonces, acá lo que hago es definir una función para hacer un

mapeo, luego pandas tiene la función map, que lo que me permite es que a cada uno de los registros le aplique esta función con

map punto, le pasamos la función y le eliminamos las columnas que no queramos tener. En este caso, solo nos tenemos que quedar con mensajes.

Entonces, lo ejecutamos, lo va hacer muy rápido, acá yo saco un ejemplito de cómo se ve y se vería de esta manera, es una

lista de mensajes que el modelo puede entender. Luego acá nada más le estoy cambiando el nombre y ahora viene la parte de el tokenizer.

Esta parte es muy importante, cada modelo tiene una forma diferente de tokenizar la información, ¿por qué? Porque esto se basa mucho en los datasets

que se usaron para entrenarlo. Entonces, con Auto tokenizer from Proteinnet le pasamos el RepoID, él automáticamente va a ir a buscar cuál es el

template correcto. Entonces, si nosotros lo traemos, va a descargarlo, se puede demorar un poco, de la calidad del Internet. Acabamos ver el template. Realmente

no quiero entrar muy a fondo a qué es esto del template porque podríamos irnos horas a entender cómo realmente el LLM por debajo entiende

la información, pero lo que es es un template para que renderizar cada uno de la de los datos para que el modelo realmente lo

entienda y lo pueda entrenar. Ya vamos a ver un ejemplo particular. Entonces, acá, por ejemplo, yo le estoy diciendo que me aplique el template,

es decir, esto a uno de mis puntos de datos. Entonces, vamos a ejecutarla y vamos a ver que lo que hace es agregar un

par de etiquetas. En vez de pasar el Jason, pasamos unas etiquetas que el modelo realmente va a entender. Recordemos que un LLM entiende lenguaje

natural. Él puede entender Jason, pero a la hora de entrenamiento hay como una un formato específico y cada modelo es diferente. Hay unos estándares,

pero puede ser diferente. En este caso, la etiqueta inicial lo que nos dice es lo que va decir el usuario. Acá ustedes pueden ver

dice usuario y luego acá me dice termina, luego dice asistente, que sería la respuesta y termina, y finalmente la última es que acá realmente

se termina toda la información de el prompt. Hay un error en particular con y esta librería, que el pack token debe ser el token.

Acá voy dejarles en los recursos de la clase un link al issue de GitHub donde explican este problema para no entrar en esta clase

en particular. Simplemente esto lo tenemos que hacer para que funcione. Ahora, ya que tenemos configurado nuestro LoRa, tenemos nuestros conjunto de datos y ya

tenemos la configuración correcta en el que el LLM lo va poder entender, vamos a hacer algo similar a lo que hicimos para OpenAI, pero

en código. Vamos a configurar el Supervised Fine Tuning. OpenAI tiene su forma, como lo vimos en la clase anterior, y pues, PF tiene su

forma específica. En este caso, acá lo podemos ver, SFT config. Lo primero que voy agregarle son dos pasos, dos configuraciones esenciales. La primera es

gradient checkpoint true, que lo que va hacer es ahorrar un montón de memoria y poderlo entrenar en este tipo de máquina específicamente. Lo que

va a hacer esta característica específica es guardar los gradientes, los back, que el gradiente viene del back propagation en la memoria del sistema y no

en la GPU, para ahorrarnos un poco de memoria, lo que a su vez hace que haya un poquito más de latencia, es decir, demora

un poquito más entrenando. Pero como quería ejecutarlo en la cuenta gratuita de Colab, pues le agregué esta característica. Luego lo voy a agregar Gradient Shapoint

Cowards, esto es por PyTorch, principalmente es por un tema de interoperabilidad, pero no voy a entrar en mucho detalle acá. Luego vamos a cada cuánto,

gradation accumulation steps, es cada cuánto queremos que se ejecute el back propagation. Eso lo que hace es que va a modificar un poco el

comportamiento de la GPU para que lo haga de una manera más rápida. Estas tres características en particular son nada más para performance, para poder

ejecutar este tipo de cosas en la máquina gratuita que nos da Google Colab. Luego vamos a configurar el backsize, que en este caso lo

dejé en dieciséis. Si recordamos, para OpenAI le lo pusimos en ocho, pero eso es porque yo sé que tienen unas máquinas mucho más potentes

de la que nosotros tenemos acá. Como tenemos una máquina menos potente, le subí al doble para que lo podamos ejecutar. Luego le agregué un

autofine batch Esto lo hago es porque si el sistema detecta de que la máquina no va a ser capaz de ejecutar tal cual como yo

lo estoy haciendo, como lo estoy configurando, pues que lo haga él automáticamente. Lo que va a hacer es empezar a dividir o a multiplicar por

dos hasta encontrar el backsize correcto. Luego voy a configurarle cuál va a ser la ventana de contexto. Acá depende mucho del problema, yo le voy

poner que la ventana de contexto sean sesenta y cuatro, ¿Por qué? Porque quiero que este modelo de Yoda, pues me pueda traducir frases pequeñas,

no quiero pasarle un texto gigante, sino frases más chiquitas. Luego el packing true lo que hace es empaquetar todo para que sea mucho más

fácil guardarlo y luego desplegarlo. Luego el número de épocas que va a tener el entrenamiento, en este caso le puse diez, el learning rate que

le puse cero coma cero cero cero cuatro tres, que va ser como esto de la patadita para sacarlo de los mínimos locales. Luego le

digo el optimizador, en este caso, recordemos que hemos hablado de y yo les comenté que existían muchos tipos de optimizadores. En este caso, le

puse Adam w de tipo paged de ocho bits. De nuevo, estos optimizadores y varias de las características que yo le puse a esta configuración

de supervised fine tuning es para poder ejecutarlo en la cuenta gratuita. Y luego, en lo que le digo es cada cuánto quiero que me

diga cómo va el proceso, no quiero que me llene la interfaz de un montón de mensajitos, sino que lo haga cada n tiempo. Y

ya el resto parámetros lo que hacen es guardar el modelo en una parte, la carpeta del Oxy y demás. Ya con esto ejecutado, podemos

venir a la parte más interesante y más importante que es el entrenamiento. Ahora, vamos a configurar el agente del sistema que va ser el

entrenamiento. Para esto vamos a usar la clase SFT trainer, que lo que va hacer es, va agarrar un modelo, va agarrar un processing class,

que en este caso hace referencia al tokenizer, va agarrar la configuración del SFT y le tenemos que dar el dataset que queremos que entrene.

Entonces, ya con esto tendríamos preparado en el sistema listo para ejecutar el entrenamiento que va a tomar un tiempo. Luego vamos a definir el

loader, el loader de la información. ¿Para qué? Acá quiero nada más mostrarte un ejemplo de cómo se ven los IDs y los labels que

el LMB al final. Entonces, acá podemos ver que nosotros configuramos que la ventana de contexto sea sesenta y cuatro tokens. Entonces, la entrada va

ser sesenta y cuatro y la salida también va ser sesenta y cuatro por la configuración que le dejamos. Y acá esta línea es la

fundamental. Cuando le demos clic acá, el proceso de entrenamiento va empezar, que puede tomar un tiempo, más o menos de lo que, por ejemplo,

vimos con OpenAI. Recordemos que OpenAI está haciendo un entrenamiento probablemente no de todo el modelo, sino un tipo de lora específico, pero tomó bastante

tiempo. Y acá nosotros estamos con una instancia gratuita de Google Colab que también puede tomar un tiempo. Acá él nos dice un tiempo aproximado

que son veintitrés minutos. Entonces, de nuevo, voy dejar en los recursos de la clase un paper para que te lo puedas leer en estos

veintitrés minutos mientras esperas que se ejecute el entrenamiento. Bueno, después de veinticinco minutos con cuarenta y siete segundos, ya terminó el entrenamiento. Esto es

tiempo suficiente para leer el paper que te dejamos en los recursos del material. Algo importante que vamos a ver acá es que nos reporta

acá diez pasos, que fue como lo configuramos, cuál es nuestro training Es importante que este loss siempre esté bajando. Si el training no está

bajando, significa que nuestro modelo no está aprendiendo. Acá, por ejemplo, en dos punto noventa y ocho y en la iteración ciento diez llegó a

cero punto cincuenta y ocho. Es decir, nuestro modelo iba aprendiendo lo que nosotros queremos que haga. Esto es muy importante, siempre tenlo en cuenta.

Ahora, esto ya podemos usar nuestro modelo, entonces vamos a definir un par de funciones, vamos de nuevo al código, vamos a encontrar la función

en code prompt. Esa función lo que va hacer es agarrar la frase que nosotros queramos traducir y la va poner en una forma en

el que el LLM la entienda. ¿Cuál es esa forma? Con el template. En este caso yo le puse the forces strong in you. La

encodeamos y él nos devuelve que el prompt se vería de esta manera. Luego vamos a definir la función de inferencia. Esa función de es

la que va agarrar nuestro prompt, va a ejecutar el modelo y nos va devolver una respuesta. Lo primero que hacemos es tokenizar esa ese,

porque recordemos que el modelo no entiende letras, entiende tokens que a su vez son embeddings, entonces le pasamos el, que es el que recibe

la función, el tipo de tensor que queremos que devuelva, siempre va a ser p t, y luego le agregamos esta función de que no

agregue tokens especiales, y le decimos que lo mueva al device que estamos usando. En este caso, recordemos que nosotros estamos usando como device CUDA

cero. Luego vamos a hacer un model eval, esto lo que va hacer es que el modelo se ponga en modo inferencia y ya podemos

generar nuestro output. Entonces con model punto generate le pasamos todos los token tokenizados, los inputs tokenizados, le decimos cuál es el n of sentence,

el ID de que le dice que ya terminó ahí, que en este caso sería el que vimos más arriba de of text, y luego

le pasamos cuántos tokens máximos nosotros queremos usar. En este caso, recordemos que es sesenta y cuatro, no setecientos sesenta y ocho, voy cambiarlo, esos

son pruebas que yo estuve haciendo. Luego, ya tendríamos unos tokens de respuesta, pero nosotros los humanos no entendemos tokens de respuesta, entendemos letras y

palabras. Entonces, lo que tenemos que hacer es descodiar esos toques a la forma en que nosotros lo entendemos, para eso usamos el tokenizer punto

batch de code, le pasamos output generado por el modelo y acá le estamos diciendo de nuevo que este output no va tener tokens especiales.

Ejecutamos esta función, ya la función está definida, eso quiere decir que ya podemos usar nuestro modelo. Es muy sencillo, nada más le decimos inference,

le pasamos el modelo, que sería el modelo que entrenamos, el tokenizer, que es el que es específico para cada modelo, y nuestro prompt. Entonces,

lo ejecutamos, él va a empezar a hacer todo el proceso, y miren, acá nos dice que lo que yo le ingresé es, the force

is strong in you, y lo que él nos da es strong in you, the for the force is, básicamente como nos hablaría Yoda de

Star Wars. Esto es increíble, básicamente acabamos de entrenar un modelo en veinticinco minutos con una herramienta gratuita sin gastar ningún centavo y podemos empezar

jugar mucho con esto. Ustedes pueden usar su conjunto de datos, eso ya depende de ustedes. Lo que vamos a hacer ahora es guardar nuestro

modelo como un adaptador. Si ustedes recuerdan y si nos vamos a Huing Face, vamos a ver que acá tenemos una categoría que se llama

adaptadores. Esos adaptadores hacen referencia a LoRa, ¿por qué? Porque es Low Rang Adaptation, de ahí viene el LoRa. Entonces, guardamos, vamos a darle al

trainer que nos guarde el modelo como Local Fee tres mini yoda adapter, ya nos vamos a ir, tú puedes ir acá y lo vamos

a ver acá, que va ser un montón de archivos, va un montón de cosas que son específicas de Hugging Face, pero con eso ya

nosotros tendríamos guardado nuestro modelo. Ahora nos falta desplegarlo, pero eso lo vamos a ver en la siguiente clase.

Resumen

Optimizar y desplegar modelos entrenados manualmente es posible gracias a los Hugging Face Inference Endpoints, facilitando la integración en servicios y aplicaciones sin complicadas configuraciones técnicas. Aprende paso a paso cómo guardar tu modelo entrenado, subirlo eficientemente a Hugging Face y desplegarlo en la nube para producción.

¿Cómo guardar tu modelo fine-tuned?

Tras entrenar tu modelo es esencial guardarlo correctamente. Utiliza el método `save_model` disponible desde tu objeto `trainer`, especificando claramente el identificador que desees. Esto generará los siguientes archivos clave:

- Configuración sobre adaptadores empleados (Lora).
- Archivos binarios (safe tensors) con los pesos entrenados.
- Configuración relevante para cargarlo posteriormente.

Mantener estos archivos debidamente almacenados posibilitará su uso posterior para inferencia.

¿Cuáles son los pasos para subir tu modelo a Hugging Face?

Hugging Face ofrece simplicidad al subir modelos, actuando similar a un repositorio Git adaptado especialmente para machine learning:

1. Creación de un token en Hugging Face con permisos de escritura.
2. Autenticación desde tu Google Colab usando este token.
3. Subir tu modelo mediante `trainer.push_to_hub()`.

Es recomendable escoger cuidadosamente los permisos del token según el nivel de acceso que desees otorgar, contando con la opción de especificar niveles más detallados si es necesario.

¿Qué es y cómo crear un archivo handler.py para inferencia?

Para desplegar efectivamente en Hugging Face Inference Endpoints debes:

- Generar un archivo handler.py que contenga:
- Inicialización del modelo base y adaptadores.
- Carga del tokenizer para procesar entradas.
- Definir la función `__call__` para manejar solicitudes de inferencia desde la API.

Este archivo permitirá al endpoint gestionar automáticamente solicitudes provenientes del usuario, realizando inferencia y retornando la información requerida de manera rápida y segura.

¿Cómo utilizar Hugging Face Inference Endpoints para despliegue en la nube?

Para desplegar modelos fácilmente debes entrar a "Inference Endpoints" de Hugging Face, indicando:

- Proveedor (AWS recomendado).
- Tipo de máquina adecuada (GPU NVIDIA L4 recomendada para optimización).
- Nivel de seguridad (Protected, Hugging Face Restricted o Public según necesidad).
- Configuración de auto-escalado para control económico.

Al usar esta herramienta se evita lidiar con infraestructura complicada, reduciendo significativamente tiempos de configuración e incrementando tu productividad.

¿Qué consideraciones de costos debes tener presentes?

Es fundamental seleccionar planos que controlen tus gastos:

- Establece un tiempo limitado para desactivar instancias si no reciben solicitudes (se recomienda una ventana de 15 minutos).

- Inicializa únicamente una réplica por defecto para mantener bajo tu gasto inicial.
- Evalúa cuidadosamente el número de solicitudes que tu endpoint atenderá para elegir correctamente el tamaño y tipo de máquina.

Adicionalmente, mediante el dashboard ofrecido por Hugging Face podrás monitorear y pausar tus endpoints para ajustar fácilmente tus costos operativos.

¿Cómo probar rápidamente tu endpoint?

Es posible validar tu endpoint directamente desde el playground disponible, facilitando pruebas rápidas y ajustes inmediatos. También, Hugging Face proporciona opciones detalladas para integrarlo en Python, JavaScript u otros métodos interactivos mediante APIs.

Despliegue de Modelos con Hugging Face Inference Endpoints

1. Guardar el Modelo Fine-Tuned

- Usa `trainer.save_model("nombre_modelo")`
- Archivos generados:
 - Configuración de adaptadores (LoRA)
 - Pesos entrenados (safe tensors)
 - Configuración de carga futura

 Importante: Guardar bien estos archivos asegura la inferencia posterior.

2. Subir el Modelo a Hugging Face

Pasos:

1. Crear token con permisos de escritura en Hugging Face.
2. Autenticarse en Google Colab.
3. Subir modelo: `trainer.push_to_hub()`

 Consejo: Ajusta los permisos del token según el nivel de acceso necesario.

3. Crear handler.py para Inferencia

Debe incluir:

- Carga del modelo y adaptadores
- Inicialización del tokenizer
- Función `__call__` para manejar peticiones

 Función: Permite al endpoint procesar y responder solicitudes automáticamente.

4. Despliegue en Inference Endpoints

Configura en Hugging Face:

- Proveedor: AWS (recomendado)

- Máquina: GPU NVIDIA L4
- Seguridad: Protected, Restricted o Public
- Auto-escalado: Actívalo para ahorrar recursos

⚙️ Beneficio: Sin necesidad de configurar infraestructura compleja.

💰 5. Consideraciones de Costo

Recomendaciones:

- Establece apagado automático (15 min de inactividad)
- Inicia con una sola réplica
- Evalúa tráfico para escoger máquina adecuada

📊 Usa el dashboard para monitorear y pausar endpoints.

🔧 6. Prueba del Endpoint

Métodos:

- Usar el Playground de Hugging Face
- Integrar vía API (Python, JavaScript, etc.)

⚡ Ventaja: Pruebas y ajustes inmediatos desde la interfaz.

Resumen

La cuantización es un concepto crucial en Machine Learning, particularmente cuando se busca optimizar modelos de inteligencia artificial reduciendo los recursos de memoria necesarios. Para implementarlo efectivamente, es vital comprender cómo se almacena y procesa la información en las computadoras actuales.

¿Qué es exactamente la cuantización?

La cuantización busca reducir la cantidad de memoria necesaria para ejecutar modelos de Machine Learning, como los LLM (*Large Language Models*), disminuyendo la precisión en la representación numérica. Esto se logra convirtiendo números en coma flotante (que requieren más memoria) en números enteros (que usan menos memoria).

Imaginemos el modelo LAMA 3.1, que utiliza ocho billones de parámetros con una precisión de 16 bits; ocupa cerca de 16 GB de memoria. La cuantización permite almacenar estos mismos parámetros utilizando menos memoria.

¿Cómo se representan los datos en una computadora?

La información digital se almacena en bits (unos y ceros). La precisión determina cuántos valores diferentes podemos representar con esos bits. Por ejemplo:

- Con 3 bits podemos representar 8 números distintos (del 0 al 7).
- Con 8 bits (byte) podemos almacenar más valores.
- La mayoría de las computadoras actuales usan precisiones estándar de 8 (byte), 16 (short), 32 (int) y 64 bits (long).

Además, las máquinas representan números negativos usando el "complemento a dos". El primer bit indica el signo del número: cero positivo, uno negativo.

¿Qué diferencia hay entre números enteros y números de coma flotante?

Para representar números con decimales, las computadoras utilizan el estándar IEEE 754. Este especifica un sistema formado por:

- Un bit de signo (positivo o negativo).
- Bits para un exponente.
- Bits para una fracción o mantisa.

Sumando estos bits obtenemos la precisión del número de coma flotante total, comúnmente 32 bits.

¿Cómo funciona la cuantización?

La cuantización transforma valores de coma flotante en números enteros y viceversa. Existen principalmente dos tipos de cuantización:

Cuantización asimétrica

Esta utiliza un rango de 0 hasta 255 (para una precisión de 8 bits). Se necesita calcular: - Alfa (valor máximo). - Beta (valor mínimo).

Luego se aplica una fórmula específica que reduce estos valores a números enteros dentro del rango.

Cuantización simétrica

Se basa exclusivamente en alfa, el mayor valor absoluto. Aquí, rango oscila típicamente entre -127 y 127, con el cero exactamente en medio.

Métodos para seleccionar parámetros en la cuantización

Los parámetros alfa y beta pueden seleccionarse usando diversas técnicas que influyen considerablemente en el rendimiento final del modelo cuantizado:

- Percentiles: Reducción del impacto de datos atípicos (outliers).
- MSE (*mean square error*): Minimiza errores a través de fuerza bruta (*grid search*).
- Cross Entropy: Importante especialmente en LLMs, optimizando para mantener intactas las distribuciones de probabilidad.

Quantization Aware Training (QAT), una técnica avanzada

Es un método innovador propuesto por Google DeepMind. En QAT, durante el entrenamiento se simula la cuantización, ayudando al modelo a reducir el error progresivamente. Esto permite que, al culminar el entrenamiento y cuantizar finalmente el modelo, la precisión general permanezca estable.

Utiliza un algoritmo llamado *straight-through estimator (STE)* para manejar el hecho de que la cuantización es una operación no derivable, permitiendo la derivación necesaria para la técnica de *back propagation*.

¿Qué debes cuantizar en una red neuronal?

En redes neuronales comunes como el perceptrón multicapa, cuantizamos: - Matriz de pesos (W). - Bias (b).

Las matrices de entrada (X) se cuantizan "sobre la marcha" (*on the fly quantization*), ya que sus valores cambian en cada consulta del usuario. Sin embargo, la matriz salida (Y) presenta desafíos adicionales y requiere métodos especializados.

La cuantización, bien implementada, permite usar modelos grandes y complejos en hardware más limitado, lo cual tiene enorme importancia práctica.

Perfecto. Aquí tienes un tutorial directo y accionable (solo pasos) para desplegar un modelo fine-tuned con LoRA en Hugging Face Inference Endpoints, incluyendo: crear token, añadir credencial git, subir al Hub y [handler.py](#) para cargar el adaptador LoRA en el endpoint.

0) Requisitos rápidos

- `pip install "huggingface_hub>=0.23" transformers peft git-lfs`

- Tener una cuenta en Hugging Face y un repo de modelo (vacío o creado al subir).
-

1) Crear Access Token (rol write o fine-grained con write sobre tu repo)

1. Ve a Settings → Access Tokens → New token y crea uno con WRITE (o fine-grained con acceso al repo). ([Hugging Face](#))
 2. Conserva el valor `hf_XXXX`.
-

2) Iniciar sesión y guardar credencial git (una sola vez)

```
huggingface-cli login --token $HUGGINGFACE_TOKEN --add-to-git-credential
# Verifica estado
huggingface-cli whoami
```

`--add-to-git-credential` guarda el token en tu helper de git; así podrás `git push` al Hub. ([Hugging Face](#))

3) Subir tu checkpoint al Hub

Opción A (sencilla): subir solo el adaptador LoRA (+ `adapter_config.json`)

Si entrenaste con PEFT y guardaste el adaptador:

```
git lfs install
```

```
git clone https://huggingface.co/<user>/<mi-modelo-lora>
cd <mi-modelo-lora>
# Copia aquí: adapter_config.json, adapter_model.safetensors, README.md, etc.
git add .
git commit -m "LoRA adapter"
git push
```

También puedes usar el CLI upload:

```
huggingface-cli upload <user>/<mi-modelo-lora> ./ruta_local .
```

(El CLI crea el repo si no existe). ([Hugging Face](#))

Opción B (alternativa): mergear LoRA+base y subir modelo completo

(omites **handler.py** y despliegas como task estándar)

```
from peft import AutoPeftModelForCausalLM
from transformers import AutoTokenizer

peft_id = "<user>/<mi-modelo-lora>"
model = AutoPeftModelForCausalLM.from_pretrained(peft_id, torch_dtype="auto",
device_map="auto")
model = model.merge_and_unload()
tok = AutoTokenizer.from_pretrained(model.base_model.model_name_or_path,
use_fast=True)

model.save_pretrained("./merged")
tok.save_pretrained("./merged")
```

Luego sube **./merged** al Hub (git o **huggingface-cli upload**). (Cargar/mergear PEFT para inferencia es la vía recomendada por la propia librería). ([GitHub](#), [Hugging Face Forums](#))

El resto del tutorial se centra en Opción A (adaptador LoRA) con **handler.py** personalizado.

4) Añade **handler.py** (Custom Inference Handler)

Estructura mínima requerida por Inference Endpoints: un archivo **handler.py** en la raíz del repo con la clase **EndpointHandler**(**__init__**, **__call__**). ([Hugging Face](#))

Ejemplo listo para LoRA (Transformers + PEFT):

```
# handler.py
from typing import Any, Dict, List
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftConfig, PeftModel

class EndpointHandler:
    def __init__(self, path: str = ""):
        """
        path: ruta local al repo del modelo en el contenedor del endpoint.
        Debe contener el adaptador LoRA (adapter_config.json + pesos).
        """

        # 1) Leer config PEFT para encontrar el base model
        peft_cfg = PeftConfig.from_pretrained(path)
        base_id = peft_cfg.base_model_name_or_path

        # 2) Cargar tokenizer del modelo base
        self.tokenizer = AutoTokenizer.from_pretrained(base_id, use_fast=True)

        # 3) Cargar modelo base y aplicar adaptador LoRA
        dtype = torch.bfloat16 if torch.cuda.is_available() else torch.float32
        self.model = AutoModelForCausalLM.from_pretrained(
            base_id, torch_dtype=dtype, device_map="auto"
        )
        self.model = PeftModel.from_pretrained(self.model, path)
        self.model.eval()

        # (Opcional) padding side & EOS
        if hasattr(self.tokenizer, "pad_token") and self.tokenizer.pad_token is
None:
            self.tokenizer.pad_token = self.tokenizer.eos_token

    def __call__(self, data: Dict[str, Any]) -> Dict[str, Any]:
        """
```

```

        Espera: {"inputs": "texto de entrada", "max_new_tokens": 128,
"temperature": 0.7, ...}
        Devuelve: {"generated_text": "..."}
        """
        prompt = data.get("inputs") or data
        gen_kwargs = {
            "max_new_tokens": int(data.get("max_new_tokens", 128)),
            "temperature": float(data.get("temperature", 0.7)),
            "top_p": float(data.get("top_p", 0.9)),
            "do_sample": bool(data.get("do_sample", True))
        }

        inputs = self.tokenizer(prompt,
return_tensors="pt").to(self.model.device)
        with torch.no_grad():
            out = self.model.generate(**inputs, **gen_kwargs)
            text = self.tokenizer.decode(out[0], skip_special_tokens=True)
            return {"generated_text": text}

```

Dependencias extra (si las necesitas) en **requirements.txt** (misma raíz):

```

peft>=0.11.0
transformers>=4.41.0

```

Inference Endpoints detecta **handler.py** y usa el modo Task: Custom automáticamente. ([Hugging Face](#))
 Carga de adaptadores LoRA con PEFT/Transformers: ver guías oficiales. ([GitHub](#), [Hugging Face](#))

Sube **handler.py** y **requirements.txt** al repo:

```

git add handler.py requirements.txt
git commit -m "Add custom handler for PEFT LoRA"
git push

```

([Hugging Face](#))

5) Desplegar el Inference Endpoint

1. Ve a Hugging Face → Inference Endpoints → Create Endpoint.
 2. Elige tu repo (<user>/<mi-modelo-lora>), proveedor/region y una instancia (p.ej., L4 si es LLM).
 3. Task aparecerá como Custom (porque tienes `handler.py`).
 4. Deploy. Luego prueba en el Playground o con código/`curl`. ([Hugging Face](#))
-

6) Prueba rápida (desde el panel o vía HTTP)

El panel te da snippets y te pide App Tokens si el endpoint es privado. (Todos los endpoints se crean privados por defecto). ([Hugging Face](#))

Notas útiles

- Si en lugar de adaptadores quieres un endpoint estándar (sin `handler.py`), merging (Sección 3, Opción B) y subir el modelo completo suele simplificar. ([GitHub](#))
- Para repos privados o de organización, usa tokens de acceso con el alcance correcto (recomendados fine-grained en producción). ([Hugging Face](#))

¿Quieres que te genere un repo plantilla con `handler.py`, `requirements.txt`, `README.md` y un workflow de push automático al Hub?

```
import time
```

```
def pausar_y_continuar(mensaje="Presiona Enter para continuar..."):
    """Función para pausar la ejecución y esperar al usuario."""
    input(f"\n{mensaje}")
    print("-" * 60)
```

```
def obtener_numeros_del_usuario():
    """Pide al usuario que ingrese números y los valida."""
```

```

while True:
    entrada = input("➡ Ingresar tus números separados por espacios (ej: 3.14 -2.5 100): ")
    try:
        # Intenta convertir cada elemento ingresado en un número flotante.
        numeros = [float(n) for n in entrada.split()]
        if not numeros: # Si la lista está vacía
            print("❌ Error: No ingresaste ningún número. Inténtalo de nuevo.")
            continue
        return numeros
    except ValueError:
        # Si la conversión falla, es porque hay un valor no numérico.
        print("❌ Error: Asegúrate de ingresar solo números separados por espacios.")

def cuantizar_paso_a_paso(lista_numeros, tipo):
    """
    Función interactiva que guía al usuario a través de la cuantización
    simétrica o asimétrica, explicando cada cálculo.
    """
    if tipo == 'simetrico':
        print("\n" + "="*60)
        print("🟦 TUTORIAL: CUANTIZACIÓN SIMÉTRICA (a int8)")
        print("="*60)
        print("Este método centra el mapa en el cero. Ideal para datos balanceados.")

        pausar_y_continuar("Presiona Enter para encontrar el rango...")

        # --- PASO 1: CALCULAR RANGO ---
        max_absoluto = max(abs(n) for n in lista_numeros)
        print(f"PASO 1: Encontrar el valor más lejano del cero (máximo absoluto).")
        print(f"    - El valor más lejano es: {max_absoluto:.4f}")
        print(f"    - Por lo tanto, nuestro 'mapa' irá de {-max_absoluto:.4f} a {max_absoluto:.4f}")

        pausar_y_continuar("Presiona Enter para calcular la Escala...")

        # --- PASO 2: CALCULAR ESCALA ---
        escala = max_absoluto / 127 if max_absoluto > 0 else 1
        print(f"PASO 2: Calcular la Escala (S).")
        print(f"    - Dividimos el rango real ({max_absoluto:.4f}) entre el rango de enteros (127).")

```

```

print(f"    - Fórmula:  $S = \text{max\_absoluto} / 127$ ")
print(f"    -  Escala (S) = {escala:.6f}")

# En la simétrica, el punto cero es siempre 0.
punto_cero = 0
print(f"\n    - En la cuantización simétrica, el Punto Cero (Z) siempre es
0.")

    pausar_y_continuar("Presiona Enter para ver la tabla de resultados
finales...")

else: # tipo == 'asimetrico'
    print("\n" + "="*60)
    print( TUTORIAL: CUANTIZACIÓN ASIMÉTRICA (a uint8)")
    print("="*60)
    print("Este método crea un mapa a medida para el rango exacto de tus
datos.")

    pausar_y_continuar("Presiona Enter para encontrar el rango...")

    # --- PASO 1: CALCULAR RANGO ---
    min_real = min(lista_numeros)
    max_real = max(lista_numeros)
    print(f"PASO 1: Encontrar el valor mínimo y máximo exacto.")
    print(f"    - Mínimo real: {min_real:.4f}")
    print(f"    - Máximo real: {max_real:.4f}")

    pausar_y_continuar("Presiona Enter para calcular la Escala...")

    # --- PASO 2: CALCULAR ESCALA ---
    rango_real = max_real - min_real
    escala = rango_real / 255 if rango_real != 0 else 1
    print(f"PASO 2: Calcular la Escala (S).")
    print(f"    - Dividimos el tamaño del rango real ({rango_real:.4f}) entre
el de enteros (255).")
    print(f"    - Fórmula:  $S = (\text{max\_real} - \text{min\_real}) / 255$ ")
    print(f"    -  Escala (S) = {escala:.6f}")

    pausar_y_continuar("Presiona Enter para calcular el Punto Cero...")

    # --- PASO 3: CALCULAR PUNTO CERO ---
    punto_cero = round(0 - min_real / escala) if escala != 0 else 0
    print(f"PASO 3: Calcular el Punto Cero (Z).")
    print(f"    - Este valor nos dice a qué entero corresponde el 0.0 del
mundo real.")

```

```

        print(f"    - Fórmula:  $Z = \text{round}(0 - \text{min\_real} / S)$ ")
        print(f"    -  Punto Cero (Z) = {punto_cero}")

    pausar_y_continuar("Presiona Enter para ver la tabla de resultados
    finales...")

    # --- PASO FINAL: MOSTRAR RESULTADOS ---
    print("PASO FINAL: Aplicar las fórmulas a cada número y mostrar los
    resultados.")
    _imprimir_tabla_resultados(lista_numeros, escala, punto_cero, tipo)

def _imprimir_tabla_resultados(lista_numeros, escala, punto_cero, tipo):
    """Función auxiliar para no repetir el código de la tabla de resultados."""
    print(f"\n{'Original':>12} → {'Cuantizado':>12} → {'Recuperado':>12} |
    {'Error':>10}")
    print("-"*60)

    errores = []
    q_min, q_max = (-127, 127) if tipo == 'simetrico' else (0, 255)

    for num in lista_numeros:
        q = round(num / escala) + punto_cero
        q = max(q_min, min(q_max, int(q)))
        recuperado = (q - punto_cero) * escala
        error = abs(num - recuperado)
        errores.append(error)
        print(f"{num:>12.4f} → {q:>12} → {recuperado:>12.4f} | {error:>10.6f}")

    print("-"*60)
    print(f"{'Error promedio':>47} | {sum(errores)/len(errores):>10.6f}")

    print(f"\n Resumen de ahorro:")
    print(f"    • Memoria original (float32): {len(lista_numeros) * 4} bytes")
    print(f"    • Memoria cuantizada (int8): {len(lista_numeros)} bytes")
    print(f"    • Ahorro de memoria: 75%")

# ===== BUCLE PRINCIPAL INTERACTIVO =====
if __name__ == "__main__":
    while True:
        print("\n" + "#"*60)
        print("### TUTORIAL INTERACTIVO DE CUANTIZACIÓN DE NÚMEROS ###")
        print("#"*60)

        numeros = obtener_numeros_del_usuario()

```

```

while True:
    print("\n¿Qué tipo de cuantización quieres ver paso a paso?")
    print("  1. Simétrica (ideal para datos centrados en cero)")
    print("  2. Asimétrica (se adapta a cualquier rango de datos)")
    print("  3. Ambas (para comparar)")
    opcion = input("Elige una opción (1, 2, o 3): ").strip()

    if opcion in ['1', '2', '3']:
        break
    else:
        print("❌ Opción no válida. Por favor, elige 1, 2, o 3.")

    if opcion == '1':
        cuantizar_paso_a_paso(numeros, 'simetrico')
    elif opcion == '2':
        cuantizar_paso_a_paso(numeros, 'asimetrico')
    elif opcion == '3':
        cuantizar_paso_a_paso(numeros, 'simetrico')
        cuantizar_paso_a_paso(numeros, 'asimetrico')

    # Preguntar si quiere volver a empezar
    while True:
        repetir = input("\n¿Quieres probar con otros números? (s/n): ").lower().strip()
        if repetir in ['s', 'n']:
            break
        else:
            print("❌ Opción no válida. Por favor, ingresa 's' para sí o 'n' para no.")

    if repetir == 'n':
        print("\n¡Hasta la próxima!")
        Break

```

CUANTIZACIÓN EN MACHINE LEARNING

¿QUÉ ES?

Técnica que reduce el uso de memoria en modelos de IA al representar números con menor precisión.

 Ejemplo: Modelos como LAMA 3.1 (8B parámetros a 16 bits → 16 GB) → Cuantizados a 8 bits → Menor uso de memoria

¿CÓMO SE REPRESENTAN LOS DATOS EN UNA COMPUTADORA?

- Todo se almacena en bits (unos y ceros).
- La precisión depende del número de bits:
 - 3 bits → 8 valores (0 a 7)

- 8 bits = 1 byte
- Comunes: 8, 16, 32, 64 bits

⚠️ Números negativos → Representados con *complemento a dos* → Primer bit indica el signo (0 positivo, 1 negativo)

1 2 3 4 NÚMEROS ENTEROS VS COMA FLOTANTE

🔴 Enteros → más simples, menos memoria 🔴 Coma flotante (IEEE 754):

- 1 bit de signo
- Bits para exponente
- Bits para mantisa

📊 Total típico: 32 bits

🔄 ¿CÓMO FUNCIONA LA CUANTIZACIÓN?

Convierte números de coma flotante → enteros Objetivo: Reducir uso de memoria sin perder precisión clave

⚙️ TIPOS DE CUANTIZACIÓN

1 Cuantización Asimétrica

- Rango: 0 a 255 (8 bits)
- Usa dos parámetros:
 - Alfa (valor máx.)
 - Beta (valor mín.)
- Fórmula para escalar valores

2 Cuantización Simétrica

- Rango: -127 a 127
- Solo usa Alfa
- Cero al centro

🎯 CÓMO ELEGIR ALFA Y BETA

Métodos más comunes:

- ✓ Percentiles → Reduce impacto de *outliers*
- ✓ MSE (Mean Square Error) → Minimiza error cuadrático medio
- ✓ Cross Entropy → Ideal para LLMs, mantiene distribuciones

🔧 QUANTIZATION AWARE TRAINING (QAT)

🚀 Técnica de Google DeepMind

💡 Simula cuantización durante el entrenamiento

🎯 Beneficio: Modelo aprende con precisión reducida → Al final, rendimiento estable tras cuantización

🔧 Usa el algoritmo STE (Straight-Through Estimator) → Permite hacer *backpropagation* pese a operaciones no derivables

🧩 ¿QUÉ SE CUANTIZA EN UNA RED NEURONAL?

- ♦ Pesos (W)
- ♦ Bías (b)
- ♦ Entradas (X) → cuantizadas *on the fly*
- ♦ Salidas (Y) → más complejas, requieren técnicas especializadas

✓ CONCLUSIÓN

La cuantización es clave para usar modelos grandes en dispositivos con hardware limitado, manteniendo un buen rendimiento.

Responder

Reportar



Mateo Montoya Henao

student•hace un mes

Here's a crisp breakdown of quantization in ML models—why it's a game-changer and how it works! 🚀 1 2 3 4

1. What is Quantization?

Quantization reduces the precision of numbers in a model (e.g., from 32-bit floats → 8-bit integers) to:

- Shrink model size (e.g., 4x smaller).
 - Speed up inference (hardware loves integers!).
 - Cut memory bandwidth (key for edge devices like phones 📱).
- Trade-off:* Tiny loss in accuracy (often negligible!).

2. How It Works

- Step 1: Map float values to integers via scaling. Example: `float[-1.8, 2.1] → int8[-128, 127]` using a *scale factor*.
- Step 2: Perform ops (matrix multiplies) in integer space.
- Step 3: Dequantize outputs back to floats if needed.

3. Types of Quantization

- Post-Training Quantization (PTQ): Convert pretrained models (easy but less accurate).
- Quantization-Aware Training (QAT): Simulate quantization *during* training (better accuracy).
- Hybrid: Mix precision (e.g., weights in 8-bit, activations in 16-bit).

4. Real-World Impact

- Mobile/Edge: Enables real-time AI on tiny devices (e.g., YOLO on drones 🚁).
- LLMs: Lets you run 10B+ parameter models on consumer GPUs. Tools: TensorFlow Lite, PyTorch Quantization, ONNX Runtime.

Durante este curso hemos escuchado muchas veces un término en particular y no hemos hablado tanto de él, y es la cuantización. Pero, ¿qué es

la cuantización? La cuantización lo que apunta es reducir la cantidad de memoria necesaria para ejecutar un modelo de machine learning o LLM. En este

caso, en particular, que se apunta a reducir la cantidad de bits necesarios para guardar esta información. Recordemos que las computadoras guardan la información con

bits, unos y ceros. Y para poner en perspectiva, LAMA tres punto uno, de ocho billones de parámetros, con una precisión de dieciséis bits en

su coma flotante, va a pesar cerca de dieciséis gigabytes de memoria. Eso es bastante, no todas las GPUs de consumo general tienen esa cantidad

de memoria. ¿Y qué es lo que hace la cuantización en particular? La cuantización en particular a lo que apunta es a reducir la precisión

de los números de coma flotante, convirtiéndolo de números de coma flotante a números enteros. Esto mediante un proceso de cuantización, que veremos más en

detalle en esta clase. Antes de llegar al proceso de cuantización, entender cómo se representan los enteros y los números de coma flotantes en una

CPU y una GPU. En las computadoras, números y la información en general está representado con bits, que los bits pueden ser un cero o

un uno, son dos números. Y los números en particular que podemos representar, por ejemplo, si tenemos tres bits de precisión, sería dos a la

n, dos a la tres. En este caso, tres bits podemos representar ocho números diferentes, que pueden ser del cero al siete, por ejemplo. Esta

precisión es la que nos dice qué tanto rango podemos tener en la cantidad de números que podamos usar. Por ejemplo, tres bits podemos

tener del cero al siete, pero tenemos diferentes cantidades de precisión en las computadoras modernas. En ese caso tenemos ocho bits, dieciséis bits, treinta y

dos bits o sesenta y cuatro bits. Cada uno de ellos tiene un nombre que nosotros usamos, por ejemplo, cuando estamos programando en lenguajes de

programación que son tipados, como es para ocho bits el byte, para dieciséis bits el chord, para treinta y dos bits el int y para

sesenta y cuatro bits el long. Notemos que cada uno de estos números es un múltiplo de ocho, ¿y por qué es un múltiplo de

ocho? Porque la mínima unidad de referencia para tener un número o información en una computadora son ocho bits. Adicional, en las computadoras modernas ellos

entienden de una manera particular los signos, y esto se hace usando una término que se llama el complemento de dos. Este complemento lo que

nos dice es que el primer bit de cada secuencia de bits siempre va ser usado para saber el signo que tiene el número, cero

para positivo y uno para negativo. Con esto vamos a tener el rango de menos n bits a más n bits. Esto ya nos da

una idea de cuántos números podemos tener en el rango, y siempre es así para CPUs y GPUs. Pero si tú eres curioso con Python,

por ejemplo, te das cuenta que tú puedes tener números mucho más grandes que sesenta y cuatro bits. Sesenta y cuatro bits es lo común

y lo estándar para las computadoras de hoy en día en precisión en los números, que en este caso sería como dos a la sesenta

y cuatro, que son billones de números. Pero por ejemplo, si tú vas a Python y ejecutas dos a la nueve mil novecientos noventa y

nueve, él va ser capaz de representar este número. Yo te voy dejar un par de recursos en la clase para que me digas y

me dejes tu respuesta a cómo Python es capaz de tener este tipo de precisión tan alta si la computadora solo es capaz de entender

hasta dos a la sesenta y cuatro. Para entender los números de coma flotante, las máquinas, las computadoras usan un estándar a nivel mundial. Este

estándar es el IEEE 754 setecientos cincuenta y cuatro. Pero veámoslo con un ejemplo. Agarremos esta cadena de bits. Esta cadena de bits la

podemos separar con el estándar. El primer bit hace referencia al signo, similar a como lo vimos con los enteros con el complemento dos. Cero

positivo, uno para negativo, en este caso es cero. Luego tenemos el exponente, que en este caso es de ocho bits. El exponente es el

valor que va antes del punto o la coma, dependiendo de cómo escribas los números. En este caso es de ciento veinticuatro por la forma

que tiene el binario, y luego tenemos la fracción mantisa. Esa fracción es de veintitrés bits, y en este caso particular hace referencia a uno

punto veinticinco. ¿Cómo sabemos que es uno punto veinticinco? Por una fórmula, que esa fórmula es que la fracción es igual a uno más uno

por dos a la el número binario, ese número binario que tenemos en en la en este ejemplo particular. Y luego para nosotros saber realmente

cuál es el valor que este número flotante nos está dando, una fórmula. ¿Esta fórmula cuál es? Es uno menos el signo, en este caso

el signo sería cero, que es para positivo, todo número elevado a cero es uno, por dos a la e menos ciento veintisiete, la e

es el valor del exponente, en este caso es ciento veinticuatro menos ciento veintisiete, porque ocho bits uno puede tener números desde cero hasta ciento

veintiséis, entonces ciento veintiséis más un valor adicional que es el cero es ciento veintisiete por uno más la sumatoria del uno al veintitrés, ¿por

qué veintitrés? Porque la mantisa tiene veintitrés bits de el valor que tenemos en la mantisa so por dos a la menos I, que es

la posición del binario. Esto nos va a dar un valor que cuando lo multipliquemos nos va dar el valor que tenemos en este número

de coma flotante, que sería más cero coma quince seis dos cinco. Entonces, este número binario hace referencia al valor numérico positivo cero uno cinco

seis dos cinco, y con esto ya tendríamos la representación de un número de coma flotante como una máquina lo entiende. En este caso, precisión,

que si la queremos saber, sería la fracción y la mantisa tiene veintitrés bits más ocho bits del exponente más un bit del signo, la

precisión es treinta y dos bits. Las GPUs y otros tipos de chips tienen una precisión para los flotantes de dieciséis u ocho bits. Esto

normalmente se hace bajando normalmente la mantisa, pero a su vez hace que tengamos menos precisión en los números. Dependiendo del problema en particular que

tengamos, ¿es bueno o malo? Eso depende mucho de qué problema, qué tanta precisión necesitamos. Por ejemplo, si queremos mandar a un humano a la

Luna y vamos a mandar un cohete, probablemente queramos tener toda la precisión posible, entonces vamos a usar sesenta y cuatro bits o ciento veintiocho,

es por eso digo, depende mucho del problema. Para entender qué es lo que necesitamos cuantizar, un poquito a lo que es una red neuronal

y volvamos a la parte más sencilla. Recordemos cuando teníamos el multilayer perceptron que decíamos, que viene de la función y es igual a x

por w más b . W siendo la matriz de pesos y b siendo el bias, pero x también es una matriz. Entonces, WYV son las

cosas que vamos a querer cuantizar, al igual que y, pero eso tiene un truco particular que vamos a ver más adelante. Pero entonces vamos

a coger estas matrices y vamos a convertirlas de números de coma flotante enteros. Esto se hace principalmente con el proceso de cuantización. Y es

que si nosotros tenemos una matriz con un montón de números con coma flotante, lo pasamos a una cuantización, vamos a tener una matriz ahora

de solo enteros, y cuando lo descuanticemos, vamos a volver a tener una matriz de pesos en coma flotante. Algo particular que queremos que veamos

en este ejemplo, en particular, que veamos cómo en el proceso de cuantización hay un error, y es que eso es muy importante. Cuando nosotros

cogemos un valor de coma flotante, lo convertimos entero y luego lo volvemos a convertir a coma flotante, estamos perdiendo un poco de precisión y

eso va a depender mucho del método que tú uses. Esos métodos que existen actualmente para hacer la cuantización son asimétrico y simétrico. En el

asimétrico lo que vamos a hacer es que el rango donde vamos a bajar la precisión, por ejemplo, de un float de treinta y dos

bits a un entero de ocho bits, va ir de cero hasta dos a la n menos uno. Dos a la ocho menos uno nos

da doscientos cincuenta y cinco. Si nosotros queremos bajar de treinta y dos bits de coma flotante que es bastante precisión hasta este nuevo rango

los números irán de cero a doscientos cincuenta y cinco y luego tenemos el simétrico, el simétrico lo que va hacer es poner el cero

justo en la mitad y el rango va cambiar un poco, Este rango ya no va ir de cero a ciento cincuenta y cinco, sino

desde menos ciento veintisiete a ciento veintisiete. Veamos un poco más a detalle cómo funciona la cuantización asimétrica. Imaginemos que tenemos esta lista de números,

son nada más nueve números, sencillo. Para esta lista lo primero que tenemos que hacer es encontrar un alfa y un beta. El alfa es

el número más grande en esta secuencia de números y el beta es el número más pequeño. En este caso, el número más grande sería

cuarenta y tres punto treinta y uno, ese sería nuestro alfa y el número más pequeño sería menos cuarenta y cuatro punto noventa y tres.

Ese sería nuestro β . ¿Cómo hacemos la cuantización? Para la cuantización, la fórmula particular es x_q , esa q hace referencia que va ser

el cuantizado, es igual a clamp , es una función que ya la vamos a explicar de redondeo entre x_f , que x_f

es el valor que yo quiero cuantizar, sobre s , que s hace referencia a la resta de α menos β sobre dividido dos

a la n , que esa n es la precisión a la que lo queremos bajar, en este caso ocho bits menos uno, eso lo usamos

más z . z hace referencia también a la redonda, al redondeo de menos uno por β sobre s , que ya lo calculamos. La función clamp

es una función muy particular, y es que él va a recibir tres parámetros. El primer parámetro es el parámetro que vamos a usar para

saber a dónde tenemos que llevar la cuantización del valor que queremos cuantizar, el segundo parámetro es un límite inferior y el tercer parámetro es

un límite superior. Entonces, si el valor que nosotros queremos cuantizar después de aplicar esta operación matemática es menor a cero, la función clamp lo

que va hacer es truncarlo a cero, y si es superior a dos a la n menos uno, lo va a truncar a dos a

la n menos uno. Si el valor está entre ellos dos, lo que va hacer es usar el valor calculado. Entonces acá, por ejemplo, vemos

que esta serie de números se cuantizó a unos números enteros. Algo particular y que nos vamos a dar cuenta es que α siempre va

ser el número superior, el número mayor más grande que pueda haber dentro del rango, que en este caso sería doscientos cincuenta y cinco, y

β al ser el número menor, va ser el número menor del rango, que en este caso sería cero. El resto de números son las

representaciones intermedias de los números de coma flotante en en este rango particular. Y para hacer el proceso de descuantización es muy sencillo, simplemente definimos

x_f , que sería el valor descuantizado, va ser igual a s multiplicado por la resta entre el valor cuantizado menos z , y eso

nos va dar una lista nueva de valores. Algo que quiero que veamos con dos números particulares con alfa y con beta, es que vamos

a ver que alfa, su valor original era cuarenta y tres punto treinta y uno, pero luego de cuantizarlo y luego descuantizarlo, ahora el valor

cambió a cuarenta y tres punto veinticinco, y algo similar pasó con beta, que pasó de menos cuarenta y cuatro punto noventa y tres a

menos cuarenta y cuatro punto noventa y nueve. Ese es el error que estamos introduciendo nosotros en la cuantización y es muy importante, porque existen

técnicas para reducirlo que vamos a ver más adelante en esta clase. Ahora la cuantización simétrica. Es bastante similar a la cuantización asimétrica. La única

diferencia, un par de ellas, es que ya no vamos a necesitar un alfa y un beta, solo vamos a necesitar un alfa que va

ser el valor más grande en valor absoluto. Entonces, usando el mismo, la misma lista de datos, vamos a darnos cuenta de que el valor

absoluto más grande es menos cuarenta y cuatro punto noventa y tres, porque cuando aplicamos la función de absoluto nos vamos dar cuenta que es

cuarenta y cuatro punto noventa y tres, que es el valor más grande de todos. Y la función que define la función simétrica de cuantizaciones

$x_{sub q}$, de nuevo, es igual a $clam$, que ya lo explicamos, pero en este caso los valores dentro de los argumentos son un

poco diferentes. Vamos a tener el redondeo entre $x_{sub f}$ sobre s y no más, no le vamos a sumar z , ya no tenemos

un cero. ¿Por qué? Porque el cero está implícito en la simetría de el rango. Y el valor uno y el valor dos van a

ser lo del rango inicial y el rango final, en este caso menos ciento veintisiete a ciento veintisiete. La s cambia un poco, la división

es sobre el mismo valor $2n$ a la menos uno, menos uno, pero el valor arriba va ser el valor absoluto del número más

grande, simple. Y vamos a tener lo mismo, vamos a tener la lista, lo cuantizamos, acá nos vamos a dar cuenta de que el cero

se mantiene en el cero, o no cambia, el valor más grande en valor absoluto va ser menos ciento veintisiete, y en el caso de

los positivos es ciento veintidós, porque no hay ningún valor idéntico en los negativos de la serie original. Y la descuantización es súper sencilla, es

$x_{sub f}$ es igual a s , que es el que tenemos calculado ya, por $x_{sub q}$, que es el valor que cuantizamos. Y

de nuevo, acá estamos introduciendo un error. Algo particular en la simétrica es que pueden haber casos en donde el valor absoluto superior sea el

mismo y no haya un error, pero eso solo va pasar con ese valor en particular. Este método de cuantización sirve mucho para los valores

estáticos. ¿Cuáles son esos valores estáticos? Los valores que ya por defecto conocemos, que son los que entrenamos, la w y la b , el y

el y y el bias. Estos valores, como ya los conocemos, podemos hacer una cuantización antes de poner el modelo en inferencia. Pero, ¿qué pasa con

el input, con la x ? Esa siempre va cambiar, depende de lo que el usuario esté preguntándole. Recordemos que ese input luego se transforma en

un vector, que es una lista de números, entonces ahí aplicamos algo que se llama on-fly quantization. Es un proceso súper rápido, ya que el

el valor de entrada es mucho menor que la matriz de pesos y la matriz de bias, entonces lo podemos hacer on-fly. Pero, ¿qué pasa

con la y ? Porque la y es el valor de multiplicar el input del usuario más la w , que serían los pesos, más el bias,

y ahí es donde entra algo particular. Y es que no lo podemos saber, realmente no sabemos cuáles son sus valores de alfa, o cero

para poder calcular la cuantización. Entonces, ahí entran un par de técnicas un poco más complicadas que vamos a ver al final de esta

clase, que quiero contarte, porque son las técnicas que se están usando a día de hoy para mejorar el entrenamiento de los modelos y que

sean menos susceptibles a los errores en la cuantización. Bueno, ya entendimos cómo funciona simétrico y simétrico, pero ¿cómo selecciono ese alfa y ese beta?

Ya vimos una forma, que es el mínimo y el máximo, pero esta forma tiene un gran problema, es que es muy sensible a outliers.

Eso quiere decir que si tenemos datos que se salen mucho del rango, estos datos van a ser cuantizados y van a hacer que nuestros

datos se cuanticen con un mayor error. Por eso, otros métodos. Uno de ellos es el método de los percentiles. Esto lo que nos va

ayudar es que esos outliers, esos datos que se salen mucho de la distribución, sean los únicos en particular que se cuantizan con un error

gigante. Esto lo que nos ayuda es que el error en general de la lista cuantizada se minimice. Recordemos que los LLMs y en general

cualquier modelo de machine learning, aprende a partir de distribuciones de datos. Entonces, tenemos que ser muy cuidadosos con estas distribuciones de datos que tienen

outliers. Para eso, yo recomiendo particularmente usar percentiles en vez de mínimos o máximos, como acabé de explicar. Existen otras dos formas de seleccionar estos

hiperparámetros. ¿Por qué son hiperparámetros? Porque realmente nosotros controlamos cuáles son los valores que seleccionamos. El primero es usando el MSE. Este error ya lo

hemos visto, MSE es mean square error. Ya lo hemos visto cuando estábamos hablando del multilayer perceptron y de la forma en seleccionar esa métrica

de pérdida. Básicamente lo que vamos a hacer es probar un montón de valores, una fuerza bruta. Este método en particular en machine learning se

llama grid search y es bueno, pero no es tan recomendado, depende de cuánto cómputo tú tienes disponible para seleccionar estos valores. Esto lo único

que hace es validar el error de los valores cuantizados y descuantizados y tratar de minimizarlo usando diferentes valores de alfa y beta. La otra

forma, que es un poco más tricky y que se usa mucho para los LLMs, es con cross entropy. Si recordamos, en la arquitectura de

los LLMs, la última capa, a nosotros lo que nos importa es la distribución. ¿Por qué? Porque nosotros vamos a seleccionar el valor o el

token o la palabra que mayor valor tiene dentro de la distribución, entonces nosotros en particular no nos importa cuál es el número, nos importa

que el primero siempre se mantenga primero, Ahí es cuando usamos cross entropy con softmax, entonces básicamente lo que hace es él no se le

va a interesar realmente cuál es el valor, sino va a optimizar para que la distribución se mantenga igual. Esto es que nos ayuda es

a reducir también la cantidad errores y es particularmente usada y yo creo que realmente solo se usa en los LLMs. Entonces, tengámosla en cuenta

porque es la que vamos a ver mucho referenciada en los papers que veamos hablando sobre cuantización en LLMs. Algo increíble en la comunidad de

investigación de los LLMs es que poco a poco y no tan poco a poco, a veces es muy rápido semana a semana, se van

agregando nuevas técnicas para reducir el error y a su vez reducir el tamaño y la cantidad de cómputo que yo necesito en la cuantización

o en la inferencia. Hace poco el equipo de Google, Google Did Mine, sacó GEMA tres QAT. Este QAT hace referencia a Quantization Aware Training,

y es que esta gente hizo algo increíble, algo que yo digo, ahí es donde están los realmente optimizaciones a la hora de entrenar un

LLM. Y es que esta gente al momento de hacer el entrenamiento simuló que estaba haciendo cuantización en cada una de las capas para que

la función de pérdida fuera capaz de detectar esos mínimos errores y pudiera modificar los pesos basado en ese error. Y esto hace que el

momento de que cuando ya tengo el modelo entrenado, yo lo pueda cuantizar y el error y la pérdida y el accuracy no sea tan

alto. Recordemos que como estamos agregando un error en cada uno de los pasos, puede ser que la precisión al momento de usar el modelo

completo y el modelo cuantizado tenga una ligera desventaja, pero estos métodos de quantization a word training lo que hace es tener el modelo bien

robusto para que incluso si tú le aplicas una cuantización, no pierdas nada de tu precisión y el modelo siga funcionando en óptimas condiciones. Y

acá quiero contarte algo un poquito técnico, y es que realmente las funciones que nosotros usamos a la hora de cuantizar son no derivables, Y

si nosotros recordamos el proceso de entrenamiento de un modelo de machine learning y de un LLM, tenemos el proceso de back propagation. Y si

recuerdas, hablábamos de que te acaba derivar. Derivar, ¿por qué? Porque queríamos minimizar el error. Y si estas funciones no son derivables, entonces, ¿cómo lo

hacen en el quantization aware training? Lo hacen de una manera muy inteligente. Usando un algoritmo que se llama straight through estimator, STE, lo que

hacen es decir, ok, si el valor que estamos tratando de derivar se sale del rango de cuantización, vamos llevar a uno, y si no,

vamos a llevar a cero. Esto hace que podamos simular esa derivada, incluso si la función que estamos usando es no derivable. Pero, ¿por qué

funciona el QAT realmente? Y es que, si nosotros recordamos, al final, lo que nosotros queremos que haga cualquier modelo de machine learning es que

minimice el error, minimice la pérdida, o que haga el trabajo que nosotros queramos que haga con la menor cantidad de errores. Entonces, imaginemos que

tenemos una gráfica donde tenemos dos valles, un valle más bajo que el otro. Ese valle es donde nosotros llegaríamos sin un entrenamiento que esté

aware, que esté al pendiente de que se va a cuantizar el modelo, y eso es bueno, porque eso va ser que al tener una

menor pérdida, modelo va ser más preciso. Pero lo que pasa es que cuando nosotros cuantizamos el modelo, vamos a hacer que la cuantización, que

el error que estamos introduciendo dentro de nuestras funciones, haga que ese mínimo ya no esté en el mínimo mínimo de la función, sino que

suba más, y ese que suba más va a ser que la precisión del modelo baje, o sea, es lo que no queremos. Lo que

hace quantization aware training es que en vez de buscar el mínimo global de la función, vamos a ir a un punto intermedio la cuantización

pueda manejar de una manera más sencilla para que no quedemos ni tan arriba ni tan abajo, sino en un punto medio. Y ahí es

donde está lo más importante de la investigación actualmente. ¿Cómo hacemos de que ya no tengamos que ir un mínimo local, sino que podamos ir

al mínimo global de la función sin perder tanta precisión al momento de la cuantización. Ya con esto tienes las bases fundamentales y teóricas para

entender cómo funciona la cuantización, pero no la hemos implementado, y es que yo te quiero dejar un reto. Quiero que dentro de los comentarios de la clase me dejes un script sencillo, lo más fácil que lo puedas hacer, la cuantización asimétrica y simétrica. Escoge cuál quieres implementar, que coja cualquier lista de números que yo le pueda pasar, una lista de Python y me los cuantice. Déjalo en los comentarios, estaré leyendo tu código y dejándote feedback ahí.

En el mundo de los modelos de lenguaje LLM, elegir el más adecuado puede parecer complicado por la cantidad de alternativas existentes. Modelos como OpenAI, Gemini de Google, Cloud de Anthropic o Mistral presentan cada uno ventajas específicas en diferentes circunstancias y tareas.

Los benchmarks son pruebas especializadas para medir la eficiencia y precisión de los modelos en diferentes tareas. Al evaluar estos benchmarks puedes decidir objetivamente cuál modelo se adapta mejor a tus necesidades.

- Aime 2024/2025: Evaluación de matemáticas olímpicas donde, por ejemplo, el modelo O3 mini alcanzó una precisión de 86.3%, superando generalmente el rendimiento humano.
- Frontier Math: Más desafiante, con una dificultad alta que ningún modelo ha superado aún el 20% al momento actual.
- Humanity Last Exam aborda múltiples áreas incluyendo biología, química, física y escritura. A la fecha ningún modelo ha superado el 21% en precisión.
- GPQA Diamond: Basado en respuestas expertas humanas en diversas materias, Gemini 2.5 Pro ha sobresalido con una precisión superior al 80%.

Para evaluar habilidades de codificación existen pruebas como: - SWE Bench, que posiciona modelos como Cloud 3.7 Sonet como líderes en programación.

- AGI (Astrap Resonant Corpus) examina si un modelo puede razonar como un humano, evaluando además el costo asociado en términos monetarios a dicha precisión.

El Chatbot Arena permite a usuarios evaluar subjetivamente y de manera anónima los distintos modelos, evitando sesgos por marketing o preferencias personales. Este método ofrece una perspectiva más equilibrada y práctica sobre su rendimiento real en el día a día.

Los modelos de razonamiento, como O1, O3, O4 o Gemini 2.5, simulan nuestro método complejo de razonar, ideales para tareas difíciles y razonamientos extensos, aunque implican mayores costos operativos. Por otro lado, los modelos estándar son menos complejos pero más económicos.

Te invito a que explores continuamente cada modelo y utilices estas herramientas y benchmarks especializados para tomar decisiones informadas acorde con tus proyectos y necesidades profesionales. Recuerda seguir las actualizaciones de los principales laboratorios de desarrollo (OpenAI, Google DeepMind, Anthropic) pues cada actualización puede influir significativamente en tu trabajo.

Evaluación de Modelos de Lenguaje (LLM)

¿Cómo elegir el mejor LLM?

- Existen múltiples modelos: OpenAI, Gemini (Google), Claude (Anthropic), Mistral
- Cada uno tiene fortalezas en tareas distintas.

¿Qué son los benchmarks?

- Pruebas para medir rendimiento y precisión en tareas específicas.
- Ayudan a comparar modelos de forma objetiva.

Tipos de Benchmarks y Ejemplos

Matemáticas y Lógica

- AIME 2024/2025 > Mide nivel olímpico en matemáticas > O3 mini alcanzó 86.3% de precisión
- Frontier Math > Nivel de dificultad muy alto > Ningún modelo supera el 20%

Conocimientos Generales

- Humanity Last Exam > Incluye biología, química, física, escritura > Precisión máxima actual: 21%
- GPQA Diamond > Basado en expertos humanos > Gemini 2.5 Pro logró más del 80%

Programación y Lógica

- SWE Bench > Evalúa habilidades de codificación > Claude 3.7 Sonnet se destaca

Razonamiento Global

- AGI (Astrap Resonant Corpus) > Evalúa razonamiento humano y costos operativos

Evaluación práctica: Chatbot Arena

- Pruebas anónimas y sin sesgos de marketing
- Usuarios eligen el modelo más convincente en cada conversación
- Ofrece una perspectiva realista y práctica

¿Modelos de razonamiento o modelos estándar?

Modelos de Razonamiento

- Ej: 01, 03, 04, Gemini 2.5
- Simulan pensamiento humano
- Útiles para tareas complejas
- Más costosos

Modelos Estándar

- Menor complejidad y costo
- Eficientes para tareas simples

Perfecto – hice un análisis global del chat y de las cuatro transcripciones que subiste (los `.vtt` convertidos a `transcripcion_completa*.txt`) y acá tienes un resumen estructurado + ampliación técnica para profundizar en fundamentos de LLMs. Voy directo al grano y con matemáticas donde ayuda.

Resumen ejecutivo (qué hay en tus transcripciones y el chat)

- Material repetido y complementario sobre tokenización, embeddings, espacio vectorial, Transformers y atención (QKV + softmax).

- Implementación práctica: RoPE (Rotary Positional Embeddings) y parche para GPT-2 (código PyTorch/HF).
 - Fine-tuning y parámetros eficientes: LoRA / PEFT, guardado de adaptadores, merging, subida a Hugging Face y `handler.py` para Endpoints.
 - Operativa de despliegue: crear token HF, añadir credenciales git, `git lfs`, deploy a Inference Endpoints.
 - Menciones prácticas: quantización, cache KV, Docker / n8n problemas, webhooks, y detalles de infra (GPU, memoria).
 - Extras: prácticas de evaluación, RAG (retrieval + embeddings + FAISS), contrastive sentence embeddings (SBERT / DPR / ColBERT) y manejos de índices ANN.
-

Resumen técnico detallado (por tema)

1) Tokenización y representación inicial

- Token: unidad discreta (wordpiece/BPE/unigram/subword). Tokenizer mapea texto $\rightarrow$ ids.
- One-hot $\rightarrow$ embedding lookup:

$$x \in \{0,1\}^V; E \in \mathbb{R}^{V \times d}; h_0 = x^T E = E[t] \quad x \in \{0,1\}^V; \quad E \in \mathbb{R}^{V \times d}; \quad h_0 = x^T E = E[t]$$

$$V \text{ vocab size, } d \text{ dim embedding.}$$
- Elección del tokenizer afecta longitud de secuencia, OOV y eficiencia.

2) Embeddings y propiedades geométricas

- Embedding iniciales se aprenden durante entrenamiento; en métodos estáticos (Word2Vec/GloVe) se optimizan con objetivos de co-ocurrencia o skip-gram.
- Similitud coseno y producto punto:

$$\cos(u,v) = \frac{u \cdot v}{\|u\| \|v\|} \quad \cos(u,v) = \frac{u \cdot v}{\|u\| \|v\|}$$

Normalizar (L2) antes de búsqueda mejora estabilidad.

- Anisotropía: embeddings tienden a agruparse en direcciones dominantes; práctica común: centrar y quitar primeras PCs (“all-but-the-top”).

3) Transformer – atención y proyectores

- Proyecciones lineales:

$$Q = XW_Q, K = XW_K, V = XW_V \quad Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

- Scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Multi-head: divide dims en h cabezas, cada una con $\text{head_dim} = d_{\text{model}}/h$.

- Pérdida de entrenamiento autoregresivo (Cross-Entropy):

$$L = -\sum_t \log p_\theta(y_t | y_{<t}) \quad \mathcal{L} = -\sum_t \log p_\theta(y_t | y_{<t})$$

con p_θ obtenido tras softmax sobre logits.

4) Positional Encodings y RoPE (Rotary)

- RoPE aplica rotación por pares de dimensiones dependiendo de la posición p :

- Frecuencias: $\omega_i = 1/\text{base}^{2i/d}$

- Ángulo: $\theta_{p,i} = p \cdot \omega_i$

- Aplicación en Q/K :

$$q' = q \odot \cos \theta + \text{rot}(q) \odot \sin \theta$$

donde rot intercambia y cambia signo por pares:

$$(x_{2i}, x_{2i+1}) \mapsto (-x_{2i+1}, x_{2i}) \quad (x_{2i}, x_{2i+1}) \mapsto (-x_{2i+1}, x_{2i})$$

- Ventaja: codifica posición multiplicativamente y funciona bien con caching incremental (generación autoregresiva).

5) Contextualización vs embeddings estáticos

- Modelos como BERT/GPT producen representación contextual por ocurrencia (mismo token distinto contexto → vectores distintos).
- Para retrieval se prefieren encoders siamés o contrastivos (SBERT, DPR) para obtener vectores de oraciones eficientes.

6) Fine-tuning eficiente: LoRA / PEFT

- LoRA: inserta matrices de bajo rango A, B en proyecciones (por ejemplo, en WQ, W_V o WQ, W_V), de modo que se aprende solo A y B en lugar de todo W :

$$W' = W + \Delta W, \Delta W = BA$$

$$W' = W + \Delta W, \Delta W = BA$$
con $B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times d}$ (r pequeño).
- Permite guardar solo adaptadores (pocos MB) en vez de todo el modelo (GBs).

7) Inferencia a escala y RAG (retrieval-augmented generation)

- Workflow RAG: query → encoder → ANN search (FAISS/HNSW) → retrieve top-k pasajes → concat/contextual prompt → generation.
- Opciones retrieval:
 - DPR / bi-encoder (rápido)
 - ColBERT / late interaction (mejor precisión, coste mayor)
- Índices ANN: IVF+PQ, HNSW; parámetros: nlist, nprobe, M (PQ), efSearch/efConstruction.

8) Optimización de inferencia

- Quantización (8-bit, 4-bit, q4_0/q4_K): reduce memoria/latencia con pérdida mínima si se hace bien.
- Pruning y distillation: reducir tamaño con pérdida de calidad.

- Caching KV: crítico para generación eficiente (evitar recomputar capas anteriores).
- Hardware: GPU VRAM y ancho de memoria son limitantes; modelos grandes requieren sharding o inference providers (HF Endpoints).

9) Despliegue (lo que detecté en tus archivos)

- Pasos prácticos: crear token HF, configurar credenciales git, `git lfs`, subir adaptador o modelo mergeado, añadir `handler.py` con carga de base model + `PeftModel.from_pretrained` para LoRA, crear Inference Endpoint y configurar instancia (L4, L5, etc.), probar con playground o API.
- Aspectos de producción: escalado automático, seguridad del endpoint, coste y políticas de apagado para ahorrar.

Fórmulas clave (resumen rápido)

1. Embedding lookup: $h_{\theta} = E[t]$ $h_{-\theta} = E[t]$.
 2. Q/K/V: $Q = XW_Q$ $Q = XW_Q$, etc.
 3. Attention: $\text{softmax}(QK^Tdk)V$ $\text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$.
 4. RoPE: $q' = q\cos\theta + \text{rot}(q)\sin\theta$ $q' = q\cos\theta + \text{rot}(q)\sin\theta$.
 5. Cross-entropy loss (autoregresivo): $-\sum_t \log p(y_t | y_{<t}) - \sum_t \log p(y_t | y_{<t})$.
-

Recomendaciones prácticas y ejercicios (para afianzar)

1. Experimentar anisotropía

- Carga embeddings (p. ej. texto entrenado), calcula media y primeras 5 PCs.
- Quita media y top-k PCs; compara distribución de cosenos antes/después.

2. Probar RoPE en mini-transformer

- Implementa RoPE simple y verifica generación incremental: generar 1 token a la vez con `past_key_values` y asegúrate que `seq_start` = longitud pasada.

3. LoRA vs merge test

- Fine-tunea con LoRA; despliega adaptador y, en paralelo, mergea y despliega modelo completo; compara latencias y calidad en prompts específicos.

4. RAG pipeline pequeño

- Construye un pequeño corpus (1k docs), indexa con FAISS HNSW, crea un bi-encoder (SBERT) y monta retrieval + generator; mide mejora en respuestas.

5. Quantización

- Prueba q8/q4 con bitsandbytes (o herramientas similares) y mide pérdida de perplexity en una muestra de validación.

Buenas prácticas (resumen práctico)

- Normaliza embeddings para búsqueda (L2).
- Q/K head_dim debe ser par para RoPE.
- Cuando uses LoRA para producción: testea merged vs adapter-only (latencia y coste).

- En endpoints, añade `handler.py` y `requirements.txt`, y prueba localmente con `docker run` si es posible antes de deploy.
 - Monitorea uso/latencia y configura políticas de "spin down" para ahorrar coste.
-

Lecturas/papers esenciales (mínimo recomendable)

- Vaswani et al., Attention is All You Need (2017).
 - Radford et al., GPT series / Brown et al., GPT-3 (2020) – para entender scaling.
 - Su et al., RoFormer / Rotary Positional Embedding (introducción RoPE).
 - Hu et al., LoRA (Parameter-Efficient Fine-Tuning).
 - Papers sobre contrastive sentence embeddings (SBERT / DPR) y ColBERT (late interaction).
(estos títulos son estables y fáciles de localizar en repositorios académicos y Hugging Face docs).
-

Concretos extra – snippets y checks rápidos

Chequeo RoPE en generación incremental: siempre pasar `seq_start = past_len` al construir cos/sin; si no, posiciones se repetirán y la generación se corrompe.

Verificación LoRA en endpoint: en `handler.py` haz `print(peft_config.base_model_name_or_path)` al iniciar para validar que la ruta del base model es la esperada.

Si quieres, hago alguna de estas tres acciones ahora (elige una, no preguntes – ya tengo contexto):

- A) Crear un resumen de 1 página (auxtable) con las variables clave y rangos típicos (head_dim, d_model, seq_len, r en LoRA, quant bits).
- B) Generar un script reproducible que: carga base GPT2, aplica RoPE (patch), carga un LoRA adapter desde carpeta y genera sample (lista para correr).
- C) Preparar un documento técnico (3-4 páginas, estilo profesor) que combine teoría (fórmulas), pipeline de entrenamiento y checklist para deploy (handler.py + HF steps).

Dime A, B o C y lo genero ya (no necesito más datos).